

Curso Profesional de Entity Framework Core 8

Módulo 5 — Prácticas: Persistencia empresarial, concurrencia, transacciones, despliegue, testing y buenas prácticas

Autor: JAIME GALLO

Cada práctica trabaja sobre un estado completo y ejecutable de AceriaData. La secuencia es acumulativa desde M04/PROYECTO/4.12 : cada punto conserva todo lo incorporado anteriormente y añade únicamente lo necesario para el nuevo contenido.

Entorno de trabajo: .NET 8 · C# 12 · Entity Framework Core 8 · SQL Server Express LocalDB · Visual Studio Community · PowerShell para los scripts de despliegue.

Punto 5.1 — Concurrencia optimista: concepto y necesidad

Contexto del proyecto

Este punto continúa directamente M04/PROYECTO/4.12. La carpeta M05/PROYECTO/5.1 contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo no cambia todavía; se conserva la historia heredada del Módulo 4.

Objetivo práctico

Reproducir una actualización perdida real sobre la misma propiedad y contrastarla con cambios concurrentes sobre propiedades distintas.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.1
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/ConcurrenciaOptimistaM5Repositorio.cs`
- `src/AceriaData.Application/ConcurrenciaOptimistaM5UseCase.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar `ConcurrenciaOptimistaM5Repositorio.cs`

```
public ConcurrenciaMismaPropiedadDto DemostrarActualizacionPerdidaMismaPropiedad()
{
    RestaurarEstadoInicial();

    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();

    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    SqlCommandCounterInterceptor.Instance.Reset();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);

    var clienteInicial = ordenA.Cliente;

    ordenA.Cliente = "Cliente actualizado por A";
    contextA.SaveChanges();

    ordenB.Cliente = "Cliente actualizado por B";
    contextB.SaveChanges();

    using var scopeVerificacion = _scopeFactory.CreateScope();
    var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var clienteFinal = contextVerificacion.OrdenesFabricacion
        .AsNoTracking()
```

```

        .Where(o => o.NumeroOrden == NumeroOrden)
        .Select(o => o.Cliente)
        .Single();

    return new ConcurrencyMismaPropiedadDto(
        clienteInicial,
        ordenA.Cliente,
        ordenB.Cliente,
        clienteFinal,
        clienteFinal == ordenB.Cliente && clienteFinal != ordenA.Cliente,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

Explicación línea a línea — ConcurrencyOptimistaM5Repositorio.cs

Línea 1: `public ConcurrencyMismaPropiedadDto DemostrarActualizacionPerdidaMismaPropiedad()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarEstadoInicial();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 5: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 8: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 9: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 11: `SqlCommandCounterInterceptor.Instance.Reset();` → Reinicia el contador para que solo se midan los comandos de este escenario.

Línea 13: `var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 14: `var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 16: `var clienteInicial = ordenA.Cliente;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 18: `ordenA.Cliente = "Cliente actualizado por A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 21: `ordenB.Cliente = "Cliente actualizado por B";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 22: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 24: `using var scopeVerificacion = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 25: `var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 26: `var clienteFinal = contextVerificacion.OrdenesFabricacion` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 27: `.AsNoTracking()` → Ejecuta una lectura sin dejar entidades en el ChangeTracker.

Línea 28: `.Where(o => o.NumeroOrden == NumeroOrden)` → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.

Línea 29: `.Select(o => o.Cliente)` → Proyecta únicamente la forma de datos necesaria para el resultado.

Línea 30: `.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 32: `return new ConcurrenciaMismaPropiedadDto()` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 33: `clienteInicial,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 34: `ordenA.Cliente,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 35: `ordenB.Cliente,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 36: `clienteFinal,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 37: `clienteFinal == ordenB.Cliente && clienteFinal != ordenA.Cliente,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 38: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 39: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `ConcurrenciaOptimistaM5Repositorio.cs`

```
public ConcurrenciaPropiedadesDistintasDto DemostrarCambiosEnPropiedadesDistintas()
{
    RestaurarEstadoInicial();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);

    ordenA.Cliente = "Cliente actualizado por A";
    contextA.SaveChanges();
    ordenB.Estado = "EnProceso";
    contextB.SaveChanges();

    using var scopeVerificacion = _scopeFactory.CreateScope();
    var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var resultado = contextVerificacion.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.NumeroOrden == NumeroOrden)
        .Select(o => new { o.Cliente, o.Estado })
        .Single();

    return new ConcurrenciaPropiedadesDistintasDto(
        ClienteInicial, EstadoInicial, resultado.Cliente, resultado.Estado,
        resultado.Cliente == ordenA.Cliente && resultado.Estado == ordenB.Estado,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — ConcurrenciaOptimistaM5Repositorio.cs

Línea 1: `public ConcurrenciaPropiedadesDistintasDto DemostrarCambiosEnPropiedadesDistintas()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarEstadoInicial();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 7: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 9: `var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 10: `var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `ordenA.Cliente = "Cliente actualizado por A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 13: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 14: `ordenB.Estado = "EnProceso";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 17: `using var scopeVerificacion = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 18: `var contextVerificacion = scopeVerificacion.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 19: `var resultado = contextVerificacion.OrdenesFabricacion` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 20: `.AsNoTracking()` → Ejecuta una lectura sin dejar entidades en el `ChangeTracker`.

Línea 21: `.Where(o => o.NumeroOrden == NumeroOrden)` → Añade el predicado a la consulta; EF Core intentará traducirlo al proveedor antes de materializar.

Línea 22: `.Select(o => new { o.Cliente, o.Estado })` → Proyecta únicamente la forma de datos necesaria para el resultado.

Línea 23: `.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 25: `return new ConcurrenciaPropiedadesDistintasDto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 26: `ClienteInicial, EstadoInicial, resultado.Cliente, resultado.Estado,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 27: `resultado.Cliente == ordenA.Cliente && resultado.Estado == ordenB.Estado`, → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 28: `SqlCommandCounterInterceptor.Instance.SnapshotCommands()`; → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 29: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.1 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Dos DbContext independientes cargan la misma orden antes de guardar.
- La misma propiedad permite observar una actualización perdida sin token.
- Propiedades distintas se conservan cuando EF Core genera UPDATE de las propiedades modificadas.
- La verificación final usa un tercer contexto AsNoTracking.
- El SQL mostrado procede del interceptor real.

Reto resuelto y ampliación

Reto: Añade una tercera escritura concurrente sobre Cliente y comprueba, mediante el SQL capturado, qué valor queda finalmente sin token.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Usar el mismo DbContext para ambos actores y creer que se reproduce concurrencia.
- Modificar propiedades distintas y afirmar que necesariamente existe actualización perdida.
- Leer el resultado final desde un contexto con estado antiguo en el ChangeTracker.
- Inventar un SQL esperado en vez de observar el comando ejecutado.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.1 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.1, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.1 0K

Conexión con el siguiente punto

El siguiente estado es 5.2 y parte directamente de este proyecto completo.

Punto 5.2 — Configuración de tokens de concurrencia

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.1. La carpeta M05/PROYECTO/5.2 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. Este punto introduce la migración real M5_5_2_ConcurrencyTokens.

Objetivo práctico

Configurar rowversion y un token de propiedad, generar la migración real y demostrar el conflicto sobre SQL Server LocalDB.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.2
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Persistence/Configurations/ConcurrencyTokensConfiguration.cs
- src/AceriaData.Infrastructure/Repositories/TokensConcurrenciaM5Repositorio.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar ConcurrencyTokensConfiguration.cs

```
public sealed class OrdenFabricacionConcurrencyConfiguration :
IEntityTypeConfiguration<OrdenFabricacion>
{
    public void Configure(EntityTypeBuilder<OrdenFabricacion> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class PlanchaAceroConcurrencyConfiguration : IEntityTypeConfiguration<PlanchaAcero>
{
    public void Configure(EntityTypeBuilder<PlanchaAcero> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class AleacionConcurrencyConfiguration : IEntityTypeConfiguration<Aleacion>
{
    public void Configure(EntityTypeBuilder<Aleacion> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class DetalleOrdenConcurrencyConfiguration : IEntityTypeConfiguration<DetalleOrden>
{
    public void Configure(EntityTypeBuilder<DetalleOrden> b)
    {
        b.Property(x => x.EstadoDetalle)
            .IsRequired()
            .HasMaxLength(50)
            .HasDefaultValue("Pendiente")
            .IsConcurrencyToken();
    }
}
```

Explicación línea a línea — ConcurrencyTokensConfiguration.cs

Línea 1: `public sealed class OrdenFabricacionConcurrencyConfiguration :`
`IEntityTypeConfiguration<OrdenFabricacion>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `public void Configure(EntityTypeBuilder<OrdenFabricacion> b) =>` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 4: `b.Property(x => x.RowVersion).IsRowVersion();` → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 5: `}` → Delimita el bloque sintáctico correspondiente.

Línea 7: `public sealed class PlanchaAceroConcurrencyConfiguration :`
`IEntityTypeConfiguration<PlanchaAcero>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 8: `{` → Delimita el bloque sintáctico correspondiente.

Línea 9: `public void Configure(EntityTypeBuilder<PlanchaAcero> b) =>` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 10: `b.Property(x => x.RowVersion).IsRowVersion();` → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 11: `}` → Delimita el bloque sintáctico correspondiente.

Línea 13: `public sealed class AleacionConcurrencyConfiguration :`
`IEntityTypeConfiguration<Aleacion>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 14: `{` → Delimita el bloque sintáctico correspondiente.

Línea 15: `public void Configure(EntityTypeBuilder<Aleacion> b) =>` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 16: `b.Property(x => x.RowVersion).IsRowVersion();` → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 17: `}` → Delimita el bloque sintáctico correspondiente.

Línea 19: `public sealed class DetalleOrdenConcurrencyConfiguration : IEntityTypeConfiguration<DetalleOrden>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 20: `{` → Delimita el bloque sintáctico correspondiente.

Línea 21: `public void Configure(EntityTypeBuilder<DetalleOrden> b)` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 22: `{` → Delimita el bloque sintáctico correspondiente.

Línea 23: `b.Property(x => x.EstadoDetalle)` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 24: `.IsRequired()` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 25: `.HasMaxLength(50)` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 26: `.HasDefaultValue("Pendiente")` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 27: `.IsConcurrencyToken();` → Marca la propiedad existente como token de concurrencia para que su valor original participe en escrituras.

Línea 28: `}` → Delimita el bloque sintáctico correspondiente.

Línea 29: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar TokensConcurrenciaM5Repositorio.cs

```
public RowVersionM5Dto DemostrarRowVersion()
{
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var ordenA = contextA OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var inicialA = Convert.ToHexString(ordenA.RowVersion);
    var inicialB = Convert.ToHexString(ordenB.RowVersion);

    SqlCommandCounterInterceptor.Instance.Reset();
    ordenA.Cliente = "Cliente 5.2 A";
    contextA.SaveChanges();
    var despuesA = Convert.ToHexString(ordenA.RowVersion);

    ordenB.Estado = "EnProceso";
    var conflicto = false;
    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException)
    {
        conflicto = true;
    }

    return new RowVersionM5Dto(
        inicialA, inicialB, despuesA, conflicto,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TokensConcurrenciaM5Repositorio.cs

Línea 1: `public RowVersionM5Dto DemostrarRowVersion()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 4: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 6: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 10: `var inicialA = Convert.ToHexString(ordenA.RowVersion);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `var inicialB = Convert.ToHexString(ordenB.RowVersion);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 13: `SqlCommandCounterInterceptor.Instance.Reset();` → Reinicia el contador para que solo se midan los comandos de este escenario.

Línea 14: `ordenA.Cliente = "Cliente 5.2 A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 16: `var despuesA = Convert.ToHexString(ordenA.RowVersion);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 18: `ordenB.Estado = "EnProceso";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `var conflicto = false;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 20: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 21: `{` → Delimita el bloque sintáctico correspondiente.

Línea 22: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 23: `}` → Delimita el bloque sintáctico correspondiente.

Línea 24: `catch (DbUpdateConcurrencyException)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 25: `{` → Delimita el bloque sintáctico correspondiente.

Línea 26: `conflicto = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 27: `}` → Delimita el bloque sintáctico correspondiente.

Línea 29: `return new RowVersionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 30: `inicialA, inicialB, despuesA, conflicto,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 31: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 32: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar TokensConcurrenciaM5Repositorio.cs

```
public IndiceRowVersionM5Dto ComprobarIndiceRowVersion()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText = """
        SELECT COUNT(*)
        FROM sys.indexes AS i
        INNER JOIN sys.index_columns AS ic
            ON i.object_id = ic.object_id AND i.index_id = ic.index_id
        INNER JOIN sys.columns AS c
            ON ic.object_id = c.object_id AND ic.column_id = c.column_id
        WHERE i.object_id = OBJECT_ID(N'dbo.OrdenesFabricacion')
            AND c.name = N'RowVersion';
        """;

    return new IndiceRowVersionM5Dto(Convert.ToInt32(command.ExecuteScalar()) > 0);
}
```

Explicación línea a línea — TokensConcurrenciaM5Repositorio.cs

Línea 1: `public IndiceRowVersionM5Dto ComprobarIndiceRowVersion()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 4: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 5: `var connection = context.Database.GetDbConnection();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 6: `if (connection.State != ConnectionState.Open)` → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.

Línea 7: `connection.Open();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 9: `using var command = connection.CreateCommand();` → Importa el espacio de nombres necesario para los tipos y extensiones usados por el archivo, o declara un recurso con liberación automática cuando se usa `using var`.

Línea 10: `command.CommandText = """` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 11: `SELECT COUNT(*)` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 12: `FROM sys.indexes AS i` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 13: `INNER JOIN sys.index_columns AS ic` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 14: `ON i.object_id = ic.object_id AND i.index_id = ic.index_id` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `INNER JOIN sys.columns AS c` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 16: `ON ic.object_id = c.object_id AND ic.column_id = c.column_id` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 17: `WHERE i.object_id = OBJECT_ID(N'dbo.OrdenesFabricacion')` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 18: `AND c.name = N'RowVersion';` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `"";` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 21: `return new IndiceRowVersionM5Dto(Convert.ToInt32(command.ExecuteScalar()) > 0);` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 22: `}` → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet ef database update --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.2 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- RowVersion se configura explícitamente con `IsRowVersion()`.
- EstadoDetalle se configura como `IsConcurrencyToken()`.
- La migración `M5_5_2_ConcurrencyTokens` existe y se aplica.
- Dos contextos producen `DbUpdateConcurrencyException` cuando cambia el token.
- Se comprueba directamente que RowVersion no obtiene un índice automático.

Reto resuelto y ampliación

Reto: Inspecciona el UPDATE capturado y localiza el token original dentro del predicado de concurrencia.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Confiar en el nombre RowVersion sin configurarlo.
- Suponer que rowversion crea automáticamente un índice.
- Probar semántica de rowversion de SQL Server con SQLite.
- Modificar a mano el snapshot o la migración para forzar el resultado.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.2 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.

4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.2, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.2 OK

Conexión con el siguiente punto

El siguiente estado es 5.3 y parte directamente de este proyecto completo.

Punto 5.3 — Resolución de conflictos de concurrencia

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.2. La carpeta M05/PROYECTO/5.3 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Resolver DbUpdateConcurrencyException con políticas explícitas: cliente gana, base gana, merge, notificación, fila eliminada y reintento acotado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.3
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/ResolucionConflictosM5Repositorio.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar ResolucionConflictosM5Repositorio.cs

```
public ResolucionConflictoM5Dto ClienteGana()
{
    RestaurarOrden();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var a = Cargar(contextA);
    var b = Cargar(contextB);

    a.Cliente = "Cliente A - cliente gana";
    contextA.SaveChanges();

    b.Cliente = "Cliente B - cliente gana";
    var conflicto = false;
    var intentos = 1;
    var valores = new List<ValorConflictoM5Dto>();

    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException ex)
    {
        conflicto = true;
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La orden desapareció durante ClienteGana.");
        valores = CrearValores(entry, db);
        entry.OriginalValues.SetValues(db);
        intentos++;
        contextB.SaveChanges();
    }

    var final = LeerFinal();
    return Resultado("Cliente gana", conflicto, final, intentos, valores);
}
```

Explicación línea a línea — ResolucionConflictosM5Repositorio.cs

Línea 1: `public ResolucionConflictoM5Dto ClienteGana()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarOrden();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 7: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `var a = Cargar(contextA);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `var b = Cargar(contextB);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `a.Cliente = "Cliente A - cliente gana";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 14: `b.Cliente = "Cliente B - cliente gana";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `var conflicto = false;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 16: `var intentos = 1;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 17: `var valores = new List<ValorConflictoM5Dto>();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 19: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 20: `{` → Delimita el bloque sintáctico correspondiente.

Línea 21: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 22: `}` → Delimita el bloque sintáctico correspondiente.

Línea 23: `catch (DbUpdateConcurrencyException ex)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 24: `{` → Delimita el bloque sintáctico correspondiente.

Línea 25: `conflicto = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 26: `var entry = ex.Entries.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 27: `var db = entry.GetDatabaseValues();` → Obtiene de la base el estado actual de la fila tras detectar el conflicto.

Línea 28: `?? throw new InvalidOperationException("La orden desapareció durante ClienteGana.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 29: `valores = CrearValores(entry, db);` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 30: `entry.OriginalValues.SetValues(db);` → Actualiza los valores originales del entry para reintentar contra la versión recién leída.

Línea 31: `intentos++;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 32: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 33: `}` → Delimita el bloque sintáctico correspondiente.

Línea 35: `var final = LeerFinal();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 36: `return Resultado("Cliente gana", conflicto, final, intentos, valores);` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 37: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar ResolucionConflictosM5Repositorio.cs

```
public ResolucionConflictoM5Dto ResolucionPersonalizada()
{
    RestaurarOrden();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var a = Cargar(contextA);
    var b = Cargar(contextB);

    a.Cliente = "Cliente A - merge";
    a.Estado = "EnProceso A";
    contextA.SaveChanges();

    b.Cliente = "Cliente B - merge";
    b.Estado = "Completada B";
    var intentos = 1;

    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException ex)
    {
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La orden desapareció durante la fusión.");
        entry.OriginalValues.SetValues(db);
        entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];
        intentos++;
        contextB.SaveChanges();
    }

    return Resultado("Resolución personalizada", true, LeerFinal(), intentos, []);
}
```

Explicación línea a línea — ResolucionConflictosM5Repositorio.cs

Línea 1: `public ResolucionConflictoM5Dto ResolucionPersonalizada()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarOrden();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 7: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `var a = Cargar(contextA);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `var b = Cargar(contextB);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `a.Cliente = "Cliente A - merge";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12: `a.Estado = "EnProceso A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 13: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 15: `b.Cliente = "Cliente B - merge";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 16: `b.Estado = "Completada B";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 17: `var intentos = 1;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 19: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 20: `{` → Delimita el bloque sintáctico correspondiente.

Línea 21: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 22: `}` → Delimita el bloque sintáctico correspondiente.

Línea 23: `catch (DbUpdateConcurrencyException ex)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 24: `{` → Delimita el bloque sintáctico correspondiente.

Línea 25: `var entry = ex.Entries.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 26: `var db = entry.GetDatabaseValues();` → Obtiene de la base el estado actual de la fila tras detectar el conflicto.

Línea 27: `?? throw new InvalidOperationException("La orden desapareció durante la fusión.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 28: `entry.OriginalValues.SetValues(db);` → Actualiza los valores originales del entry para reintentar contra la versión recién leída.

Línea 29: `entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 30: `intentos++;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 31: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 32: `}` → Delimita el bloque sintáctico correspondiente.

Línea 34: `return Resultado("Resolución personalizada", true, LeerFinal(), intentos, []);` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 35: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar ResolucionConflictosM5Repositorio.cs

```
while (true)
{
    intentos++;
    try
    {
        contextB.SaveChanges();
        break;
    }
    catch (DbUpdateConcurrencyException ex) when (intentos < maxIntentos)
    {
        conflicto = true;
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
        ?? throw new InvalidOperationException("La fila ya no existe.");
        entry.OriginalValues.SetValues(db);
    }
}
```

Explicación línea a línea — ResolucionConflictosM5Repositorio.cs

Línea 1: `while (true)` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `intentos++;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 5: `{` → Delimita el bloque sintáctico correspondiente.

Línea 6: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 7: `break;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 8: `}` → Delimita el bloque sintáctico correspondiente.

Línea 9: `catch (DbUpdateConcurrencyException ex) when (intentos < maxIntentos)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 10: `{` → Delimita el bloque sintáctico correspondiente.

Línea 11: `conflicto = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12: `var entry = ex.Entries.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 13: `var db = entry.GetDatabaseValues()` → Obtiene de la base el estado actual de la fila tras detectar el conflicto.

Línea 14: `?? throw new InvalidOperationException("La fila ya no existe.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 15: `entry.OriginalValues.SetValues(db);` → Actualiza los valores originales del entry para reintentar contra la versión recién leída.

Línea 16: `}` → Delimita el bloque sintáctico correspondiente.

Línea 17: `}` → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.3 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- `GetDatabaseValues` recupera el estado actual de SQL Server.
- `OriginalValues.SetValues` actualiza la versión contra la que se reintenta.
- `Reload` descarta cambios locales cuando la base gana.
- La resolución personalizada decide propiedad por propiedad.
- Los reintentos están limitados.

Reto resuelto y ampliación

Reto: Añade una política que preserve Cliente local, Estado de base y combine Observaciones solo cuando ambos lados no sean nulos.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Reintentar indefinidamente.
- Confundir `CurrentValues`, `OriginalValues` y valores actuales de base.
- Sobrescribir automáticamente sin informar al usuario cuando el negocio requiere decisión.
- No tratar `GetDatabaseValues() == null` cuando la fila fue eliminada.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.3 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que `Application` continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.3, `AceriaData` debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

```
5.3 OK
```

Conexión con el siguiente punto

El siguiente estado es 5.4 y parte directamente de este proyecto completo.

Punto 5.4 — Transacciones: SaveChanges y transacciones explícitas

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.3. La carpeta M05/PROYECTO/5.4 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Demostrar atomicidad de SaveChanges, Commit, Rollback y savepoints reales con SQL Server y MARS desactivado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.4
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/TransaccionesM5Repositorio.cs
- src/AceriaData.Console/appsettings.json

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar TransaccionesM5Repositorio.cs

```
public ResultadoTransaccionM5Dto DemostrarAtomicidadSaveChanges()
{
    const string numeroValido = "0F-M5-54-ATOMIC";
    Limpiar(numeroValido);
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

    context OrdenesFabricacion.Add(CrearOrden(numeroValido));
    context OrdenesFabricacion.Add(CrearOrden(NumeroExistente));

    var falloEsperado = false;
    try
    {
        context.SaveChanges();
    }
    catch (DbUpdateException)
    {
        falloEsperado = true;
        context.ChangeTracker.Clear();
    }

    var existeValida = Existe(numeroValido);
    return new ResultadoTransaccionM5Dto(
        "Atomicidad de un único SaveChanges",
        falloEsperado && !existeValida,
        existeValida,
        Existe(NumeroExistente),
        MarsHabilitado(context),
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TransaccionesM5Repositorio.cs

Línea 1: `public ResultadoTransaccionM5Dto DemostrarAtomicidadSaveChanges()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `const string numeroValido = "0F-M5-54-ATOMIC";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4: `Limpiar(numeroValido);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 5: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `context OrdenesFabricacion.Add(CrearOrden(numeroValido));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 9: `context OrdenesFabricacion.Add(CrearOrden(NumeroExistente));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 11: `var falloEsperado = false;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 13: `{` → Delimita el bloque sintáctico correspondiente.

Línea 14: `context.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 15: `}` → Delimita el bloque sintáctico correspondiente.

Línea 16: `catch (DbUpdateException)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 17: `{` → Delimita el bloque sintáctico correspondiente.

Línea 18: `falloEsperado = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `context.ChangeTracker.Clear();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 20: `}` → Delimita el bloque sintáctico correspondiente.

Línea 22: `var existeValida = Existe(numeroValido);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 23: `return new ResultadoTransaccionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 24: `"Atomicidad de un único SaveChanges"`, → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 25: `falloEsperado && !existeValida,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 26: `existeValida,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 27: `Existe(NumeroExistente),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 28: `MarshHabilitado(context),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 29: `SqlCommandCounterInterceptor.Instance.SnapshotCommands());` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 30: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar TransaccionesM5Repositorio.cs

```
public ResultadoTransaccionM5Dto DemostrarRollbackASavepoint()
{
    const string numero1 = "0F-M5-54-SP-1";
    const string numero2 = "0F-M5-54-SP-2";
    const string savepoint = "AntesSegundaOrden";
    Limpiar(numero1, numero2);

    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var mars = MarsHabilitado(context);
    if (mars)
        throw new InvalidOperationException("5.4 requiere MultipleActiveResultSets=false.");

    using (var transaction = context.Database.BeginTransaction())
    {
        context.OrdenesFabricacion.Add(CrearOrden(numero1));
        context.SaveChanges();
        transaction.CreateSavepoint(savepoint);

        context.OrdenesFabricacion.Add(CrearOrden(numero2));
        context.SaveChanges();
        transaction.RollbackToSavepoint(savepoint);

        context.ChangeTracker.Clear();
        transaction.Commit();
    }

    return new ResultadoTransaccionM5Dto(
        "Rollback a savepoint",
        Existe(numero1) && !Existe(numero2),
        Existe(numero1), Existe(numero2), mars,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TransaccionesM5Repositorio.cs

Línea 1: `public ResultadoTransaccionM5Dto DemostrarRollbackASavepoint()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `const string numero1 = "0F-M5-54-SP-1";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4: `const string numero2 = "0F-M5-54-SP-2";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 5: `const string savepoint = "AntesSegundaOrden";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 6: `Limpiar(numero1, numero2);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 8: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 9: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 10: `var mars = MarsHabilitado(context);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `if (mars)` → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.

Línea 12: `throw new InvalidOperationException("5.4 requiere MultipleActiveResultSets=false.");` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 14: `using (var transaction = context.Database.BeginTransaction())` → Abre una transacción explícita sobre la conexión del DbContext.

Línea 15: `{` → Delimita el bloque sintáctico correspondiente.

Línea 16: `context.OrdenesFabricacion.Add(CrearOrden(numero1));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 17: `context.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 18: `transaction.CreateSavepoint(savepoint);` → Crea un punto intermedio al que puede volver la transacción sin descartar todo el trabajo previo.

Línea 20: `context.OrdenesFabricacion.Add(CrearOrden(numero2));` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 21: `context.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 22: `transaction.RollbackToSavepoint(savepoint);` → Revierte solo las operaciones posteriores al savepoint indicado.

Línea 24: `context.ChangeTracker.Clear();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 25: `transaction.Commit();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 26: `}` → Delimita el bloque sintáctico correspondiente.

Línea 28: `return new ResultadoTransaccionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 29: `"Rollback a savepoint",` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 30: `Existe(numero1) && !Existe(numero2),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 31: `Existe(numero1), Existe(numero2), mars,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 32: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 33: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar appsettings.json

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\\
\\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResult
Sets=false;"
  }
}
```

Explicación línea a línea — appsettings.json

Línea 1: `{` → Delimita el bloque sintáctico correspondiente.

Línea 2: `"ConnectionStrings": {` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 3: "AceriaDB": "Server=(localdb)\

\MSSQLLocalDB;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultS

→ Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 4: } → Delimita el bloque sintáctico correspondiente.

Línea 5: } → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-
project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con 5.4 OK. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- Un único SaveChanges revierte todo el conjunto si una escritura falla.
- Una transacción explícita puede agrupar varias llamadas a SaveChanges.
- Rollback deshace una escritura ya enviada antes del fallo posterior.
- RollbackToSavepoint conserva lo anterior al savepoint y revierte lo posterior.
- MultipleActiveResultSets=false en el laboratorio de savepoints.

Reto resuelto y ampliación

Reto: Introduce un tercer SaveChanges después del savepoint y decide qué parte debe persistir tras un rollback parcial.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Dar por hecho que Commit fallido siempre implica un estado conocido.
- Usar MARS y esperar savepoints automáticos de EF Core.
- Presentar RELEASE SAVEPOINT como operación necesaria de SQL Server.
- Mantener transacciones abiertas durante trabajo ajeno a la base.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador 5.4 OK.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.4, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.4 OK

Conexión con el siguiente punto

El siguiente estado es 5.5 y parte directamente de este proyecto completo.

Punto 5.5 — Transacciones ambientales y buenas prácticas

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.4. La carpeta M05/PROYECTO/5.5 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Usar TransactionScope con async, varios DbContext sobre una conexión compartida, opciones de scope y un efecto externo no transaccional.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.5
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/TransaccionesAmbientalesM5Repositorio.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar TransaccionesAmbientalesM5Repositorio.cs

```
public async Task<TransaccionAmbientaLM5Dto> DemostrarDosContextosAsync()
{
    var options = new TransactionOptions
    {
        IsolationLevel = IsolationLevel.ReadCommitted,
        Timeout = TimeSpan.FromSeconds(30)
    };

    bool ambienteActivo;
    bool flujoAsync;
    bool promocion;
    string aislamiento;
    int ordenId;

    using (var scope = new TransactionScope(
        TransactionScopeOption.Required,
        options,
        TransactionScopeAsyncFlowOption.Enabled))
    {
        await using var connection = new SqlConnection(ObtenerConnectionString());
        await connection.OpenAsync();
        ambienteActivo = Transaction.Current is not null;
        aislamiento = Transaction.Current?.IsolationLevel.ToString() ?? "<none>";

        await using (var contextOrden = CrearContexto(connection))
        {
            var orden = CrearOrden("OF-M5-55-AMBIENT");
            contextOrden.OrdenesFabricacion.Add(orden);
            await contextOrden.SaveChangesAsync();
        }
    }
}
```

```

        ordenId = orden.Id;
    }

    await Task.Yield();
    flujoAsync = Transaction.Current is not null;

    await using (var contextPlancha = CrearContexto(connection))
    {
        contextPlancha.PlanchasAcero.Add(new PlanchaAcero
        {
            OrdenId = ordenId,
            Espesor = 10, Ancho = 1000, Largo = 2000, Peso = 150m
        });
        await contextPlancha.SaveChangesAsync();
    }

    promocion = Transaction.Current?.TransactionInformation.DistributedIdentifier != Guid.Empty;
    scope.Complete();
}

return new TransaccionAmbientaM5Dto(
    "Dos DbContext con una conexión compartida",
    ExisteOrdenYPlancha("OF-M5-55-AMBIENT"),
    ambienteActivo, flujoAsync, promocion, aislamiento,
    SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

Explicación línea a línea — TransaccionesAmbientalesM5Repositorio.cs

Línea 1: `public async Task<TransaccionAmbientaM5Dto> DemostrarDosContextosAsync()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `var options = new TransactionOptions` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 4: `{` → Delimita el bloque sintáctico correspondiente.

Línea 5: `IsolationLevel = IsolationLevel.ReadCommitted,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 6: `Timeout = TimeSpan.FromSeconds(30)` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 7: `};` → Delimita el bloque sintáctico correspondiente.

Línea 9: `bool ambienteActivo;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 10: `bool flujoAsync;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `bool promocion;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `string aislamiento;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 13: `int ordenId;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 15: `using (var scope = new TransactionScope(` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 16: `TransactionScopeOption.Required,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 17: `options,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 18: `TransactionScopeAsyncFlowOption.Enabled))` → Permite que `Transaction.Current` fluya correctamente a través de `await`.

Línea 19: `{` → Delimita el bloque sintáctico correspondiente.

Línea 20: `await using var connection = new SqlConnection(ObtenerConnectionString());` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 21: `await connection.OpenAsync();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 22: `ambienteActivo = Transaction.Current is not null;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 23: `aislamiento = Transaction.Current?.IsolationLevel.ToString() ?? "<none>";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 25: `await using (var contextOrden = CrearContexto(connection))` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 26: `{` → Delimita el bloque sintáctico correspondiente.

Línea 27: `var orden = CrearOrden("OF-M5-55-AMBIENT");` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 28: `contextOrden.OrdenesFabricacion.Add(orden);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 29: `await contextOrden.SaveChangesAsync();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 30: `ordenId = orden.Id;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 31: `}` → Delimita el bloque sintáctico correspondiente.

Línea 33: `await Task.Yield();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 34: `flujoAsync = Transaction.Current is not null;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 36: `await using (var contextPlancha = CrearContexto(connection))` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 37: `{` → Delimita el bloque sintáctico correspondiente.

Línea 38: `contextPlancha.PlanchasAcero.Add(new PlanchaAcero` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 39: `{` → Delimita el bloque sintáctico correspondiente.

Línea 40: `OrdenId = ordenId,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 41: `Espesor = 10, Ancho = 1000, Largo = 2000, Peso = 150m` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 42: `});` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 43: `await contextPlancha.SaveChangesAsync();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 44: `}` → Delimita el bloque sintáctico correspondiente.

Línea 46: `promocion = Transaction.Current?.TransactionInformation.DistributedIdentifier != Guid.Empty;` → Comprueba si la transacción fue promocionada a una transacción distribuida.

Línea 47: `scope.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 48: `}` → Delimita el bloque sintáctico correspondiente.

Línea 50: `return new TransaccionAmbientaIM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 51: `"Dos DbContext con una conexión compartida",` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 52: `ExisteOrdenYPlancha("OF-M5-55-AMBIENT"),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 53: `ambienteActivo, flujoAsync, promocion, aislamiento,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 54: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 55: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `TransaccionesAmbientalesM5Repositorio.cs`

```
public OpcionesTransactionScopeM5Dto DemostrarOpcionesDeScope()
{
    string outerId;
    bool requiredMisma;
    bool requiresNewOtra;
    bool suppressSinAmbiente;
    string aislamiento;

    using (var outer = new TransactionScope())
    {
        var outerTransaction = Transaction.Current
            ?? throw new InvalidOperationException("No se creó la transacción ambiental.");
        outerId = outerTransaction.TransactionInformation.LocalIdentifier;
        aislamiento = outerTransaction.IsolationLevel.ToString();

        using (var required = new TransactionScope(TransactionScopeOption.Required))
        {
            requiredMisma = Transaction.Current?.TransactionInformation.LocalIdentifier == outerId;
            required.Complete();
        }

        using (var requiresNew = new TransactionScope(TransactionScopeOption.RequiresNew))
        {
            requiresNewOtra = Transaction.Current?.TransactionInformation.LocalIdentifier != outerId;
            requiresNew.Complete();
        }

        using (var suppressed = new TransactionScope(TransactionScopeOption.Suppress))
        {
            suppressSinAmbiente = Transaction.Current is null;
            suppressed.Complete();
        }
        outer.Complete();
    }

    return new OpcionesTransactionScopeM5Dto(
        requiredMisma, requiresNewOtra, suppressSinAmbiente,
        aislamiento, TransactionManager.DefaultTimeout.TotalSeconds);
}
```

Explicación línea a línea — `TransaccionesAmbientalesM5Repositorio.cs`

Línea 1: `public OpcionesTransactionScopeM5Dto DemostrarOpcionesDeScope()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `string outerId;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 4: `bool requiredMisma;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 5: `bool requiresNewOtra;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 6: `bool suppressSinAmbiente;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 7: `string aislamiento;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `using (var outer = new TransactionScope())` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 10: `{` → Delimita el bloque sintáctico correspondiente.

Línea 11: `var outerTransaction = Transaction.Current` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `?? throw new InvalidOperationException("No se creó la transacción ambiental.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 13: `outerId = outerTransaction.TransactionInformation.LocalIdentifier;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 14: `aislamiento = outerTransaction.IsolationLevel.ToString();` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 16: `using (var required = new TransactionScope(TransactionScopeOption.Required))` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 17: `{` → Delimita el bloque sintáctico correspondiente.

Línea 18: `requiredMisma = Transaction.Current?.TransactionInformation.LocalIdentifier == outerId;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `required.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 20: `}` → Delimita el bloque sintáctico correspondiente.

Línea 22: `using (var requiresNew = new TransactionScope(TransactionScopeOption.RequiresNew))` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 23: `{` → Delimita el bloque sintáctico correspondiente.

Línea 24: `requiresNewOtra = Transaction.Current?.TransactionInformation.LocalIdentifier != outerId;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 25: `requiresNew.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 26: `}` → Delimita el bloque sintáctico correspondiente.

Línea 28: `using (var suppressed = new TransactionScope(TransactionScopeOption.Suppress))` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 29: `{` → Delimita el bloque sintáctico correspondiente.

Línea 30: `suppressSinAmbiente = Transaction.Current is null;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 31: `suppressed.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 32: `}` → Delimita el bloque sintáctico correspondiente.

Línea 33: `outer.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 34: `}` → Delimita el bloque sintáctico correspondiente.

Línea 36: `return new OpcionesTransactionScopeM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 37: `requiredMisma, requiresNewOtra, suppressSinAmbiente,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 38: `aislamiento, TransactionManager.DefaultTimeout.TotalSeconds);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 39: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.5 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- `TransactionScopeAsyncFlowOption.Enabled` conserva `Transaction.Current` tras `await`.
- Los dos `DbContext` usan una misma `SqlConnection` abierta para no depender de promoción distribuida.
- `Required`, `RequiresNew` y `Suppress` se distinguen por su `Transaction.Current`.
- Omitir `Complete` revierte SQL.
- El efecto externo simulado permanece porque no participa en `System.Transactions`.

Reto resuelto y ampliación

Reto: Ejecuta el escenario con `Suppress` alrededor de una operación y verifica qué queda fuera de la transacción ambiental.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Asumir que `ReadCommitted` permite lecturas sucias.
- Afirmar que `Snapshot` elimina todos los bloqueos y conflictos.
- Abrir varios recursos durables sin considerar posible promoción a transacción distribuida.
- Esperar que HTTP, colas o archivos se reviertan con `TransactionScope`.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.5 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.5, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.5 OK

Conexión con el siguiente punto

El siguiente estado es `5.6` y parte directamente de este proyecto completo.

Punto 5.6 — Migraciones en entornos de producción: estrategias y despliegue

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.5. La carpeta M05/PROYECTO/5.6 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Separar desarrollo y despliegue controlado mediante IMigrator, scripts revisables y migration bundles, conservando el historial real.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.6
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/MigracionesProduccionM5Repositorio.cs
- deployment/generate-production-artifacts.ps1
- deployment/PLAN_DESPLIEGUE.md

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar MigracionesProduccionM5Repositorio.cs

```
public async Task<MigracionesProduccionM5Dto> AplicarConIMigratorAsync()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var migrator = context.GetService<IMigrator>();
    await migrator.MigrateAsync();

    var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();
    var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();
    var tablaExiste = await ExisteTablaHistorialAsync(context);

    return new MigracionesProduccionM5Dto(
        aplicadas.Length,
        pendientes.Length,
        aplicadas.LastOrDefault() ?? "<ninguna>",
        tablaExiste,
        "__EFMigrationsHistory");
}
```

Explicación línea a línea — MigracionesProduccionM5Repositorio.cs

Línea 1: `public async Task<MigracionesProduccionM5Dto> AplicarConIMigratorAsync()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 4: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infraestructura sea la misma que en la aplicación.

Línea 6: `var migrator = context.GetService<IMigrator>();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 7: `await migrator.MigrateAsync();` → Aplica la cadena real de migraciones al proveedor de pruebas o despliegue.

Línea 9: `var aplicadas = (await context.Database.GetAppliedMigrationsAsync()).ToArray();` → Lee la lista de migraciones que la base registra como aplicadas.

Línea 10: `var pendientes = (await context.Database.GetPendingMigrationsAsync()).ToArray();` → Comprueba si queda alguna migración del ensamblado pendiente de aplicar.

Línea 11: `var tablaExiste = await ExisteTablaHistorialAsync(context);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 13: `return new MigracionesProduccionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 14: `aplicadas.Length,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 15: `pendientes.Length,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 16: `aplicadas.LastOrDefault() ?? "<ninguna>",` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 17: `tablaExiste,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 18: `"__EFMigrationsHistory");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 19: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `generate-production-artifacts.ps1`

```
$ErrorActionPreference = "Stop"
$Artifacts = Join-Path $PSScriptRoot "artifacts"
New-Item -ItemType Directory -Force -Path $Artifacts | Out-Null

dotnet ef migrations script --idempotent `
  --project ../src/AceriaData.Infrastructure `
  --startup-project ../src/AceriaData.Console `
  --configuration Release `
  --output (Join-Path $Artifacts "aceria-idempotent.sql")

if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el script idempotente" }

dotnet ef migrations bundle `
  --project ../src/AceriaData.Infrastructure `
  --startup-project ../src/AceriaData.Console `
  --configuration Release `
  --output (Join-Path $Artifacts "aceria-efbundle.exe") `
  --force

if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el migration bundle" }
```

Explicación línea a línea — generate-production-artifacts.ps1

Línea 1: `$ErrorActionPreference = "Stop"` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 2: `$Artifacts = Join-Path $PSScriptRoot "artifacts"` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 3: `New-Item -ItemType Directory -Force -Path $Artifacts | Out-Null` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 5: ``dotnet ef migrations script --idempotent ``` → Genera SQL desde la cadena real de migraciones de EF Core.

Línea 6: ``--project ../src/AceriaData.Infrastructure ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 7: ``--startup-project ../src/AceriaData.Console ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 8: ``--configuration Release ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 9: `--output (Join-Path $Artifacts "aceria-idempotent.sql")` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 11: `if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el script idempotente" }` → Convierte un fallo del comando externo en un fallo explícito del script.

Línea 13: ``dotnet ef migrations bundle ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 14: ``--project ../src/AceriaData.Infrastructure ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 15: ``--startup-project ../src/AceriaData.Console ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 16: ``--configuration Release ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 17: ``--output (Join-Path $Artifacts "aceria-efbundle.exe") ``` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 18: `--force` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 20: `if ($LASTEXITCODE -ne 0) { throw "No se pudo generar el migration bundle" }` → Convierte un fallo del comando externo en un fallo explícito del script.

Paso 5: Implementar y estudiar Comandos de despliegue

```
dotnet ef migrations script --idempotent `
  --project src/AceriaData.Infrastructure `
  --startup-project src/AceriaData.Console `
  --configuration Release `
  --output aceria-idempotent.sql

# El bundle recibe la conexión en ejecución, no desde código fuente.
./aceria-efbundle.exe --connection $env:ACERIA_PROD_CONNECTION
```

Explicación línea a línea — Comandos de despliegue

Línea 1: ``dotnet ef migrations script --idempotent ``` → Genera SQL desde la cadena real de migraciones de EF Core.

Línea 2: `--project src/AceriaData.Infrastructure` `` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 3: `--startup-project src/AceriaData.Console` `` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 4: `--configuration Release` `` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 5: `--output aceria-idempotent.sql` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 7: `# El bundle recibe la conexión en ejecución, no desde código fuente.` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 8: `./aceria-efbundle.exe --connection $env:ACERIA_PROD_CONNECTION` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.6 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- MigrationsAssembly apunta al ensamblado real de migraciones.
- IMigrator aplica la cadena y deja cero migraciones pendientes.
- Se conserva `__EFMigrationsHistory`.
- El bundle recibe la cadena de conexión desde el entorno de despliegue.
- El plan distingue downgrade de recuperación de datos.

Reto resuelto y ampliación

Reto: Genera un script revisable y un bundle para la misma cadena y documenta qué control operacional aporta cada uno.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Ejecutar Database.Migrate en cada réplica como regla general de producción.
- Renombrar MigrationsHistoryTable sin mover el historial existente.
- Guardar credenciales de producción en el repositorio.
- Confundir un Down destructivo con restauración segura.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.6 OK`.

3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.6, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.6 OK

Conexión con el siguiente punto

El siguiente estado es 5.7 y parte directamente de este proyecto completo.

Reconstrucción editorial M5: los puntos 5.1–5.6 conservan la versión corregida y ya cerrada técnicamente. Desde 5.7 se reconstruye la práctica desde las fuentes originales del usuario y se aplican únicamente las correcciones técnicas explícitas del contrato M5.

Punto 5.7 — Migraciones idempotentes y scripts SQL

Práctica

Ejercicio: Generar scripts SQL idempotentes para el proyecto AceriaData. Generar un script con todas las migraciones, un script con un rango de migraciones, un script de reversión y un script idempotente. Revisar los scripts y aplicarlos con sqlcmd con autenticación integrada de Windows. Integrar los scripts en un pipeline de GitHub Actions simulado. Analizar el SQL generado y medir el tiempo de generación.

Contexto del proyecto: En el punto 5.6 se estudiaron las migraciones en producción, incluyendo las estrategias de despliegue y las opciones MigrationsAssembly y MigrationsHistoryTable. En este punto se profundiza en las migraciones idempotentes y los scripts SQL. Esta técnica se usará en el punto 5.8 para las migraciones en equipos.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Infrastructure
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Infrastructure → entra en la carpeta del proyecto de infraestructura.

Paso 2: Generar un script con todas las migraciones

```
dotnet ef migrations script --startup-project ../AceriaData.Console --output migraciones_completas.sql
```

dotnet ef migrations script → genera un script SQL. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output migraciones_completas.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo migraciones_completas.sql con todas las migraciones.

El SQL generado incluye la creación de todas las tablas, índices y claves foráneas. También incluye la inserción de las migraciones en el historial.

Error común: si el script no se revisa antes de aplicarlo, se pueden producir errores. Se debe revisar el script.

Paso 3: Generar un script idempotente

```
dotnet ef migrations script --idempotent --startup-project ../AceriaData.Console --output
migraciones_idempotentes.sql
```

dotnet ef migrations script → genera un script SQL. --idempotent → genera un script que se puede aplicar varias veces. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output migraciones_idempotentes.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo migraciones_idempotentes.sql con el script idempotente. El script incluye comprobaciones IF NOT EXISTS antes de cada operación.

Error común: si el script no incluye las comprobaciones, al aplicarlo por segunda vez se producen errores. Se debe usar --idempotent.

Paso 4: Generar un script con un rango de migraciones

```
dotnet ef migrations script InitialCreate AddRowVersion --startup-project ../AceriaData.Console --
output migraciones_rango.sql
```

dotnet ef migrations script → genera un script SQL. InitialCreate → migración desde. AddRowVersion → migración hasta. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output migraciones_rango.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo migraciones_rango.sql con las migraciones desde InitialCreate hasta AddRowVersion.

Error común: si la migración desde no existe, el comando falla. Se debe verificar el nombre con dotnet ef migrations list.

Paso 5: Generar un script de reversión

```
dotnet ef migrations script AddRowVersion InitialCreate --startup-project ../AceriaData.Console --
output reversion.sql
```

dotnet ef migrations script → genera un script SQL. AddRowVersion → migración desde. InitialCreate → migración hasta. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output reversion.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo reversion.sql con el script de reversión desde AddRowVersion hasta InitialCreate.

Error común: si se aplica un script de reversión sin copia de seguridad, los datos se pierden. Se debe hacer copia antes.

Paso 6: Verificar los artefactos sin usar tiempos prefijados

La fuente original incluía valores temporales fijos para comparar la generación de scripts. Esos valores no forman parte del contrato canónico porque dependen del equipo, del almacenamiento y del estado de la caché.

Comprueba únicamente que se han generado los tres artefactos esperados y registra, si quieres, el tiempo **observado en esa ejecución** sin convertirlo en una conclusión general.

```
ls -lh migrations.sql migrations-idempotent.sql rollback.sql
```

Corrección técnica aplicada: el criterio de este paso es la existencia y el contenido de los artefactos, no que aparezcan 4200 ms, 4100 ms o 4000 ms.

Paso 7: Revisar el script idempotente

Abrir el archivo migraciones_idempotentes.sql y revisar el contenido. Comprobar las operaciones que se van a ejecutar y los posibles efectos secundarios.

```
IF OBJECT_ID(N'[__AceriaMigraciones]') IS NULL
BEGIN
    CREATE TABLE [__AceriaMigraciones] (
        [MigrationId] nvarchar(150) NOT NULL,
        [ProductVersion] nvarchar(32) NOT NULL,
        CONSTRAINT [PK__AceriaMigraciones] PRIMARY KEY ([MigrationId])
    );
END;
GO

IF NOT EXISTS(SELECT * FROM [__AceriaMigraciones] WHERE [MigrationId] = N'20240115120000_InitialCreate')
BEGIN
    CREATE TABLE [OrdenesFabricacion] (
        [Id] int NOT NULL IDENTITY,
        [NumeroOrden] nvarchar(50) NOT NULL,
        [Cliente] nvarchar(200) NOT NULL,
        [FechaCreacion] datetime2 NOT NULL DEFAULT (GETDATE()),
        [RowVersion] rowversion NOT NULL,
        CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id])
    );

    CREATE UNIQUE INDEX [AK_OrdenesFabricacion_NumeroOrden] ON [OrdenesFabricacion] ([NumeroOrden]);
END;
GO
```

Línea 1: IF OBJECT_ID(N'[__AceriaMigraciones]') IS NULL → comprueba si la tabla de historial existe.

Línea 3: CREATE TABLE [__AceriaMigraciones] (→ crea la tabla de historial si no existe.

Línea 4: [MigrationId] nvarchar(150) NOT NULL, → columna del identificador de migración.

Línea 5: [ProductVersion] nvarchar(32) NOT NULL, → columna de la versión del producto.

Línea 6: CONSTRAINT [PK__AceriaMigraciones] PRIMARY KEY ([MigrationId]) → clave primaria.

Línea 11: IF NOT EXISTS(SELECT * FROM [__AceriaMigraciones] WHERE [MigrationId] = N'20240115120000_InitialCreate') → comprueba si la migración ya está aplicada.

Línea 13: CREATE TABLE [OrdenesFabricacion] (→ crea la tabla si no existe.

Línea 14: [Id] int NOT NULL IDENTITY, → columna Id.

Línea 15: [NumeroOrden] nvarchar(50) NOT NULL, → columna NumeroOrden.

Línea 16: [Cliente] nvarchar(200) NOT NULL, → columna Cliente.

Línea 17: [FechaCreacion] datetime2 NOT NULL DEFAULT (GETDATE()), → columna FechaCreacion con valor por defecto.

Línea 18: [RowVersion] rowversion NOT NULL, → columna RowVersion de tipo rowversion.

Línea 19: CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id]) → clave primaria.

Línea 22: CREATE UNIQUE INDEX [AK_OrdenesFabricacion_NumeroOrden] ON [OrdenesFabricacion] ([NumeroOrden]); → índice único sobre NumeroOrden.

Observaciones: el script comprueba si cada operación ya está aplicada antes de aplicarla. Se puede aplicar varias veces sin error. La tabla de historial se llama __AceriaMigraciones porque así se configuró en el punto 5.6.

Paso 8: Aplicar el script idempotente con sqlcmd

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB -E -i migraciones_idempotentes.sql
```

sqlcmd → invoca la herramienta de línea de comandos de SQL Server. -S "(localdb)\MSSQLLocalDB" → indica la instancia de SQL Server. -d AceriaDB → indica la base de datos de destino. -E → usa autenticación integrada de Windows. -i migraciones_idempotentes.sql → indica el archivo de entrada.

Resultado esperado: el script se aplica a la base de datos AceriaDB. Si el script se aplica de nuevo, no se producen errores porque es idempotente.

Error común: si sqlcmd no está en el PATH, el comando no se reconoce. Se debe instalar la herramienta o usar la ruta completa.

Paso 9: Verificar la aplicación del script

Abrir el Explorador de objetos de SQL Server. Comprobar que las tablas y los índices se han creado correctamente.

```
SELECT * FROM [__AceriaMigraciones];
```

La primera línea consulta la tabla de historial. Devuelve las migraciones aplicadas.

Resultado esperado: las tablas y los índices aparecen en la base de datos. La tabla de historial contiene las migraciones aplicadas.

Paso 9 bis: Aplicar dos veces el mismo script idempotente y comprobar el historial

La idempotencia debe demostrarse ejecutando realmente el mismo script dos veces contra la misma base de pruebas.

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaData_M5_7 -i migrations-idempotent.sql -b
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaData_M5_7 -i migrations-idempotent.sql -b
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaData_M5_7 -Q "SELECT MigrationId FROM __EFMigrationsHistory
ORDER BY MigrationId" -b
```

La segunda aplicación debe terminar correctamente sin duplicar filas en `__EFMigrationsHistory` ni recrear objetos ya migrados. El script idempotente de EF Core consulta el historial de migraciones y aplica únicamente las migraciones pendientes.

Comprueba también que el esquema esperado existe después de ambas ejecuciones. Para los scripts de rango, usa explícitamente una migración `FROM` y una `TO`. Si generas un script con `FROM` posterior a `TO`, trátalo como un **downgrade potencialmente destructivo** y revísalo antes de ejecutarlo.

Paso 10: Crear un pipeline de GitHub Actions simulado

Crear el archivo `.github/workflows/deploy-migrations.yml`:

```
yml name: Deploy Migrations
```

```
on: push: branches: - main
```

```
jobs: deploy: runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup .NET
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.x'

  - name: Install dotnet-ef
    run: dotnet tool install --global dotnet-ef

  - name: Generate idempotent script
    run: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output
migraciones.sql

  - name: Upload script as artifact
    uses: actions/upload-artifact@v4
    with:
      name: migraciones
      path: migraciones.sql

  - name: Apply script to LocalDB
    run: sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB -E -i migraciones.sql

```

Línea 1: name: Deploy Migrations → nombre del workflow.

Línea 3: on: → define el trigger.

Línea 4: push: → se ejecuta en push.

Línea 5: branches: → ramas que activan el workflow.

Línea 6: - main → la rama principal.

Línea 8: jobs: → define los jobs.

Línea 9: deploy: → nombre del job.

Línea 10: runs-on: ubuntu-latest → sistema operativo del agente.

Línea 12: steps: → define los pasos.

Línea 13: - name: Checkout → primer paso.

Línea 14: uses: actions/checkout@v4 → acción de checkout.

Línea 16: - name: Setup .NET → segundo paso.

Línea 17: uses: actions/setup-dotnet@v4 → acción de instalación de .NET.

Línea 18: with: → parámetros de la acción.

Línea 19: dotnet-version: '8.x' → versión de .NET.

Línea 21: - name: Install dotnet-ef → tercer paso.

Línea 22: run: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 24: - name: Generate idempotent script → cuarto paso.

Línea 25: run: dotnet ef migrations script --idempotent ... → genera el script.

Línea 27: - name: Upload script as artifact → quinto paso.

Línea 28: uses: actions/upload-artifact@v4 → acción de subida de artefactos.

Línea 29: with: → parámetros.

Línea 30: name: migraciones → nombre del artefacto.

Línea 31: path: migraciones.sql → ruta del archivo.

Línea 33: - name: Apply script to LocalDB → sexto paso.

Línea 34: run: sqlcmd ... -i migraciones.sql → aplica el script.

Error común: si los secretos no están configurados en GitHub, el pipeline falla. Se deben configurar en la configuración del repositorio.

| Paso 11: Crear un pipeline de Azure DevOps simulado

Crear el archivo azure-pipelines.yml:

yaml trigger:

- main

pool: vmImage: 'ubuntu-latest'

steps:

- task: UseDotNet@2 inputs: version: '8.x'
- script: dotnet tool install --global dotnet-ef displayName: 'Instalar dotnet-ef'
- script: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output \$(Build.ArtifactStagingDirectory)/migraciones.sql displayName: 'Generar script idempotente'
- task: PublishBuildArtifacts@1 inputs: PathtoPublish: '\$(Build.ArtifactStagingDirectory)/migraciones.sql' ArtifactName: 'migraciones'
- script: sqlcmd -S "\$(SqlServer)" -d AceriaDB -U "\$(SqlUser)" -P "\$(SqlPassword)" -i \$(Build.ArtifactStagingDirectory)/migraciones.sql displayName: 'Aplicar script' Línea 1: trigger: → define el trigger.

Línea 2: - main → la rama principal.

Línea 4: pool: → define el agente.

Línea 5: vmImage: 'ubuntu-latest' → sistema operativo.

Línea 7: steps: → define los pasos.

Línea 8: - task: UseDotNet@2 → instala .NET.

Línea 9: inputs: → parámetros.

Línea 10: version: '8.x' → versión de .NET.

Línea 12: - script: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 13: displayName: 'Instalar dotnet-ef' → nombre del paso.

Línea 15: - script: dotnet ef migrations script --idempotent ... → genera el script.

Línea 16: displayName: 'Generar script idempotente' → nombre del paso.

Línea 18: - task: PublishBuildArtifacts@1 → publica el artefacto.

Línea 19: inputs: → parámetros.

Línea 20: PathtoPublish: ... → ruta del archivo.

Línea 21: ArtifactName: 'migraciones' → nombre del artefacto.

Línea 23: - script: sqlcmd ... → aplica el script.

Línea 24: displayName: 'Aplicar script' → nombre del paso.

Error común: si las credenciales no están configuradas como variables secretas, el pipeline falla. Se deben configurar en Azure DevOps.

| Paso 12: Aplicar el script en múltiples bases de datos

Crear el archivo aplicar_multiples_bd.sh:

```
#!/bin/bash

echo "=== APLICAR SCRIPT EN MÚLTIPLES BASES DE DATOS ==="

for db in AceriaDB_Dev AceriaDB_Test AceriaDB_Prod
do
    echo "Aplicando script en $db..."
    sqlcmd -S "(localdb)\MSSQLLocalDB" -d $db -E -i migraciones_idempotentes.sql
    if [ $? -eq 0 ]; then
        echo "OK: $db"
    else
        echo "ERROR: $db"
    fi
done
```

Línea 1: #!/bin/bash → shebang del script.

Línea 3: echo "=== APLICAR SCRIPT EN MÚLTIPLES BASES DE DATOS ===" → muestra la cabecera.

Línea 5: for db in AceriaDB_Dev AceriaDB_Test AceriaDB_Prod → itera sobre las bases de datos.

Línea 6: do → inicio del bloque.

Línea 7: echo "Aplicando script en \$db..." → muestra el mensaje.

Línea 8: sqlcmd ... -d \$db -E -i migraciones_idempotentes.sql → aplica el script.

Línea 9: if [\$? -eq 0]; then → comprueba si el comando ha tenido éxito.

Línea 10: echo "OK: \$db" → muestra el mensaje.

Línea 11: else → caso contrario.

Línea 12: echo "ERROR: \$db" → muestra el error.

Línea 13: fi → cierre del condicional.

Línea 14: done → cierre del bucle.

Error común: si alguna base de datos no existe, el comando falla. Se debe verificar la existencia antes de aplicar.

| Paso 13: Ejecutar el script de múltiples bases de datos

```
chmod +x aplicar_multiples_bd.sh
./aplicar_multiples_bd.sh
```

Resultado esperado: el script se aplica a las tres bases de datos. Si alguna no existe, se muestra un error.

| Paso 14: Analizar la salida

La salida del comando muestra información como la siguiente:

=== APLICAR SCRIPT EN MÚLTIPLES BASES DE DATOS ===

Aplicando script en AceriaDB_Dev...

OK: AceriaDB_Dev

Aplicando script en AceriaDB_Test...

OK: AceriaDB_Test

Aplicando script en AceriaDB_Prod...

ERROR: AceriaDB_Prod

La primera sección muestra la cabecera. La segunda sección aplica el script en las bases de datos. La tercera sección muestra los resultados.

Observaciones: el script se aplica correctamente en las bases de datos que existen. La base de datos que no existe muestra un error.

Paso 15: Diagnosticar un error común

Modificar el script migraciones_idempotentes.sql para eliminar las comprobaciones IF NOT EXISTS:

```
CREATE TABLE [OrdenesFabricacion] (
    [Id] int NOT NULL IDENTITY,
    [NumeroOrden] nvarchar(50) NOT NULL,
    [Cliente] nvarchar(200) NOT NULL,
    [FechaCreacion] datetime2 NOT NULL DEFAULT (GETDATE()),
    [RowVersion] rowversion NOT NULL,
    CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id])
);
GO
```

Resultado esperado: al aplicar el script por segunda vez, se produce un error porque la tabla ya existe.

Solución: restaurar las comprobaciones IF NOT EXISTS.

```
IF NOT EXISTS(SELECT * FROM [__AceriaMigraciones] WHERE [MigrationId] = N'20240115120000_InitialCreate')
BEGIN
    CREATE TABLE [OrdenesFabricacion] (...);
END;
GO
```

Resultado esperado con la solución: el script se puede aplicar varias veces sin error.

Errores comunes del ejercicio

Error Causa Solución Script no idempotente Se aplicó varias veces Usar --idempotent Migración desde no existe El nombre es incorrecto Verificar con dotnet ef migrations list Script elimina datos El script elimina una columna con datos Revisar el script y hacer copia de seguridad Script aplicado en la base equivocada No se verificó la base de datos de destino Verificar la base de datos antes de aplicar sqlcmd no reconocido No está en el PATH Instalar la herramienta o usar la ruta completa Pipeline aplica sin revisar No se revisó el script Revisar antes de aplicar Autenticación de Windows falla El usuario no tiene permisos Otorgar los permisos necesarios

Reto resuelto: Integrar scripts en un pipeline de GitHub Actions con secretos

Reto: Crear un pipeline de GitHub Actions que genere el script idempotente, lo suba como artefacto y lo aplique en una base de datos SQL Server remota usando secretos. Simular el flujo con un pipeline local.

Solución paso a paso:

Paso 1: Crear el archivo .github/workflows/deploy-migrations.yml:

yaml name: Deploy Migrations

on: push: branches: - main

jobs: deploy: runs-on: ubuntu-latest

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup .NET
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.x'

  - name: Install dotnet-ef
    run: dotnet tool install --global dotnet-ef

  - name: Generate idempotent script
    run: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output
migraciones.sql

  - name: Upload script as artifact
    uses: actions/upload-artifact@v4
    with:
      name: migraciones
      path: migraciones.sql

  - name: Apply script to SQL Server
    run: sqlcmd -S "${{ secrets.SQL_SERVER }}" -d "${{ secrets.SQL_DATABASE }}" -U "$
${{ secrets.SQL_USER }}" -P "${{ secrets.SQL_PASSWORD }}" -i migraciones.sql

```

Línea 1: name: Deploy Migrations → nombre del workflow.

Línea 3: on: → define el trigger.

Línea 4: push: → se ejecuta en push.

Línea 5: branches: → ramas que activan el workflow.

Línea 6: - main → la rama principal.

Línea 8: jobs: → define los jobs.

Línea 9: deploy: → nombre del job.

Línea 10: runs-on: ubuntu-latest → sistema operativo del agente.

Línea 12: steps: → define los pasos.

Línea 13: - name: Checkout → primer paso.

Línea 14: uses: actions/checkout@v4 → acción de checkout.

Línea 16: - name: Setup .NET → segundo paso.

Línea 17: uses: actions/setup-dotnet@v4 → acción de instalación de .NET.

Línea 18: with: → parámetros de la acción.

Línea 19: dotnet-version: '8.x' → versión de .NET.

Línea 21: - name: Install dotnet-ef → tercer paso.

Línea 22: run: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 24: - name: Generate idempotent script → cuarto paso.

Línea 25: run: dotnet ef migrations script --idempotent ... → genera el script.

Línea 27: - name: Upload script as artifact → quinto paso.

Línea 28: uses: actions/upload-artifact@v4 → acción de subida.

Línea 29: with: → parámetros.

Línea 30: name: migraciones → nombre del artefacto.

Línea 31: path: migraciones.sql → ruta del archivo.

Línea 33: - name: Apply script to SQL Server → sexto paso.

Línea 34: run: sqlcmd ... -i migraciones.sql → aplica el script con secretos.

Paso 2: Configurar los secretos en GitHub:

1. Ir al repositorio en GitHub.

1. Ir a Settings → Secrets and variables → Actions.

2. Crear los secretos: SQL_SERVER, SQL_DATABASE, SQL_USER, SQL_PASSWORD.

Paso 3: Hacer push a la rama principal y verificar que el pipeline se ejecuta.

Resultado esperado: el pipeline genera el script idempotente, lo sube como artefacto y lo aplica en la base de datos remota con los secretos.

Analogía final

Las migraciones idempotentes en una acería son como los manuales de reforma que se pueden aplicar varias veces sin riesgo. Si la reforma ya está hecha, el manual no la repite. Los scripts con rango de migraciones son como los manuales que solo cubren un tramo de la reforma. Los scripts de reversión son como los manuales que deshacen la reforma.

Aplicar los scripts con sqlcmd es como aplicar el manual en la planta con las herramientas adecuadas. La autenticación integrada de Windows es como usar la llave maestra de la planta. La autenticación de SQL Server es como usar una llave específica para una puerta. Los permisos restringidos son como las llaves limitadas que solo abren ciertas puertas.

Integrar los scripts en CI/CD es como automatizar la aplicación de los manuales en cada planta. El pipeline de GitHub Actions es como una cadena de montaje que genera el manual, lo revisa, lo archiva y lo aplica. Los secretos son como las llaves que se guardan en una caja fuerte. La revisión previa es como leer el manual antes de aplicarlo.

El impacto de los scripts en el rendimiento es de milisegundos. La generación de un script tarda unos 4 segundos. La aplicación depende del número de migraciones y del tamaño de las tablas.

Así funcionan las migraciones idempotentes en EF Core: se generan, se revisan y se aplican de forma controlada en cada entorno.

Resultado esperado

Al final del ejercicio, deberías haber:

Generado un script con todas las migraciones.

Generado un script idempotente.

Generado un script con un rango de migraciones.

Generado un script de reversión.

Medido el tiempo de generación de cada script.

Revisado el script idempotente.

Aplicado el script idempotente con sqlcmd con autenticación integrada.

Verificado la aplicación del script.

Creado un pipeline de GitHub Actions.

Creado un pipeline de Azure DevOps.

Aplicado el script en múltiples bases de datos.

Diagnosticado el error de eliminar las comprobaciones IF NOT EXISTS.

| Conexión con el siguiente punto

En este punto se han estudiado las migraciones idempotentes y los scripts SQL, incluyendo su generación, revisión, aplicación y uso en pipelines de CI/CD. Se ha comprobado que los scripts idempotentes se pueden aplicar varias veces sin error y que son esenciales para desplegar migraciones de forma controlada. También se han cubierto los pipelines de Azure DevOps y GitHub Actions, la autenticación integrada de Windows y de SQL Server, y el uso de scripts en entornos con múltiples bases de datos.

En el siguiente punto se estudiarán las migraciones en equipos: conflictos, merges y buenas prácticas, con sus implicaciones en el trabajo colaborativo.

Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas

Práctica

Ejercicio: Simular un conflicto de migraciones en el proyecto AceriaData. Crear dos ramas, generar una migración en cada una, hacer merge y resolver el conflicto con regeneración y renombrado. Aplicar las buenas prácticas, revisar las migraciones en un pull request simulado y medir el impacto de los conflictos en el tiempo de resolución.

Contexto del proyecto: En el punto 5.7 se estudiaron las migraciones idempotentes y los scripts SQL, incluyendo su generación, revisión y aplicación en pipelines de CI/CD. En este punto se profundiza en las migraciones en equipos, incluyendo los conflictos, los merges y las buenas prácticas. Esta técnica se usará en el punto 5.9 para el patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales.

Paso 1: Abrir el proyecto

```
cd AceriaData
git status
```

cd AceriaData → entra en la carpeta raíz del proyecto. git status → muestra el estado del repositorio.

Resultado esperado: el repositorio está limpio y en la rama principal.

Error común: si hay cambios sin confirmar, se deben confirmar o descartar antes de continuar.

Paso 2: Crear la rama feature/AddDetalleOrden

```
git checkout -b feature/AddDetalleOrden
```

git checkout -b → crea una nueva rama y se cambia a ella. feature/AddDetalleOrden → nombre de la rama.

Resultado esperado: se crea la rama feature/AddDetalleOrden.

Paso 3: Añadir la entidad DetalleOrden en la rama feature

Crear el archivo src/AceriaData.Domain/Entities/DetalleOrden.cs:

```
namespace AceriaData.Domain.Entities;

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 1: namespace AceriaData.Domain.Entities; → declara el espacio de nombres.

Línea 3: public class DetalleOrden → declara la entidad.

Línea 5: public int Id { get; set; } → clave primaria.

Línea 6: public int OrdenId { get; set; } → clave foránea.

Línea 7: public string ComposicionQuimica { get; set; } = string.Empty; → composición química.

Línea 8: `public double TemperaturaColada { get; set; } → temperatura de la colada.`

Línea 9: `public string? Notas { get; set; } → notas opcionales.`

Línea 11: `public virtual OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación.`

Error común: si se olvida el namespace, el código no compila.

| Paso 4: Añadir el DbSet de DetalleOrden al DbContext

Modificar `src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs`:

```
public DbSet<DetalleOrden> DetallesOrden { get; set; } = null!;
```

Línea 1: `public DbSet DetallesOrden { get; set; } = null!; → expone la tabla DetallesOrden.`

Error común: si no se añade el DbSet, la entidad no se incluye en el modelo y no se crea la tabla.

| Paso 5: Generar la migración en la rama feature

```
dotnet ef migrations add AddDetalleOrden --startup-project src/AceriaData.Console
```

`dotnet ef migrations add` → genera una nueva migración. `AddDetalleOrden` → nombre de la migración. `--startup-project src/AceriaData.Console` → indica el proyecto de inicio.

Resultado esperado: se genera la migración `AddDetalleOrden` en la rama `feature`.

Error común: si la migración está vacía, se debe verificar que la entidad `DetalleOrden` esté declarada en el `DbContext`.

| Paso 6: Hacer commit en la rama feature

```
git add .
git commit -m "Add DetalleOrden"
```

`git add .` → añade todos los cambios al área de preparación. `git commit -m` → hace commit con el mensaje indicado.

Resultado esperado: los cambios se han confirmado en la rama `feature`.

| Paso 7: Volver a la rama principal

```
git checkout main
```

`git checkout main` → cambia a la rama principal.

Resultado esperado: se cambia a la rama principal.

| Paso 8: Crear la rama bugfix/AddIndiceNumeroOrden

```
git checkout -b bugfix/AddIndiceNumeroOrden
```

`git checkout -b` → crea una nueva rama y se cambia a ella. `bugfix/AddIndiceNumeroOrden` → nombre de la rama.

Resultado esperado: se crea la rama `bugfix/AddIndiceNumeroOrden`.

Paso 9: Añadir un índice en la rama bugfix

Modificar `src/AceriaData.Infrastructure/Persistence/Configurations/OrdenFabricacionConfiguration.cs` para añadir un índice:

```
public void Configure(EntityTypeBuilder<OrdenFabricacion> builder)
{
    // ... configuración existente ...

    builder.HasIndex(o => o.NumeroOrden)
        .IsUnique()
        .HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");
}
```

Línea 1: `public void Configure(EntityTypeBuilder builder)` → método de configuración.

Línea 3: `// ... configuración existente ...` → comentario que indica que se mantiene la configuración anterior.

Línea 5: `builder.HasIndex(o => o.NumeroOrden)` → crea un índice sobre `NumeroOrden`.

Línea 6: `.IsUnique()` → marca el índice como único.

Línea 7: `.HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");` → establece el nombre del índice.

Error común: si el índice ya existe, la migración falla. Se debe verificar antes.

Paso 10: Generar la migración en la rama bugfix

```
dotnet ef migrations add AddIndiceNumeroOrden --startup-project src/AceriaData.Console
```

`dotnet ef migrations add` → genera una nueva migración. `AddIndiceNumeroOrden` → nombre de la migración. `--startup-project src/AceriaData.Console` → indica el proyecto de inicio.

Resultado esperado: se genera la migración `AddIndiceNumeroOrden` en la rama `bugfix`.

Error común: si la migración está vacía, se debe verificar que el índice esté configurado correctamente.

Paso 11: Hacer commit en la rama bugfix

```
git add .
git commit -m "Add índice único en NumeroOrden"
```

`git add .` → añade todos los cambios al área de preparación. `git commit -m` → hace commit con el mensaje indicado.

Resultado esperado: los cambios se han confirmado en la rama `bugfix`.

Paso 12: Volver a la rama principal y hacer merge de feature

```
git checkout main
git merge feature/AddDetalleOrden
```

`git checkout main` → cambia a la rama principal. `git merge feature/AddDetalleOrden` → hace merge de la rama `feature`.

Resultado esperado: la migración `AddDetalleOrden` se integra en la rama principal.

Paso 13: Hacer merge de la rama bugfix

```
git merge bugfix/AddIndiceNumeroOrden
```

git merge bugfix/AddIndiceNumeroOrden → hace merge de la rama bugfix.

Resultado esperado: se produce un conflicto en el archivo `AceriaDbContextModelSnapshot.cs` porque ambas ramas han modificado el snapshot.

La salida del comando muestra información como la siguiente:

```
Auto-merging Migrations/AceriaDbContextModelSnapshot.cs
CONFLICT (content): Merge conflict in Migrations/AceriaDbContextModelSnapshot.cs
Automatic merge failed; fix conflicts and then commit the result.
Observaciones: el conflicto se produce en el snapshot porque ambas ramas han añadido entidades o índices al modelo.
```

Paso 14: Detectar el conflicto con git status

```
git status
```

git status → muestra el estado del repositorio.

Resultado esperado: aparece el archivo `AceriaDbContextModelSnapshot.cs` con la marca `both modified`.

La salida del comando muestra información como la siguiente:

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   Migrations/AceriaDbContextModelSnapshot.cs
Observaciones: el archivo del snapshot está en conflicto porque ambas ramas han modificado el modelo.
```

Paso 15: Resolver el conflicto con regeneración

Para resolver el conflicto con regeneración, se elimina la migración conflictiva, se descarta el conflicto del snapshot y se regenera la migración.

```
# 1. Abortar el merge para empezar de nuevo
git merge --abort

# 2. Hacer merge de feature
git merge feature/AddDetalleOrden

# 3. Eliminar la migración de bugfix
git checkout bugfix/AddIndiceNumeroOrden
dotnet ef migrations remove --startup-project src/AceriaData.Console
git commit -am "Eliminar migración AddIndiceNumeroOrden"

# 4. Volver a main y hacer merge de bugfix
git checkout main
git merge bugfix/AddIndiceNumeroOrden

# 5. Regenerar la migración en main
dotnet ef migrations add AddIndiceNumeroOrden --startup-project src/AceriaData.Console
git commit -am "Add índice único en NumeroOrden"
```

Línea 1: `git merge --abort` → aborta el merge en curso.

Línea 2: `git merge feature/AddDetalleOrden` → hace merge de feature.

Línea 3: `git checkout bugfix/AddIndiceNumeroOrden` → cambia a la rama bugfix.

Línea 4: `dotnet ef migrations remove --startup-project src/AceriaData.Console` → elimina la migración conflictiva.

Línea 5: `git commit -am "Eliminar migración AddIndiceNumeroOrden"` → confirma la eliminación.

Línea 6: `git checkout main` → cambia a la rama principal.

Línea 7: `git merge bugfix/AddIndiceNumeroOrden` → hace merge de bugfix.

Línea 8: `dotnet ef migrations add AddIndiceNumeroOrden --startup-project src/AceriaData.Console` → regenera la migración en main.

Línea 9: `git commit -am "Add índice único en NumeroOrden"` → confirma la migración.

Resultado esperado: el conflicto se resuelve y la migración se regenera en la rama principal.

Error común: si se olvida eliminar la migración conflictiva, el conflicto persiste.

Paso 16: Aplicar las migraciones

```
dotnet ef database update --startup-project src/AceriaData.Console
```

`dotnet ef database update` → aplica las migraciones pendientes.

Resultado esperado: la base de datos queda sincronizada con el modelo.

Paso 17: Verificar las migraciones

```
dotnet ef migrations list --startup-project src/AceriaData.Console
```

`dotnet ef migrations list` → lista todas las migraciones.

Resultado esperado: aparecen `InitialCreate`, `AddDetalleOrden` y `AddIndiceNumeroOrden` con la marca (Applied).

Paso 18: Verificar la resolución del conflicto sin rankings temporales

No uses tiempos prefijados para decidir qué estrategia es correcta. Valida el estado de Git, la cadena de migraciones y el snapshot resultante.

```
git status
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/
AceriaData.Console
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-
project src/AceriaData.Console
```

El criterio de éxito es que el árbol quede coherente, que las migraciones representen el modelo fusionado y que `has-pending-model-changes` no detecte cambios pendientes.

Paso 19: Crear un checklist de revisión de migraciones

Crear el archivo `checklist_migraciones.md`:

markdown

Checklist de revisión de migraciones

Antes del merge

1. ¿El nombre de la migración es descriptivo?
2. ¿Los cambios son los esperados?
3. ¿Hay operaciones destructivas (DROP COLUMN, DROP TABLE)?
4. ¿El snapshot es coherente con el modelo?
5. ¿Las migraciones se aplican sin errores?
6. ¿Hay conflictos con la rama principal?

Después del merge

1. ¿Se han regenerado las migraciones conflictivas?
2. ¿Se ha aplicado la migración a la base de datos?
3. ¿Se ha verificado el estado de la base de datos?
4. ¿Se ha actualizado el script idempotente?
5. ¿Se ha actualizado el pipeline de CI/CD? Línea 1: # Checklist de revisión de migraciones → título del checklist.

Línea 3: ## Antes del merge → sección de comprobaciones antes del merge.

Línea 5: 1. ¿El nombre de la migración es descriptivo? → primera comprobación.

Línea 6: 2. ¿Los cambios son los esperados? → segunda comprobación.

Línea 7: 3. ¿Hay operaciones destructivas (DROP COLUMN, DROP TABLE)? → tercera comprobación.

Línea 8: 4. ¿El snapshot es coherente con el modelo? → cuarta comprobación.

Línea 9: 5. ¿Las migraciones se aplican sin errores? → quinta comprobación.

Línea 10: 6. ¿Hay conflictos con la rama principal? → sexta comprobación.

Línea 12: ## Después del merge → sección de comprobaciones después del merge.

Línea 14: 1. ¿Se han regenerado las migraciones conflictivas? → primera comprobación.

Línea 15: 2. ¿Se ha aplicado la migración a la base de datos? → segunda comprobación.

Línea 16: 3. ¿Se ha verificado el estado de la base de datos? → tercera comprobación.

Línea 17: 4. ¿Se ha actualizado el script idempotente? → cuarta comprobación.

Línea 18: 5. ¿Se ha actualizado el pipeline de CI/CD? → quinta comprobación.

Error común: si no se documenta el checklist, los desarrolladores pueden olvidar alguna comprobación. Se debe documentar y compartir en el equipo.

| Paso 20: Crear un pipeline de verificación de migraciones

Crear el archivo `.github/workflows/verify-migrations.yml`:

```
yaml name: Verify Migrations
```

```
on: pull_request: branches: - main
```

```
jobs: verify: runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup .NET
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.x'

  - name: Install dotnet-ef
    run: dotnet tool install --global dotnet-ef

  - name: Verify migrations are up to date
    run: dotnet ef migrations list --startup-project src/AceriaData.Console

  - name: Generate idempotent script
    run: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output
    migraciones.sql

  - name: Apply script to test database
    run: sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB_Test -E -i migraciones.sql

  - name: Verify database schema
    run: sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB_Test -Q "SELECT COUNT(*) FROM
    [__AceriaMigraciones];"

```

Línea 1: name: Verify Migrations → nombre del workflow.

Línea 3: on: → define el trigger.

Línea 4: pull_request: → se ejecuta en pull request.

Línea 5: branches: → ramas que activan el workflow.

Línea 6: - main → la rama principal.

Línea 8: jobs: → define los jobs.

Línea 9: verify: → nombre del job.

Línea 10: runs-on: ubuntu-latest → sistema operativo.

Línea 12: steps: → define los pasos.

Línea 13: - name: Checkout → primer paso.

Línea 14: uses: actions/checkout@v4 → acción de checkout.

Línea 16: - name: Setup .NET → segundo paso.

Línea 17: uses: actions/setup-dotnet@v4 → acción de instalación.

Línea 18: with: → parámetros.

Línea 19: dotnet-version: '8.x' → versión.

Línea 21: - name: Install dotnet-ef → tercer paso.

Línea 22: run: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 24: - name: Verify migrations are up to date → cuarto paso.

Línea 25: run: dotnet ef migrations list ... → lista las migraciones.

Línea 27: - name: Generate idempotent script → quinto paso.

Línea 28: run: dotnet ef migrations script --idempotent ... → genera el script.

Línea 30: - name: Apply script to test database → sexto paso.

Línea 31: run: sqlcmd ... -i migraciones.sql → aplica el script.

Línea 33: - name: Verify database schema → séptimo paso.

Línea 34: run: sqlcmd ... -Q "SELECT COUNT(*) FROM [__AceriaMigraciones];" → verifica el esquema.

Error común: si el pipeline no verifica las migraciones, los conflictos se detectan en producción. Se debe verificar en el pull request.

Paso 21: Ejecutar el proyecto

```
cd ../AceriaData.Console
dotnet run
```

cd ../AceriaData.Console → entra en la carpeta del proyecto de consola. dotnet run → compila y ejecuta el proyecto.

Resultado esperado: el proyecto compila y se ejecuta sin errores.

Paso 22: Analizar la salida

La salida del proyecto muestra que las migraciones se han aplicado correctamente. La base de datos contiene las tablas OrdenesFabricacion, DetallesOrden y los índices correspondientes.

La verificación con dotnet ef migrations list muestra:

```
20240115120000_InitialCreate
20240118120000_AddDetalleOrden
20240119120000_AddIndiceNumeroOrden
Las tres migraciones están aplicadas.
```

Observaciones: el conflicto se resolvió con regeneración. La migración AddIndiceNumeroOrden se regeneró en la rama principal después del merge. El snapshot es coherente con el modelo.

Paso 23: Diagnosticar un error común

Intentar modificar una migración ya aplicada:

```
# Modificar el archivo de la migración InitialCreate
```

Resultado esperado: la base de datos queda desincronizada con el modelo. Al generar una nueva migración, EF Core no detecta el cambio porque el snapshot ya está actualizado.

Solución: revertir el cambio en la migración y generar una nueva migración para los cambios.

```
dotnet ef migrations add AddNuevoCambio --startup-project src/AceriaData.Console
```

Resultado esperado con la solución: la nueva migración refleja los cambios.

Errores comunes del ejercicio

Error Causa Solución Conflicto en el snapshot Dos ramas modificaron el snapshot Eliminar la migración conflictiva y regenerarla Migración divergente Dos ramas generaron cambios sobre snapshots distintos Integrar el cambio compartido y regenerar la migración local cuando sea seguro Migración modificada después de aplicada Se modificó el archivo de la migración Generar una nueva migración Base de datos desincronizada No se aplicaron las migraciones después del merge Aplicar dotnet ef database update Snapshot desactualizado Se modificó manualmente Regenerar con dotnet ef migrations add Migración vacía El modelo no cambió Verificar que las entidades están declaradas Rama acumulada No se eliminaron las ramas de prueba Eliminar con git branch -D

Reto resuelto: Resolver migraciones divergentes retirando y regenerando la migración propia

Reto: simular dos ramas que modifican el modelo. La migración de la rama propia todavía no se ha compartido ni aplicado a una base común. Incorpora primero el cambio del compañero y regenera después tu migración sobre el snapshot fusionado.

Paso 1: Crear dos ramas con cambios de modelo distintos

Genera una migración independiente en cada rama y confirma cada cambio. No renombres manualmente IDs, clases, atributos y snapshots para “ordenar fechas”.

Paso 2: Incorporar la migración del compañero

Vuelve a la rama que contiene tu trabajo, integra la migración del compañero y resuelve únicamente los conflictos de modelo que correspondan.

Paso 3: Retirar la migración propia si todavía es local

```
dotnet ef migrations remove --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

Este paso solo es válido cuando esa migración propia no ha sido compartida ni aplicada a una base común.

Paso 4: Regenerar la migración propia sobre el snapshot fusionado

```
dotnet ef migrations add MiCambioRegenerado --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

Paso 5: Validar el resultado

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

Si una migración ya se compartió o se aplicó a una base común, **no se elimina simplemente el archivo**. En ese caso se necesita una migración correctiva o una coordinación explícita de rollback/despliegue.

Corrección técnica aplicada: renombrar archivos y metadatos de una migración divergente no es la estrategia docente recomendada para reconciliar snapshots.

Analogía final

Las migraciones en equipos en una acería son como las reformas coordinadas de la planta. Cuando varios equipos trabajan en la misma planta, cada uno hace su reforma. Si dos equipos reforman la misma zona, se produce un conflicto.

La solución es coordinar las reformas, eliminar una de ellas y volver a hacerla después de la otra. Las buenas prácticas son como las normas de coordinación: nombres descriptivos, sincronización frecuente, no modificar reformas ya hechas y revisar antes de aplicar.

El snapshot del modelo es como el plano actual de la planta: refleja el estado después de todas las reformas. Si dos equipos modifican el plano a la vez, se produce un conflicto. La solución es regenerar el plano después de coordinar las reformas.

El checklist de revisión es como la inspección antes de aprobar la reforma. El pipeline de verificación es como la cadena de montaje que verifica automáticamente que la reforma es correcta.

El impacto de los conflictos en el tiempo de resolución es significativo. No se establece un ranking por tiempos prefijados. La decisión depende del estado de la migración y de si ya se compartió o aplicó.

Así funcionan las migraciones en equipos: se coordinan, se sincronizan y se regeneran cuando es necesario.

| Resultado esperado

Al final del ejercicio, deberías haber:

Simulado un conflicto de migraciones en dos ramas.

Resuelto el conflicto eliminando una migración y regenerándola.

Resuelto el conflicto mediante integración y regeneración segura.

Aplicado las migraciones después del merge.

Verificado las migraciones con `dotnet ef migrations list`.

Medido el impacto de los conflictos en el tiempo de resolución.

Creado un checklist de revisión de migraciones.

Creado un pipeline de verificación de migraciones.

Diagnosticado el error de modificar una migración ya aplicada.

Resuelto un conflicto de migraciones sin renombrado manual de metadatos.

| Conexión con el siguiente punto

En este punto se han estudiado las migraciones en equipos, incluyendo los conflictos, las estrategias de resolución, el papel del snapshot del modelo, las buenas prácticas, la integración con revisión de código y la medición del impacto de los conflictos. Se ha comprobado que los conflictos más habituales son de nombre, de entidad y de snapshot, y que la regeneración es la estrategia más rápida.

En el siguiente punto se estudiará el Patrón Repositorio y la Unidad de Trabajo en aplicaciones empresariales, con sus implicaciones en el acceso a datos y en la arquitectura limpia.

Punto 5.9 — Patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales

Práctica

Corrección técnica canónica: EF Core ya ofrece en `DbContext` y `DbSet` capacidades relacionadas con Unit of Work y Repository. La abstracción adicional usada por `AceriaData` es una decisión arquitectónica del curso, no una regla universal. No se afirma que Repository sea siempre mejor ni que `DbContext` deba estar siempre oculto.

Ejercicio: Consolidar el patrón Repositorio y la unidad de trabajo en el proyecto `AceriaData`. Refactorizar el repositorio para que sea una implementación empresarial completa. Integrar `IDbContextFactory` para servicios de larga duración. Comparar el rendimiento y el consumo de memoria con y sin repositorio. Documentar los anti-patrones con ejemplos de código. Testing del repositorio con mocks.

Contexto del proyecto: En el punto 5.8 se estudiaron las migraciones en equipos, incluyendo los conflictos, las estrategias de resolución y las buenas prácticas. En este punto se profundiza en el patrón Repositorio y la unidad de trabajo en aplicaciones empresariales, que son la base del acceso a datos en la arquitectura limpia. Esta técnica se usará en el punto 5.10 para el logging y el diagnóstico.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

`cd AceriaData` → entra en la carpeta raíz del proyecto. `cd src/AceriaData.Console` → entra en la carpeta del proyecto de consola.

Paso 2: Crear la interfaz genérica IRepository

Crear el archivo `src/AceriaData.Application/Interfaces/IRepository.cs`:

```
namespace AceriaData.Application.Interfaces;

public interface IRepository<T> where T : class
{
    T? ObtenerPorId(int id);
    List<T> ObtenerTodas();
    void Agregar(T entidad);
    void Eliminar(T entidad);
}
```

Línea 1: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 3: `public interface IRepository where T : class` → declara la interfaz genérica.

Línea 5: `T? ObtenerPorId(int id);` → declara el método que obtiene por Id.

Línea 6: `List ObtenerTodas();` → declara el método que obtiene todas.

Línea 7: `void Agregar(T entidad);` → declara el método que agrega.

Línea 8: `void Eliminar(T entidad);` → declara el método que elimina.

Error común: si se olvida la restricción `where T : class`, EF Core no puede usar `context.Set()` porque requiere que `T` sea una clase.

Paso 3: Crear la implementación genérica Repositorio

Crear el archivo src/AceriaData.Infrastructure/Repositories/Repositorio.cs:

```
using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Repositories;

public class Repositorio<T> : IRepository<T> where T : class
{
    protected readonly AceriaDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public Repositorio(AceriaDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public virtual T? ObtenerPorId(int id)
    {
        return _dbSet.Find(id);
    }

    public virtual List<T> ObtenerTodas()
    {
        return _dbSet.ToList();
    }

    public virtual void Agregar(T entidad)
    {
        _dbSet.Add(entidad);
    }

    public virtual void Eliminar(T entidad)
    {
        _dbSet.Remove(entidad);
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public class Repositorio : IRepository where T : class → declara la implementación genérica.

Línea 9: protected readonly AceriaDbContext _context; → campo del DbContext.

Línea 10: protected readonly DbSet _dbSet; → campo del DbSet.

Línea 12: public Repositorio(AceriaDbContext context) → constructor.

Línea 14: _context = context; → asigna el DbContext.

Línea 15: _dbSet = context.Set(); → obtiene el DbSet.

Línea 18: public virtual T? ObtenerPorId(int id) → declara el método.

Línea 20: return _dbSet.Find(id); → busca por clave primaria.

Línea 23: public virtual List ObtenerTodas() → declara el método.

Línea 25: return _dbSet.ToList(); → materializa la consulta.

Línea 28: public virtual void Agregar(T entidad) → declara el método.

Línea 30: `_dbSet.Add(entidad);` → registra la entidad.

Línea 33: `public virtual void Eliminar(T entidad)` → declara el método.

Línea 35: `_dbSet.Remove(entidad);` → marca la entidad para eliminar.

Error común: si se usa `_context.Set()` en cada método en lugar de almacenarlo en el campo `_dbSet`, se realizan llamadas repetidas al contexto. Es más eficiente almacenarlo en el constructor.

| Paso 4: Refactorizar IOrdenRepositorio para heredar de IRepository

Modificar `src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio : IRepository<OrdenFabricacion>
{
    OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);
    List<OrdenFabricacion> ObtenerPendientes();
    List<OrdenFabricacion> ObtenerPorCliente(string cliente);
    List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado);
    List<OrdenResumenDto> ObtenerResumenes();
    int ContarOrdenes();
    bool ExisteAlgunaOrden();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude();
    List<OrdenFabricacion> ObtenerConAleacionesThenInclude();
    List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery();
    OrdenFabricacion? ObtenerPorIdParaActualizar(int id);
    bool ActualizarClienteConClienteGana(OrdenFabricacion orden, string nuevoCliente);
    bool InsertarDosOrdenesConTransaccion(string numero1, string numero2);
    void AplicarMigracionHasta(string nombreMigracion);
}
```

Línea 1: `using AceriaData.Application.Dtos;` → importa los DTOs.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 4: `namespace AceriaData.Application.Interfaces;` → declara el espacio de nombres.

Línea 6: `public interface IOrdenRepositorio : IRepository` → declara la interfaz específica que hereda de la genérica.

Línea 8: `OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden);` → declara el método.

Línea 9: `List ObtenerPendientes();` → declara el método.

Línea 10: `List ObtenerPorCliente(string cliente);` → declara el método.

Línea 11: `List ObtenerPorEstadoOrdenadasPorFecha(string estado);` → declara el método.

Línea 12: `List ObtenerResumenes();` → declara el método.

Línea 13: `int ContarOrdenes();` → declara el método.

Línea 14: `bool ExisteAlgunaOrden();` → declara el método.

Línea 15: `List ObtenerOrdenesConPlanchasInclude();` → declara el método.

Línea 16: `List ObtenerConAleacionesThenInclude();` → declara el método.

Línea 17: `List ObtenerConPlanchasYDetalleSplitQuery();` → declara el método.

Línea 18: `OrdenFabricacion? ObtenerPorIdParaActualizar(int id);` → declara el método.

Línea 19: `bool ActualizarClienteConClienteGana(OrdenFabricacion orden, string nuevoCliente);` → declara el método.

Línea 20: `bool InsertarDosOrdenesConTransaccion(string numero1, string numero2);` → declara el método.

Línea 21: `void AplicarMigracionHasta(string nombreMigracion);` → declara el método.

Error común: si se añaden demasiados métodos específicos al repositorio, se convierte en una envoltura de todas las consultas posibles. Se deben añadir solo los métodos que la capa de negocio necesita.

| Paso 5: Refactorizar OrdenRepositorio para heredar de Repositorio

Modificar `src/AceriaData.Infrastructure.Repositories/OrdenRepositorio.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;

namespace AceriaData.Infrastructure.Repositories;

public class OrdenRepositorio : Repositorio<OrdenFabricacion>, IOrdenRepositorio
{
    private readonly ILogger<OrdenRepositorio> _logger;

    public OrdenRepositorio(AceriaDbContext context, ILogger<OrdenRepositorio> logger) : base(context)
    {
        _logger = logger;
    }

    public OrdenFabricacion? ObtenerPorNumeroOrden(string numeroOrden)
    {
        return _context.OrdenesFabricacion
            .FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public List<OrdenFabricacion> ObtenerPendientes()
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == "Pendiente")
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorCliente(string cliente)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Cliente == cliente)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerPorEstadoOrdenadasPorFecha(string estado)
    {
        return _context.OrdenesFabricacion
            .Where(o => o.Estado == estado)
            .OrderByDescending(o => o.FechaCreacion)
            .ToList();
    }

    public List<OrdenResumenDto> ObtenerResumenes()
    {
        return _context.OrdenesFabricacion
            .AsNoTracking()
            .Select(o => new OrdenResumenDto
            {
                NumeroOrden = o.NumeroOrden,
                Cliente = o.Cliente,
                Estado = o.Estado,
                FechaCreacion = o.FechaCreacion
            })
            .ToList();
    }
}
```

```

    }

    public int ContarOrdenes()
    {
        return _context.OrdenesFabricacion.Count();
    }

    public bool ExisteAlgunaOrden()
    {
        return _context.OrdenesFabricacion.Any();
    }

    public List<OrdenFabricacion> ObtenerOrdenesConPlanchasInclude()
    {
        return _context.OrdenesFabricacion
            .AsNoTracking()
            .Include(o => o.Planchas)
            .OrderBy(o => o.NumeroOrden)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerConAleacionesThenInclude()
    {
        return _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.OrdenesAleaciones)
            .ThenInclude(oa => oa.Aleacion)
            .OrderBy(o => o.NumeroOrden)
            .ToList();
    }

    public List<OrdenFabricacion> ObtenerConPlanchasYDetalleSplitQuery()
    {
        return _context.OrdenesFabricacion
            .AsNoTrackingWithIdentityResolution()
            .Include(o => o.Planchas)
            .Include(o => o.Detalle)
            .AsSplitQuery()
            .OrderBy(o => o.NumeroOrden)
            .ToList();
    }

    public OrdenFabricacion? ObtenerPorIdParaActualizar(int id)
    {
        return _context.OrdenesFabricacion
            .FirstOrDefault(o => o.Id == id);
    }

    public bool ActualizarClienteConClienteGana(OrdenFabricacion orden, string nuevoCliente)
    {
        orden.Cliente = nuevoCliente;

        try
        {
            _context.SaveChanges();
            return true;
        }
        catch (DbUpdateConcurrencyException ex)
        {
            var entry = ex.Entries.First();
            var valoresBd = entry.GetDatabaseValues();

            if (valoresBd is null) return false;

            entry.OriginalValues.SetValues(valoresBd);
            _context.SaveChanges();
            return true;
        }
    }

    public bool InsertarDosOrdenesConTransaccion(string numero1, string numero2)
    {
        using var transaction = _context.Database.BeginTransaction();
    }

```

```

        try
        {
            _context.OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = numero1, Cliente =
"Norte", Estado = "Pendiente", FechaCreacion = DateTime.Now });
            _context.SaveChanges();

            _context.OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = numero2, Cliente =
"Sur", Estado = "Pendiente", FechaCreacion = DateTime.Now });
            _context.SaveChanges();

            transaction.Commit();
            return true;
        }
        catch (DbUpdateException ex)
        {
            transaction.Rollback();
            _logger.LogError(ex, "Error al insertar las órdenes");
            return false;
        }
    }

    public void AplicarMigracionHasta(string nombreMigracion)
    {
        var migrator =
        _context.Database.GetService<Microsoft.EntityFrameworkCore.Migrations.IMigrator>();
        migrator.Migrate(nombreMigracion);
    }
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 3: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 5: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 6: using Microsoft.Extensions.Logging; → importa el logging.

Línea 8: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 10: public class OrdenRepositorio : Repositorio, IOrdenRepositorio → declara la implementación específica que hereda de la genérica.

Línea 12: private readonly ILogger _logger; → campo del logger.

Línea 14: public OrdenRepositorio(AceriaDbContext context, ILogger logger) : base(context) → constructor.

Línea 16: _logger = logger; → asigna el logger.

Error común: si el repositorio específico llama a _context.SaveChanges(), rompe el patrón de unidad de trabajo. El repositorio no debe guardar cambios: eso lo hace la unidad de trabajo.

| Paso 6: Refactorizar UnidadDeTrabajo con IDbContextFactory

Modificar src/AceriaData.Infrastructure.Repositories/UnidadDeTrabajo.cs:

```

using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    private IPlanchaRepositorio? _planchas;
    private IAleacionRepositorio? _aleaciones;
    private IDetalleOrdenRepositorio? _detalles;

    public UnidadDeTrabajo(AceriaDbContext context)
    {
        _context = context;
    }

    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context,
    NullLogger<OrdenRepositorio>.Instance);
    public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context);
    public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context);
    public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context);

    public int Guardar()
    {
        return _context.SaveChanges();
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 4: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 6: public class UnidadDeTrabajo : IUnidadDeTrabajo → declara la implementación.

Línea 8: private readonly AceriaDbContext _context; → campo del DbContext.

Línea 9: private IOrdenRepositorio? _ordenes; → campo del repositorio de órdenes.

Línea 10: private IPlanchaRepositorio? _planchas; → campo del repositorio de planchas.

Línea 11: private IAleacionRepositorio? _aleaciones; → campo del repositorio de aleaciones.

Línea 12: private IDetalleOrdenRepositorio? _detalles; → campo del repositorio de detalles.

Línea 14: public UnidadDeTrabajo(AceriaDbContext context) → constructor.

Línea 16: _context = context; → asigna el parámetro al campo.

Línea 19: public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context, NullLogger.Instance); → expone el repositorio de órdenes.

Línea 20: public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context); → expone el repositorio de planchas.

Línea 21: public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context); → expone el repositorio de aleaciones.

Línea 22: public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context); → expone el repositorio de detalles.

Línea 24: public int Guardar() → declara el método.

Línea 26: `return _context.SaveChanges();` → ejecuta `SaveChanges`.

Línea 29: `public void Dispose()` → declara el método.

Línea 31: `_context.Dispose();` → libera el `DbContext`.

Error común: si la unidad de trabajo crea los repositorios en el constructor en lugar de usar inicialización perezosa, se crean todos los repositorios aunque no se usen.

| Paso 7: Crear un servicio de larga duración con `IDbContextFactory`

Crear el archivo `src/AceriaData.Infrastructure.Services/ProcesadorOrdenes.cs`:

```
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Hosting;

namespace AceriaData.Infrastructure.Services;

public class ProcesadorOrdenes : BackgroundService
{
    private readonly IDbContextFactory<AceriaDbContext> _factory;

    public ProcesadorOrdenes(IDbContextFactory<AceriaDbContext> factory)
    {
        _factory = factory;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            try
            {
                using var context = await _factory.CreateDbContextAsync(stoppingToken);
                using var unidad = new UnidadDeTrabajo(context);

                var pendientes = unidad.Ordenes.ObtenerPendientes();
                Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");

                await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Error en el procesador: {ex.Message}");
                await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);
            }
        }
    }
}
```

Línea 1: `using AceriaData.Infrastructure.Persistence;` → importa el `DbContext`.

Línea 2: `using AceriaData.Infrastructure.Repositories;` → importa la unidad de trabajo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa EF Core.

Línea 4: `using Microsoft.Extensions.Hosting;` → importa el host.

Línea 6: `namespace AceriaData.Infrastructure.Services;` → declara el espacio de nombres.

Línea 8: `public class ProcesadorOrdenes : BackgroundService` → declara el servicio.

Línea 10: `private readonly IDbContextFactory _factory;` → campo de la fábrica.

Línea 12: `public ProcesadorOrdenes(IDbContextFactory factory)` → constructor.

Línea 14: `_factory = factory;` → asigna la fábrica.

Línea 17: `protected override async Task ExecuteAsync(CancellationToken stoppingToken)` → declara el método.

Línea 19: `while (!stoppingToken.IsCancellationRequested)` → bucle hasta la cancelación.

Línea 21: `try` → inicio del bloque.

Línea 23: `using var context = await _factory.CreateDbContextAsync(stoppingToken);` → crea un contexto desde la fábrica.

Línea 24: `using var unidad = new UnidadDeTrabajo(context);` → crea la unidad de trabajo.

Línea 26: `var pendientes = unidad Ordenes.ObtenerPendientes();` → obtiene las órdenes pendientes.

Línea 27: `Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");` → muestra el número.

Línea 29: `await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);` → espera cinco segundos.

Línea 31: `catch (Exception ex)` → captura la excepción.

Línea 33: `Console.WriteLine($"Error en el procesador: {ex.Message}");` → muestra el mensaje.

Línea 34: `await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);` → espera diez segundos.

Error común: si se inyecta el `DbContext` directamente en el `BackgroundService`, el contexto vive más que el ámbito para el que fue creado y se producen errores. Se debe usar `IDbContextFactory`.

Paso 8: Registrar los servicios en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddDbContextFactory<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sqlOptions =>
            {
                sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                sqlOptions.CommandTimeout(60);
                sqlOptions.MigrationsAssembly("AceriaData.Infrastructure");
                sqlOptions.MigrationsHistoryTable("__AceriaMigraciones");
            }
        ));

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IPlanchaRepositorio, PlanchaRepositorio>();
services.AddScoped<IAleacionRepositorio, AleacionRepositorio>();
services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();
services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
services.AddHostedService<ProcesadorOrdenes>();
```

Línea 1: `services.AddDbContextFactory(options =>` → registra la fábrica de `DbContext`.

Línea 2: `options` → objeto de configuración.

Línea 3: `.UseSqlServer(connectionString, sqlOptions =>` → registra el proveedor.

Línea 4: `{` → inicio del bloque.

Línea 5: `sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);` → habilita los reintentos.

Línea 6: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera.

Línea 7: `sqlOptions.MigrationsAssembly("AceriaData.Infrastructure");` → especifica el ensamblado de migraciones.

Línea 8: `sqlOptions.MigrationsHistoryTable("__AceriaMigraciones");` → especifica el nombre de la tabla de historial.

Línea 9: `});` → cierra el bloque.

Línea 11: `services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();` → registra el repositorio de órdenes.

Línea 12: `services.AddScoped<IPlanchaRepositorio, PlanchaRepositorio>();` → registra el repositorio de planchas.

Línea 13: `services.AddScoped<IAleacionRepositorio, AleacionRepositorio>();` → registra el repositorio de aleaciones.

Línea 14: `services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();` → registra el repositorio de detalles.

Línea 15: `services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();` → registra la unidad de trabajo.

Línea 16: `services.AddHostedService();` → registra el servicio de larga duración.

Error común: si se registra el `DbContext` con `AddDbContext` en lugar de `AddDbContextFactory`, el `BackgroundService` no puede resolver el contexto porque su ciclo de vida es `Scoped` y el servicio es `Singleton`. Se debe usar `AddDbContextFactory`.

▮ Paso 9: Crear el caso de uso de patrón Repositorio y Unidad de Trabajo

Crear el archivo `src/AceriaData.Application/UseCases/RepositorioUnidadTrabajoUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.UseCases;

public class RepositorioUnidadTrabajoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public RepositorioUnidadTrabajoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PATRÓN REPOSITORIO Y UNIDAD DE TRABAJO ===");

        DemostrarRepositorioGenerico();
        DemostrarRepositorioEspecifico();
        DemostrarUnidadDeTrabajo();
        CompararRendimientoConYSinRepositorio();
        CompararMemoriaConYSinRepositorio();
        MostrarAntiPatrones();
    }

    private void DemostrarRepositorioGenerico()
    {
        Console.WriteLine("\n--- Repositorio genérico ---");

        var todas = _unidad.Ordenes.ObtenerTodas();
        var porId = _unidad.Ordenes.ObtenerPorId(1);

        Console.WriteLine($"Todas las órdenes: {todas.Count}");
        Console.WriteLine($"Orden con Id 1: {porId?.NumeroOrden}");
    }

    private void DemostrarRepositorioEspecifico()
    {
        Console.WriteLine("\n--- Repositorio específico ---");

        var pendientes = _unidad.Ordenes.ObtenerPendientes();
        var resúmenes = _unidad.Ordenes.ObtenerResúmenes();
        var total = _unidad.Ordenes.ContarOrdenes();
    }
}
```

```

        Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");
        Console.WriteLine($"Resúmenes: {resúmenes.Count}");
        Console.WriteLine($"Total de órdenes: {total}");
    }

    private void DemostrarUnidadDeTrabajo()
    {
        Console.WriteLine("\n--- Unidad de trabajo ---");

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-UOW-001",
            Cliente = "Constructora Unidad",
            Estado = "Pendiente",
            FechaCreacion = DateTime.Now
        };

        _unidad.Ordenes.Agregar(orden);

        var plancha = new PlanchaAcero
        {
            Orden = orden,
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 370.5m,
            Activa = true
        };

        _unidad.Planchas.Agregar(plancha);

        var filas = _unidad.Guardar();
        Console.WriteLine($"Filas afectadas: {filas}");
        Console.WriteLine($"Orden creada: {orden.NumeroOrden} con Id {orden.Id}");
    }

    private void CompararRendimientoConYSinRepositorio()
    {
        Console.WriteLine("\n--- Comparación de rendimiento con y sin repositorio ---");

        var cronometroSin = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            var ordenes = _unidad.Ordenes.ObtenerPendientes();
        }
        cronometroSin.Stop();

        var cronometroCon = Stopwatch.StartNew();
        for (int i = 0; i < 100; i++)
        {
            var ordenes = _unidad.Ordenes.ObtenerPendientes();
        }
        cronometroCon.Stop();

        Console.WriteLine($"100 consultas con repositorio: {cronometroSin.ElapsedMilliseconds} ms");
        Console.WriteLine($"100 consultas con repositorio (mismo método): {cronometroCon.ElapsedMilliseconds} ms");
    }

    private void CompararMemoriaConYSinRepositorio()
    {
        Console.WriteLine("\n--- Comparación de memoria con y sin repositorio ---");

        GC.Collect();
        GC.WaitForPendingFinalizers();
        GC.Collect();

        var memoriaAntes = GC.GetTotalMemory(true);

        for (int i = 0; i < 1000; i++)
        {
            var repositorio = _unidad.Ordenes;
            var total = repositorio.ContarOrdenes();
        }
    }

```

```

    }

    GC.Collect();
    GC.WaitForPendingFinalizers();
    GC.Collect();

    var memoriaDespues = GC.GetTotalMemory(true);
    var diferencia = (memoriaDespues - memoriaAntes) / 1024;

    Console.WriteLine($"Memoria antes: {memoriaAntes / 1024} KB");
    Console.WriteLine($"Memoria después: {memoriaDespues / 1024} KB");
    Console.WriteLine($"Diferencia: {diferencia} KB");
}

private void MostrarAntiPatrones()
{
    Console.WriteLine("\n--- Anti-patrones del patrón Repositorio ---");
    Console.WriteLine("1. Exponer IQueryable en lugar de listas.");
    Console.WriteLine("2. Exponer operaciones de EF Core (Include, AsNoTracking) en la interfaz.");
    Console.WriteLine("3. Repositorio genérico excesivo con métodos para todas las operaciones.");
    Console.WriteLine("4. Repositorio que llama a SaveChanges en lugar de la unidad de trabajo.");
    Console.WriteLine("5. Repositorio que devuelve entidades desconectadas sin control.");
    Console.WriteLine();
    Console.WriteLine("Buenas prácticas:");
    Console.WriteLine("1. Devolver listas o entidades concretas.");
    Console.WriteLine("2. Encapsular las consultas específicas del dominio.");
    Console.WriteLine("3. Añadir solo los métodos que la capa de negocio necesita.");
    Console.WriteLine("4. El repositorio no guarda: la unidad de trabajo coordina.");
    Console.WriteLine("5. Documentar las decisiones de acceso a datos.");
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces.

Línea 3: using AceriaData.Domain.Entities; → importa las entidades.

Línea 5: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 7: public class RepositorioUnidadTrabajoUseCase → declara el caso de uso.

Línea 9: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 11: public RepositorioUnidadTrabajoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 13: _unidad = unidad; → asigna el parámetro al campo.

Línea 16: public void Ejecutar() → declara el método principal.

Línea 18: Console.WriteLine("=== PATRÓN REPOSITORIO Y UNIDAD DE TRABAJO ==="); → muestra la cabecera.

Línea 20: DemostrarRepositorioGenerico(); → llama al método de repositorio genérico.

Línea 21: DemostrarRepositorioEspecifico(); → llama al método de repositorio específico.

Línea 22: DemostrarUnidadDeTrabajo(); → llama al método de unidad de trabajo.

Línea 23: CompararRendimientoConYSinRepositorio(); → llama al método de comparación de rendimiento.

Línea 24: CompararMemoriaConYSinRepositorio(); → llama al método de comparación de memoria.

Línea 25: MostrarAntiPatrones(); → llama al método de anti-patrones.

Línea 28: private void DemostrarRepositorioGenerico() → declara el método.

Línea 30: Console.WriteLine("\n--- Repositorio genérico ---"); → muestra la cabecera.

Línea 32: var todas = _unidad Ordenes.ObtenerTodas(); → llama al método genérico.

Línea 33: `var porId = _unidad Ordenes.ObtenerPorId(1);` → llama al método genérico.

Línea 35: `Console.WriteLine($"Todas las órdenes: {todas.Count}");` → muestra el número.

Línea 36: `Console.WriteLine($"Orden con Id 1: {porId?.NumeroOrden}");` → muestra el número de orden.

Línea 39: `private void DemostrarRepositorioEspecifico()` → declara el método.

Línea 41: `Console.WriteLine("\n--- Repositorio específico ---");` → muestra la cabecera.

Línea 43: `var pendientes = _unidad Ordenes.ObtenerPendientes();` → llama al método específico.

Línea 44: `var resúmenes = _unidad Ordenes.ObtenerResúmenes();` → llama al método específico.

Línea 45: `var total = _unidad Ordenes.ContarOrdenes();` → llama al método específico.

Línea 47: `Console.WriteLine($"Órdenes pendientes: {pendientes.Count}");` → muestra el número.

Línea 48: `Console.WriteLine($"Resúmenes: {resúmenes.Count}");` → muestra el número.

Línea 49: `Console.WriteLine($"Total de órdenes: {total}");` → muestra el total.

Línea 52: `private void DemostrarUnidadDeTrabajo()` → declara el método.

Línea 54: `Console.WriteLine("\n--- Unidad de trabajo ---");` → muestra la cabecera.

Línea 56: `var orden = new OrdenFabricacion` → crea la orden.

Línea 64: `_unidad Ordenes.Agregar(orden);` → agrega la orden.

Línea 66: `var plancha = new PlanchaAcero` → crea la plancha.

Línea 76: `_unidad.Planchas.Agregar(plancha);` → agrega la plancha.

Línea 78: `var filas = _unidad.Guardar();` → guarda los cambios.

Línea 79: `Console.WriteLine($"Filas afectadas: {filas}");` → muestra el número de filas.

Línea 80: `Console.WriteLine($"Orden creada: {orden.NumeroOrden} con Id {orden.Id}");` → muestra la orden creada.

Línea 83: `private void CompararRendimientoConYSinRepositorio()` → declara el método.

Línea 85: `Console.WriteLine("\n--- Comparación de rendimiento con y sin repositorio ---");` → muestra la cabecera.

Línea 87: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 88: `for (int i = 0; i < 100; i++)` → repite cien veces.

Línea 90: `var ordenes = _unidad Ordenes.ObtenerPendientes();` → llama al método del repositorio.

Línea 92: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 94: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 95: `for (int i = 0; i < 100; i++)` → repite cien veces.

Línea 97: `var ordenes = _unidad Ordenes.ObtenerPendientes();` → llama al método del repositorio.

Línea 99: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 101: `Console.WriteLine($"100 consultas con repositorio: {cronometroSin.ElapsedMilliseconds} ms");` → muestra el tiempo.

Línea 102: `Console.WriteLine($"100 consultas con repositorio (mismo método): {cronometroCon.ElapsedMilliseconds} ms");` → muestra el tiempo.

Línea 105: `private void CompararMemoriaConYSinRepositorio()` → declara el método.

Línea 107: `Console.WriteLine("\n--- Comparación de memoria con y sin repositorio ---");` → muestra la cabecera.

Línea 109: `GC.Collect();` → recolecta la basura.

Línea 110: `GC.WaitForPendingFinalizers();` → espera a los finalizadores.

Línea 111: `GC.Collect();` → recolecta de nuevo.

Línea 113: `var memoriaAntes = GC.GetTotalMemory(true);` → obtiene la memoria antes.

Línea 115: `for (int i = 0; i < 1000; i++)` → repite mil veces.

Línea 117: `var repositorio = _unidad Ordenes;` → obtiene el repositorio.

Línea 118: `var total = repositorio.ContarOrdenes();` → cuenta las órdenes.

Línea 121: `GC.Collect();` → recolecta la basura.

Línea 122: `GC.WaitForPendingFinalizers();` → espera a los finalizadores.

Línea 123: `GC.Collect();` → recolecta de nuevo.

Línea 125: `var memoriaDespues = GC.GetTotalMemory(true);` → obtiene la memoria después.

Línea 126: `var diferencia = (memoriaDespues - memoriaAntes) / 1024;` → calcula la diferencia.

Línea 128: `Console.WriteLine($"Memoria antes: {memoriaAntes / 1024} KB");` → muestra la memoria antes.

Línea 129: `Console.WriteLine($"Memoria después: {memoriaDespues / 1024} KB");` → muestra la memoria después.

Línea 130: `Console.WriteLine($"Diferencia: {diferencia} KB");` → muestra la diferencia.

Línea 133: `private void MostrarAntiPatrones()` → declara el método.

Línea 135: `Console.WriteLine("\n--- Anti-patrones del patrón Repositorio ---");` → muestra la cabecera.

Línea 136: `Console.WriteLine("1. Exponer IQueryable en lugar de listas.");` → describe el primer anti-patrón.

Línea 137: `Console.WriteLine("2. Exponer operaciones de EF Core (Include, AsNoTracking) en la interfaz.");` → describe el segundo.

Línea 138: `Console.WriteLine("3. Repositorio genérico excesivo con métodos para todas las operaciones.");` → describe el tercero.

Línea 139: `Console.WriteLine("4. Repositorio que llama a SaveChanges en lugar de la unidad de trabajo.");` → describe el cuarto.

Línea 140: `Console.WriteLine("5. Repositorio que devuelve entidades desconectadas sin control.");` → describe el quinto.

Línea 142: `Console.WriteLine("Buenas prácticas.");` → muestra la cabecera de buenas prácticas.

Línea 143: `Console.WriteLine("1. Devolver listas o entidades concretas.");` → describe la primera.

Línea 144: `Console.WriteLine("2. Encapsular las consultas específicas del dominio.");` → describe la segunda.

Línea 145: `Console.WriteLine("3. Añadir solo los métodos que la capa de negocio necesita.");` → describe la tercera.

Línea 146: `Console.WriteLine("4. El repositorio no guarda: la unidad de trabajo coordina.");` → describe la cuarta.

Línea 147: `Console.WriteLine("5. Documentar las decisiones de acceso a datos.");` → describe la quinta.

Error común: si se llama a Guardar varias veces, se ejecutan varias transacciones. Se debe llamar una sola vez al final.

Paso 10: Registrar el caso de uso en el contenedor

Modificar src/AceriaData.Console/Program.cs:

```
services.AddScoped<RepositorioUnidadTrabajoUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 11: Insertar datos de prueba iniciales

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 3, 10) }
    };

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... } → tercera orden.

Línea 12: }; → cierra la lista.

Línea 14: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 15: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 12: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados del repositorio genérico, el repositorio específico, la unidad de trabajo, la comparación de rendimiento, la comparación de memoria y los anti-patrones.

Paso 13: Analizar la salida

La salida del programa muestra información como la siguiente:

```

=== PATRÓN REPOSITORIO Y UNIDAD DE TRABAJO ===

--- Repositorio genérico ---
Todas las órdenes: 3
Orden con Id 1: OF-2024-0001

--- Repositorio específico ---
Órdenes pendientes: 2
Resúmenes: 3
Total de órdenes: 3

--- Unidad de trabajo ---
Filas afectadas: 2
Orden creada: OF-UOW-001 con Id 4

--- Comparación de rendimiento con y sin repositorio ---
100 consultas con repositorio: 220 ms
100 consultas con repositorio (mismo método): 215 ms

--- Comparación de memoria con y sin repositorio ---
Memoria antes: 8500 KB
Memoria después: 8520 KB
Diferencia: 20 KB

--- Anti-patrones del patrón Repositorio ---
1. Exponer IQueryable en lugar de listas.
2. Exponer operaciones de EF Core (Include, AsNoTracking) en la interfaz.
3. Repositorio genérico excesivo con métodos para todas las operaciones.
4. Repositorio que llama a SaveChanges en lugar de la unidad de trabajo.
5. Repositorio que devuelve entidades desconectadas sin control.

Buenas prácticas:
1. Devolver listas o entidades concretas.
2. Encapsular las consultas específicas del dominio.
3. Añadir solo los métodos que la capa de negocio necesita.
4. El repositorio no guarda: la unidad de trabajo coordina.
5. Documentar las decisiones de acceso a datos.
La primera sección muestra el repositorio genérico. La segunda sección muestra el repositorio específico. La tercera sección muestra la unidad de trabajo. La cuarta sección muestra la comparación de rendimiento. La quinta sección muestra la comparación de memoria. La sexta sección muestra los anti-patrones.

Observaciones: el repositorio genérico proporciona los métodos comunes. El repositorio específico añade métodos del dominio. La unidad de trabajo coordina ambos repositorios en una sola transacción. La diferencia de rendimiento entre repositorio y uso directo es de 5 ms en cien consultas, aproximadamente 0,05 ms por consulta. La diferencia de memoria es de 20 KB en mil operaciones, aproximadamente 20 bytes por operación.

```

Paso 14: Diagnosticar un error común

Modificar el repositorio para exponer IQueryable:

```

public interface IOrdenRepositorio : IRepository<OrdenFabricacion>
{
    IQueryable<OrdenFabricacion> ObtenerQueryable();
}

```

Resultado esperado: la interfaz expone IQueryable, lo que rompe la abstracción. La capa de negocio puede añadir filtros y ordenaciones que el repositorio no controla.

Solución: devolver listas o entidades concretas en lugar de IQueryable.

```
public interface IOordenRepositorio : IRepository<OrdenFabricacion>
{
    List<OrdenFabricacion> ObtenerPendientes();
    List<OrdenFabricacion> ObtenerPorCliente(string cliente);
}
```

Resultado esperado con la solución: la interfaz devuelve listas y mantiene la abstracción.

Paso 15: Crear un test con Moq

Crear el archivo tests/AceriaData.Tests/Repositories/OrdenRepositorioMockTests.cs:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Moq;

namespace AceriaData.Tests.Repositories;

public class OrdenRepositorioMockTests
{
    [Fact]
    public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes()
    {
        var mockRepositorio = new Mock<IOordenRepositorio>();
        mockRepositorio.Setup(r => r.ObtenerTodas()).Returns(new List<OrdenFabricacion>
        {
            new OrdenFabricacion { NumeroOrden = "OF-001", Estado = "Pendiente" },
            new OrdenFabricacion { NumeroOrden = "OF-002", Estado = "EnProceso" },
            new OrdenFabricacion { NumeroOrden = "OF-003", Estado = "Pendiente" }
        });

        var ordenes = mockRepositorio.Object.ObtenerTodas().Where(o => o.Estado ==
"Pendiente").ToList();

        Assert.Equal(2, ordenes.Count);
    }

    [Fact]
    public void Agregar_ConOrdenValida_LlamaAlMetodoDelRepositorio()
    {
        var mockRepositorio = new Mock<IOordenRepositorio>();
        var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Estado = "Pendiente" };

        mockRepositorio.Object.Agregar(orden);

        mockRepositorio.Verify(r => r.Agregar(orden), Times.Once);
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using Moq; → importa Moq.

Línea 5: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 7: public class OrdenRepositorioMockTests → declara la clase de tests.

Línea 9: [Fact] → atributo de test.

Línea 10: public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes() → declara el test.

Línea 12: `var mockRepositorio = new Mock();` → crea el mock.

Línea 13: `mockRepositorio.Setup(r => r.ObtenerTodas()).Returns(new List` → configura el método.

Línea 19: `var ordenes = mockRepositorio.Object.ObtenerTodas().Where(o => o.Estado == "Pendiente").ToList();` → llama al método.

Línea 21: `Assert.Equal(2, ordenes.Count);` → comprueba el número.

Línea 24: `[Fact]` → atributo de test.

Línea 25: `public void Agregar_ConOrdenValida_LlamaAlMetodoDelRepositorio()` → declara el test.

Línea 27: `var mockRepositorio = new Mock();` → crea el mock.

Línea 28: `var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Estado = "Pendiente" };` → crea la orden.

Línea 30: `mockRepositorio.Object.Agregar(orden);` → llama al método.

Línea 32: `mockRepositorio.Verify(r => r.Agregar(orden), Times.Once);` → comprueba que se llamó una vez.

Error común: si no se instala el paquete Moq, el código no compila. Se debe añadir con `dotnet add package Moq`.

Paso 16: Ejecutar los tests

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

`dotnet test` → ejecuta los tests del proyecto.

Resultado esperado: todos los tests pasan.

La salida del comando muestra información como la siguiente:

```
Passed! - Failed: 0, Passed: 2, Skipped: 0, Total: 2, Duration: 1 s
Observaciones: los tests con Moq no necesitan la base de datos. Verifican el comportamiento del
repositorio de forma aislada.
```

Errores comunes del ejercicio

Error Causa Solución Repositorio llama a SaveChanges Rompe la unidad de trabajo El repositorio no guarda: solo la unidad de trabajo Repositorio expone IQueryable Rompe la abstracción Devolver listas o entidades concretas Repositorio expone Include Acopla la capa de negocio a EF Core Usar métodos específicos del dominio Registro como Singleton Problemas de concurrencia Registrar con Scoped DbContext compartido entre ámbitos Datos obsoletos Usar CreateScope para cada operación Olvidar Guardar Los cambios no se persisten Llamar a unidad.Guardar() Repositorio genérico demasiado amplio Se convierte en una envoltura de DbSet Añadir solo los métodos necesarios BackgroundService con DbContext inyectado El contexto vive más que el ámbito Usar IDbContextFactory Mock sin configurar Los métodos devuelven null Configurar con Setup Moq no instalado Falta el paquete Añadir con `dotnet add package Moq`

Reto resuelto: Añadir un repositorio específico para DetalleOrden con mocking

Reto: Crear la interfaz `IDetalleOrdenRepositorio` y su implementación `DetalleOrdenRepositorio`. Añadirla a la unidad de trabajo. Crear un test con Moq que verifique el método `ObtenerPorOrden`. Verificar que funciona con un caso de uso.

Solución paso a paso:

Paso 1: Crear la interfaz IDetalleOrdenRepositorio:

```
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IDetalleOrdenRepositorio : IRepository<DetalleOrden>
{
    DetalleOrden? ObtenerPorOrden(int ordenId);
    List<DetalleOrden> ObtenerConTemperaturaMayorA(double temperatura);
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 5: public interface IDetalleOrdenRepositorio : IRepository → declara la interfaz específica.

Línea 7: DetalleOrden? ObtenerPorOrden(int ordenId); → declara el método.

Línea 8: List ObtenerConTemperaturaMayorA(double temperatura); → declara el método.

Paso 2: Crear la implementación DetalleOrdenRepositorio:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;

namespace AceriaData.Infrastructure.Repositories;

public class DetalleOrdenRepositorio : Repositorio<DetalleOrden>, IDetalleOrdenRepositorio
{
    public DetalleOrdenRepositorio(AceriaDbContext context) : base(context) { }

    public DetalleOrden? ObtenerPorOrden(int ordenId)
    {
        return _context.DetallesOrden
            .FirstOrDefault(d => d.OrdenId == ordenId);
    }

    public List<DetalleOrden> ObtenerConTemperaturaMayorA(double temperatura)
    {
        return _context.DetallesOrden
            .Where(d => d.TemperaturaColada > temperatura)
            .ToList();
    }
}
```

Línea 1: using AceriaData.Application.Interfaces; → importa la interfaz.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 5: namespace AceriaData.Infrastructure.Repositories; → declara el espacio de nombres.

Línea 7: public class DetalleOrdenRepositorio : Repositorio, IDetalleOrdenRepositorio → declara la implementación.

Línea 9: public DetalleOrdenRepositorio(AceriaDbContext context) : base(context) { } → constructor.

Línea 11: public DetalleOrden? ObtenerPorOrden(int ordenId) → declara el método.

Línea 13: return _context.DetallesOrden → inicia la consulta.

Línea 14: .FirstOrDefault(d => d.OrdenId == ordenId); → filtra por orden.

Línea 17: public List ObtenerConTemperaturaMayorA(double temperatura) → declara el método.

Línea 19: `return _context.DetallesOrden` → inicia la consulta.

Línea 20: `.Where(d => d.TemperaturaColada > temperatura)` → filtra por temperatura.

Línea 21: `.ToList();` → materializa la consulta.

Paso 3: Añadir el repositorio a la unidad de trabajo:

```
public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    IPlanchaRepositorio Planchas { get; }
    IAleacionRepositorio Aleaciones { get; }
    IDetalleOrdenRepositorio Detalles { get; }
    int Guardar();
}
```

Línea 1: `public interface IUnidadDeTrabajo : IDisposable` → declara la interfaz.

Línea 3: `IOrdenRepositorio Ordenes { get; }` → expone el repositorio de órdenes.

Línea 4: `IPlanchaRepositorio Planchas { get; }` → expone el repositorio de planchas.

Línea 5: `IAleacionRepositorio Aleaciones { get; }` → expone el repositorio de aleaciones.

Línea 6: `IDetalleOrdenRepositorio Detalles { get; }` → expone el repositorio de detalles.

Línea 7: `int Guardar();` → declara el método que guarda los cambios.

Paso 4: Implementar el repositorio en la unidad de trabajo:

```
public class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;
    private IPlanchaRepositorio? _planchas;
    private IAleacionRepositorio? _aleaciones;
    private IDetalleOrdenRepositorio? _detalles;

    public UnidadDeTrabajo(AceriaDbContext context)
    {
        _context = context;
    }

    public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context,
NullLogger<OrdenRepositorio>.Instance);
    public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context);
    public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context);
    public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context);

    public int Guardar() => _context.SaveChanges();

    public void Dispose() => _context.Dispose();
}
```

Línea 1: `public class UnidadDeTrabajo : IUnidadDeTrabajo` → declara la implementación.

Línea 3: `private readonly AceriaDbContext _context;` → campo del DbContext.

Línea 4: `private IOrdenRepositorio? _ordenes;` → campo del repositorio de órdenes.

Línea 5: `private IPlanchaRepositorio? _planchas;` → campo del repositorio de planchas.

Línea 6: `private IAleacionRepositorio? _aleaciones;` → campo del repositorio de aleaciones.

Línea 7: `private IDetalleOrdenRepositorio? _detalles;` → campo del repositorio de detalles.

Línea 9: `public UnidadDeTrabajo(AceriaDbContext context)` → constructor.

Línea 11: `_context = context;` → asigna el parámetro al campo.

Línea 14: `public IOrdenRepositorio Ordenes => _ordenes ??= new OrdenRepositorio(_context, NullLogger.Instance);` → expone el repositorio de órdenes.

Línea 15: `public IPlanchaRepositorio Planchas => _planchas ??= new PlanchaRepositorio(_context);` → expone el repositorio de planchas.

Línea 16: `public IAleacionRepositorio Aleaciones => _aleaciones ??= new AleacionRepositorio(_context);` → expone el repositorio de aleaciones.

Línea 17: `public IDetalleOrdenRepositorio Detalles => _detalles ??= new DetalleOrdenRepositorio(_context);` → expone el repositorio de detalles.

Línea 19: `public int Guardar() => _context.SaveChanges();` → guarda los cambios.

Línea 21: `public void Dispose() => _context.Dispose();` → libera el DbContext.

| Paso 5: Registrar el repositorio en el contenedor:

```
services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();
```

Línea 1: `services.AddScoped<IDetalleOrdenRepositorio, DetalleOrdenRepositorio>();` → registra el repositorio.

| Paso 6: Crear el test con Moq:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Moq;

namespace AceriaData.Tests.Repositories;

public class DetalleOrdenRepositorioMockTests
{
    [Fact]
    public void ObtenerPorOrden_ConDetalleExistente_DevuelveElDetalle()
    {
        var mockRepositorio = new Mock<IDetalleOrdenRepositorio>();
        mockRepositorio.Setup(r => r.ObtenerPorOrden(1)).Returns(new DetalleOrden
        {
            Id = 1,
            OrdenId = 1,
            ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
            TemperaturaColada = 1550.5
        });

        var detalle = mockRepositorio.Object.ObtenerPorOrden(1);

        Assert.NotNull(detalle);
        Assert.Equal(1550.5, detalle.TemperaturaColada);
    }

    [Fact]
    public void ObtenerPorOrden_ConOrdenInexistente_DevuelveNull()
    {
        var mockRepositorio = new Mock<IDetalleOrdenRepositorio>();
        mockRepositorio.Setup(r => r.ObtenerPorOrden(99999)).Returns((DetalleOrden?)null);

        var detalle = mockRepositorio.Object.ObtenerPorOrden(99999);

        Assert.Null(detalle);
    }
}
```

Línea 1: `using AceriaData.Application.Interfaces;` → importa la interfaz.

Línea 2: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 3: `using Moq;` → importa Moq.

Línea 5: `namespace AceriaData.Tests.Repositories;` → declara el espacio de nombres.

Línea 7: `public class DetalleOrdenRepositorioMockTests` → declara la clase de tests.

Línea 9: `[Fact]` → atributo de test.

Línea 10: `public void ObtenerPorOrden_ConDetalleExistente_DevuelveElDetalle()` → declara el test.

Línea 12: `var mockRepositorio = new Mock();` → crea el mock.

Línea 13: `mockRepositorio.Setup(r => r.ObtenerPorOrden(1)).Returns(new DetalleOrden` → configura el método.

Línea 21: `var detalle = mockRepositorio.Object.ObtenerPorOrden(1);` → llama al método.

Línea 23: `Assert.NotNull(detalle);` → comprueba que existe.

Línea 24: `Assert.Equal(1550.5, detalle.TemperaturaColada);` → comprueba la temperatura.

Línea 27: `[Fact]` → atributo de test.

Línea 28: `public void ObtenerPorOrden_ConOrdenInexistente_DevuelveNull()` → declara el test.

Línea 30: `var mockRepositorio = new Mock();` → crea el mock.

Línea 31: `mockRepositorio.Setup(r => r.ObtenerPorOrden(99999)).Returns((DetalleOrden?)null);` → configura el método.

Línea 33: `var detalle = mockRepositorio.Object.ObtenerPorOrden(99999);` → llama al método.

Línea 35: `Assert.Null(detalle);` → comprueba que es nulo.

| Paso 7: Ejecutar los tests:

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

Resultado esperado: todos los tests pasan.

| Analogía final

El patrón Repositorio y la unidad de trabajo en una acería son como la organización de las ventanillas y el parte de producción. Cada repositorio es una ventanilla especializada: una para órdenes, otra para planchas, otra para aleaciones. La unidad de trabajo es el parte de producción que coordina todas las ventanillas del turno.

El operario no va al almacén directamente: se acerca a la ventanilla y pide lo que necesita. La ventanilla sabe dónde está cada cosa. El parte de producción agrupa todas las operaciones del turno y las confirma al final. Si algo falla, el parte se cancela y ninguna operación se aplica. El repositorio no guarda cambios: eso lo hace el parte.

El `IDbContextFactory` es como un almacén que abre y cierra la puerta cada vez que un operario necesita entrar. No mantiene la puerta abierta durante todo el turno. Cada operario abre la puerta, hace su trabajo y la cierra.

La comparación no debe presentar un overhead fijo como conclusión universal. `DbContext` ya implementa conceptos propios de Unit of Work y Repository; `AceriaData` mantiene una abstracción adicional como decisión arquitectónica deliberada para aislar `Application` y facilitar determinados tests. Si se mide rendimiento o memoria, el resultado se registra como observación local y con un benchmark que compare caminos realmente distintos.

Los anti-patrones son los errores que se deben evitar: exponer IQueryable, exponer operaciones de EF Core, usar repositorios genéricos excesivos, llamar a SaveChanges en el repositorio y devolver entidades desconectadas sin control.

Así funciona el patrón Repositorio y la unidad de trabajo en EF Core: se encapsula el acceso a datos y se coordinan las operaciones en una sola transacción.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado la interfaz genérica IRepositoryo.

Creado la implementación genérica Repositorio.

Refactorizado IOrdenRepositorio para heredar de IRepositoryo.

Refactorizado OrdenRepositorio para heredar de Repositorio con logger.

Refactorizado UnidadDeTrabajo con inicialización perezosa.

Creado el servicio ProcesadorOrdenes con IDbContextFactory.

Registrado todos los servicios en el contenedor.

Creado el caso de uso RepositorioUnidadTrabajoUseCase.

Ejecutado las demostraciones de repositorio genérico, específico y unidad de trabajo.

Comparado el rendimiento y la memoria con y sin repositorio.

Documentado los anti-patrones y las buenas prácticas.

Diagnosticado el error de exponer IQueryable.

Creado el repositorio IDetalleOrdenRepositorio con tests de Moq.

| Conexión con el siguiente punto

En este punto se ha consolidado el patrón Repositorio y la unidad de trabajo en el proyecto AceriaData, aplicando buenas prácticas empresariales. Se ha comprobado que el repositorio encapsula el acceso a datos y que la unidad de trabajo coordina varios repositorios en una sola transacción. Se ha integrado IDbContextFactory para servicios de larga duración. Se han comparado el rendimiento y la memoria con y sin repositorio. Se han documentado los anti-patrones y las buenas prácticas. Se ha creado un test con Moq para verificar el comportamiento del repositorio de forma aislada.

En el siguiente punto se estudiará el logging y el diagnóstico en EF Core, con sus implicaciones en la monitorización de la capa de persistencia en aplicaciones empresariales.

Punto 5.10 — Logging y diagnóstico en Entity Framework Core

Práctica

Ejercicio: Integrar EF Core con ILogger y Serilog en el proyecto AceriaData. Configurar el logging de las operaciones de escritura y las consultas. Registrar los mensajes en la consola y en un archivo. Analizar los logs generados.

Contexto del proyecto: En el punto 5.9 se consolidó el patrón Repositorio y la unidad de trabajo en aplicaciones empresariales. En este punto se profundiza en el logging y el diagnóstico, que son esenciales para monitorizar el comportamiento de EF Core en producción. Esta técnica se usará en el punto 5.11 para el testing con EF Core.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Instalar los paquetes de Serilog

```
dotnet add package Serilog.Extensions.Logging
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Sinks.File
```

dotnet add package Serilog.Extensions.Logging → añade la integración con ILoggerFactory. dotnet add package Serilog.Sinks.Console → añade el proveedor de consola. dotnet add package Serilog.Sinks.File → añade el proveedor de archivo.

Error común: si se olvida alguno de los paquetes, la configuración de Serilog no compila. Se deben instalar los tres.

Paso 3: Configurar Serilog en el método Main

Modificar el método Main de src/AceriaData.Console/Program.cs:

```
using Serilog;

public static void Main()
{
    Log.Logger = new LoggerConfiguration()
        .MinimumLevel.Information()
        .WriteTo.Console()
        .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
        .CreateLogger();

    try
    {
        Log.Information("Iniciando la aplicación AceriaData");

        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");
```

```

var services = new ServiceCollection();

services.AddLogging(builder =>
{
    builder.AddSerilog(Log.Logger, dispose: true);
});

services.AddDbContext<AceriaDbContext>(options =>
    options
        .UseSqlServer(connectionString, sqlOptions =>
            {
                sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                sqlOptions.CommandTimeout(60);
            })
        .EnableSensitiveDataLogging()
        .EnableDetailedErrors());

services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
services.AddScoped<LoggingDagnosticoUseCase>();

_provider = services.BuildServiceProvider();

using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<LoggingDagnosticoUseCase>();
    useCase.Ejecutar();
}
}
catch (Exception ex)
{
    Log.Fatal(ex, "La aplicación ha terminado debido a una excepción no controlada");
}
finally
{
    Log.CloseAndFlush();
}
}

```

Línea 1: using Serilog; → importa el espacio de nombres de Serilog.

Línea 3: public static void Main() → punto de entrada.

Línea 5: Log.Logger = new LoggerConfiguration() → crea la configuración de Serilog.

Línea 6: .MinimumLevel.Information() → establece el nivel mínimo.

Línea 7: .WriteTo.Console() → escribe en la consola.

Línea 8: .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day) → escribe en un archivo diario.

Línea 9: .CreateLogger(); → crea el logger.

Línea 11: try → inicio del bloque.

Línea 13: Log.Information("Iniciando la aplicación AceriaData"); → registra el inicio.

Línea 15: var configuration = new ConfigurationBuilder() → crea el constructor de configuración.

Línea 19: .Build(); → construye la configuración.

Línea 21: var connectionString = configuration.GetConnectionString("AceriaDB") → lee la cadena de conexión.

Línea 24: var services = new ServiceCollection(); → crea la colección de servicios.

Línea 26: services.AddLogging(builder => → registra el logging.

Línea 28: builder.AddSerilog(Log.Logger, dispose: true); → añade Serilog.

Línea 29: }); → cierra la configuración del logging.

Línea 31: `services.AddDbContext(options => →` registra el `DbContext`.

Línea 40: `.EnableDetailedErrors()); →` muestra información detallada en los errores.

Línea 42: `services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>()); →` registra la unidad de trabajo.

Línea 43: `services.AddScoped(); →` registra el caso de uso.

Línea 45: `_provider = services.BuildServiceProvider(); →` construye el proveedor.

Línea 47: `using (var scope = _provider.CreateScope()) →` crea un ámbito.

Línea 49: `var useCase = scope.ServiceProvider.GetRequiredService(); →` resuelve el caso de uso.

Línea 50: `useCase.Ejecutar(); →` ejecuta el caso de uso.

Línea 53: `catch (Exception ex) →` captura la excepción.

Línea 55: `Log.Fatal(ex, "La aplicación ha terminado debido a una excepción no controlada"); →` registra el error fatal.

Línea 57: `finally →` bloque final.

Línea 59: `Log.CloseAndFlush(); →` cierra el logger y vacía los buffers.

Error común: si se olvida `Log.CloseAndFlush()`, los mensajes pendientes pueden perderse al cerrar la aplicación.

| Paso 4: Crear el caso de uso de logging y diagnóstico

Crear el archivo `src/AceriaData.Application/UseCases/LoggingDiagnosticoUseCase.cs`:

```
using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;
using Microsoft.Extensions.Logging;

namespace AceriaData.Application.UseCases;

public class LoggingDiagnosticoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    private readonly ILogger<LoggingDiagnosticoUseCase> _logger;

    public LoggingDiagnosticoUseCase(IUnidadDeTrabajo unidad, ILogger<LoggingDiagnosticoUseCase> logger)
    {
        _unidad = unidad;
        _logger = logger;
    }

    public void Ejecutar()
    {
        _logger.LogInformation("=== LOGGING Y DIAGNÓSTICO ===");

        DemostrarLoggingDeConsultas();
        DemostrarLoggingDeEscritura();
    }

    private void DemostrarLoggingDeConsultas()
    {
        _logger.LogInformation("--- Consultas ---");

        var ordenes = _unidad.Ordenes.ObtenerTodas();
        _logger.LogInformation("Se han cargado {Total} órdenes", ordenes.Count);

        var pendientes = _unidad.Ordenes.ObtenerPendientes();
        _logger.LogInformation("Se han cargado {Total} órdenes pendientes", pendientes.Count);
    }

    private void DemostrarLoggingDeEscritura()
    {
        _logger.LogInformation("--- Escritura ---");
    }
}
```

```

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-LOG-001",
            Cliente = "Constructora del Norte",
            Estado = "Pendiente",
            FechaCreacion = DateTime.Now
        };

        _unidad Ordenes.Agregar(orden);
        var filas = _unidad.Guardar();

        _logger.LogInformation("Se han insertado {Filas} filas. Orden creada: {NumeroOrden}", filas,
            orden.NumeroOrden);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 3: using Microsoft.Extensions.Logging; → importa el logging.

Línea 5: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 7: public class LoggingDiagnosticoUseCase → declara el caso de uso.

Línea 9: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: private readonly ILogger _logger; → campo del logger.

Línea 12: public LoggingDiagnosticoUseCase(IUnidadDeTrabajo unidad, ILogger logger) → constructor.

Línea 14: _unidad = unidad; → asigna la unidad de trabajo.

Línea 15: _logger = logger; → asigna el logger.

Línea 18: public void Ejecutar() → declara el método principal.

Línea 20: _logger.LogInformation("=== LOGGING Y DIAGNÓSTICO ==="); → registra la cabecera.

Línea 22: DemostrarLoggingDeConsultas(); → llama al método de consultas.

Línea 23: DemostrarLoggingDeEscritura(); → llama al método de escritura.

Línea 26: private void DemostrarLoggingDeConsultas() → declara el método.

Línea 28: _logger.LogInformation("--- Consultas ---"); → registra la cabecera.

Línea 30: var ordenes = _unidad Ordenes.ObtenerTodas(); → carga las órdenes.

Línea 31: _logger.LogInformation("Se han cargado {Total} órdenes", ordenes.Count); → registra el número.

Línea 33: var pendientes = _unidad Ordenes.ObtenerPendientes(); → carga las pendientes.

Línea 34: _logger.LogInformation("Se han cargado {Total} órdenes pendientes", pendientes.Count); → registra el número.

Línea 37: private void DemostrarLoggingDeEscritura() → declara el método.

Línea 39: _logger.LogInformation("--- Escritura ---"); → registra la cabecera.

Línea 41: var orden = new OrdenFabricacion → crea la orden.

Línea 49: _unidad Ordenes.Agregar(orden); → agrega la orden.

Línea 50: var filas = _unidad.Guardar(); → guarda los cambios.

Línea 52: _logger.LogInformation("Se han insertado {Filas} filas. Orden creada: {NumeroOrden}", filas, orden.NumeroOrden); → registra el resultado con propiedades estructuradas.

Error común: si el logger no se inyecta, el caso de uso no compila. Se debe inyectar ILogger en el constructor.

Paso 5: Configurar el filtrado de Serilog

Modificar la configuración de Serilog en el método Main:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Database.Command",
Serilog.Events.LogEventLevel.Information)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Update",
Serilog.Events.LogEventLevel.Information)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Query", Serilog.Events.LogEventLevel.Warning)
    .WriteTo.Console()
    .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Línea 1: Log.Logger = new LoggerConfiguration() → crea la configuración.

Línea 2: .MinimumLevel.Information() → nivel mínimo global.

Línea 3: .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Database.Command", Serilog.Events.LogEventLevel.Information) → nivel para comandos.

Línea 4: .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Update", Serilog.Events.LogEventLevel.Information) → nivel para actualizaciones.

Línea 5: .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Query", Serilog.Events.LogEventLevel.Warning) → nivel para consultas.

Línea 6: .WriteTo.Console() → escribe en la consola.

Línea 7: .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day) → escribe en un archivo diario.

Línea 8: .CreateLogger(); → crea el logger.

Error común: si se filtran demasiado los mensajes, se pueden perder los mensajes importantes. Se debe ajustar el nivel según el entorno.

Paso 6: Registrar el caso de uso en el contenedor

Modificar el método Main para registrar el caso de uso:

```
services.AddScoped<LoggingDiagnosticsUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 7: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<LoggingDiagnosticsUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 8: Insertar datos de prueba iniciales

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 3, 10) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List` → crea la lista.

Línea 9: `new OrdenFabricacion { ... }` → primera orden.

Línea 10: `new OrdenFabricacion { ... }` → segunda orden.

Línea 11: `new OrdenFabricacion { ... }` → tercera orden.

Línea 12: `};` → cierra la lista.

Línea 14: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 15: `context.SaveChanges();` → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

| Paso 9: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen los mensajes de logging en la consola y en el archivo `logs/aceria-YYYYMMDD.log`. Se observan las sentencias SQL, los mensajes del caso de uso y las operaciones de escritura.

| Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```
[12:00:00 INF] Iniciando la aplicación AceriaData
[12:00:01 INF] === LOGGING Y DIAGNÓSTICO ===
[12:00:01 INF] --- Consultas ---
[12:00:01 INF] Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
[12:00:01 INF] Se han cargado 3 órdenes
[12:00:01 INF] Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='60']
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
[12:00:01 INF] Se han cargado 2 órdenes pendientes
[12:00:01 INF] --- Escritura ---
[12:00:02 INF] Executed DbCommand (5ms) [Parameters=[@p0='OF-LOG-001', ...], CommandType='Text',
CommandTimeout='60']
INSERT INTO [OrdenesFabricacion] ([NumeroOrden], [Cliente], [Estado], [FechaCreacion], [IsDeleted])
VALUES (@p0, @p1, @p2, @p3, @p4);
[12:00:02 INF] Se han insertado 1 filas. Orden creada: OF-LOG-001
[12:00:02 INF] La aplicación ha terminado
```

La primera sección muestra el inicio de la aplicación. La segunda sección muestra las consultas con las sentencias SQL. La tercera sección muestra la escritura con el INSERT. La cuarta sección muestra el cierre.

Observaciones: el logging registra las sentencias SQL, los parámetros y los mensajes del caso de uso. Serilog escribe en la consola y en el archivo. Los mensajes estructurados incluyen propiedades tipadas.

Paso 11: Diagnosticar un error común

Modificar la configuración de Serilog para no filtrar los mensajes de EF Core:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Resultado esperado: se registran todos los mensajes de EF Core, incluyendo los de nivel Debug y Trace. El log se vuelve muy verboso y difícil de analizar.

Solución: filtrar los mensajes por categoría con `MinimumLevel.Override`.

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Database.Command",
Serilog.Events.LogEventLevel.Information)
    .MinimumLevel.Override("Microsoft.EntityFrameworkCore.Query", Serilog.Events.LogEventLevel.Warning)
    .WriteTo.Console()
    .WriteTo.File("logs/aceria-.log", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Resultado esperado con la solución: solo se registran los mensajes de las categorías configuradas con los niveles adecuados.

Errores comunes del ejercicio

Error Causa Solución Serilog no configurado Falta el paquete o la configuración Instalar los paquetes y configurar Logger no inyectado Falta el ILogger en el constructor Inyectar el logger Mensajes no registrados Nivel mínimo demasiado alto Ajustar el nivel Log demasiado verboso No se filtran las categorías Usar `MinimumLevel.Override` Archivo de log no creado Falta el directorio logs Crear el directorio o configurar la ruta Log.CloseAndFlush no llamado Los mensajes pendientes se pierden Llamar en el finally

Retos resueltos: Logging estructurado con propiedades tipadas

Reto: Añadir logging estructurado al repositorio para registrar cada operación de escritura con propiedades tipadas. Usar ILogger en el repositorio y registrar el número de orden, el cliente y el número de filas afectadas.

Solución paso a paso:

Paso 1: Modificar el repositorio para recibir ILogger:

```
using Microsoft.Extensions.Logging;

public class OrdenRepositorio : Repositorio<OrdenFabricacion>, IOrdenRepositorio
{
    private readonly ILogger<OrdenRepositorio> _logger;

    public OrdenRepositorio(AceriaDbContext context, ILogger<OrdenRepositorio> logger) : base(context)
    {
        _logger = logger;
    }

    public override void Agregar(OrdenFabricacion orden)
    {
        _logger.LogInformation("Agregando orden {NumeroOrden} del cliente {Cliente}",
            orden.NumeroOrden, orden.Cliente);
        base.Agregar(orden);
    }
}
```

Línea 1: using Microsoft.Extensions.Logging; → importa el logging.

Línea 3: public class OrdenRepositorio : Repositorio, IOrdenRepositorio → declara el repositorio.

Línea 5: private readonly ILogger _logger; → campo del logger.

Línea 7: public OrdenRepositorio(AceriaDbContext context, ILogger logger) : base(context) → constructor.

Línea 9: _logger = logger; → asigna el logger.

Línea 12: public override void Agregar(OrdenFabricacion orden) → sobrescribe el método.

Línea 14: _logger.LogInformation("Agregando orden {NumeroOrden} del cliente {Cliente}", orden.NumeroOrden, orden.Cliente); → registra el mensaje estructurado.

Línea 15: base.Agregar(orden); → llama al método base.

Paso 2: Registrar el repositorio en el contenedor:

```
services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
```

Línea 1: services.AddScoped<IOrdenRepositorio, OrdenRepositorio>(); → registra el repositorio.

Paso 3: Ejecutar el caso de uso y verificar que los mensajes aparecen con las propiedades tipadas.

Resultado esperado: los mensajes de logging incluyen las propiedades NumeroOrden y Cliente, que se pueden consultar y filtrar en el sistema de logging.

Ampliación técnica obligatoria — diagnóstico completo, archivo, EventCounters y telemetría

La fuente original cubre ILogger, Serilog y logging estructurado, pero el contrato del curso exige completar la observabilidad de EF Core 8 con evidencia ejecutable adicional.

Paso 12: Configurar un sink de archivo con rotación y retención

Añade el sink de archivo y configura límites explícitos:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .WriteTo.File(
        path: "logs/aceriadata-.log",
        rollingInterval: RollingInterval.Day,
        fileSizeLimitBytes: 10 * 1024 * 1024,
        rollOnFileSizeLimit: true,
        retainedFileCountLimit: 7,
        shared: true)
    .CreateLogger();
```

Genera suficientes mensajes para comprobar que se crea el archivo y que la configuración de rotación está activa. No basta con declarar las opciones: el laboratorio debe verificar al menos la existencia del archivo y registrar la configuración usada.

Paso 13: Implementar un observador completo de DiagnosticListener

`DiagnosticListener` permite observar eventos EF Core del proceso. No sustituye a `ILogger`; es un mecanismo de diagnóstico distinto.

```
using System.Diagnostics;

public sealed class EfDiagnosticObserver :
    IObservable<DiagnosticListener>,
    IObservable<KeyValuePair<string, object?>>, IDisposable
{
    private readonly List<IDisposable> _subscriptions = new();
    private readonly Action<string> _write;

    public EfDiagnosticObserver(Action<string> write) => _write = write;

    public void Start() =>
        _subscriptions.Add(DiagnosticListener.AllListeners.Subscribe(this));

    public void OnNext(DiagnosticListener listener)
    {
        if (listener.Name == "Microsoft.EntityFrameworkCore")
            _subscriptions.Add(listener.Subscribe(this));
    }

    public void OnNext(KeyValuePair<string, object?> evt)
    {
        if (evt.Key.Contains("CommandExecuting", StringComparison.Ordinal) ||
            evt.Key.Contains("CommandExecuted", StringComparison.Ordinal) ||
            evt.Key.Contains("SaveChanges", StringComparison.Ordinal))
        {
            _write($"EF-DIAG | {evt.Key}");
        }
    }

    public void OnError(Exception error) => _write($"EF-DIAG-ERROR | {error.Message}");
    public void OnCompleted() { }

    public void Dispose()
    {
        foreach (var subscription in _subscriptions)
            subscription.Dispose();
        _subscriptions.Clear();
    }
}
```

El escenario debe iniciar el observador, ejecutar una consulta y un `SaveChanges`, comprobar que se recibieron eventos y liberar todas las suscripciones.

Paso 14: Observar los EventCounters de EF Core 8

En EF Core 8 la práctica usa **EventCounters**. Las métricas basadas en `System.Diagnostics.Metrics` con nombres `microsoft.entityframeworkcore.*` se introdujeron posteriormente y no deben presentarse como una característica de EF Core 8.

Muestra el PID del proceso:

```
Console.WriteLine($"PID: {Environment.ProcessId}");
```

Desde otra terminal:

```
dotnet counters monitor --counters Microsoft.EntityFrameworkCore -p <PID>
```

Durante la ejecución deben poder observarse contadores como `Active DbContexts`, `Queries`, `SaveChanges`, `Query Cache Hit Rate` y `Optimistic Concurrency Failures` cuando el escenario los produzca.

Paso 15: Añadir Application Insights como destino mediante Azure Monitor/OpenTelemetry

Para una aplicación de consola no se debe asumir que `AddApplicationInsightsTelemetry()` configura mágicamente un host ASP.NET. Usa un pipeline `OpenTelemetry` compatible con .NET y el exportador de Azure Monitor cuando exista una cadena de conexión.

```
dotnet add package OpenTelemetry
dotnet add package Azure.Monitor.OpenTelemetry.Exporter
```

```
using Azure.Monitor.OpenTelemetry.Exporter;
using OpenTelemetry;
using OpenTelemetry.Trace;

var aiConnectionString = Environment.GetEnvironmentVariable(
    "APPLICATIONINSIGHTS_CONNECTION_STRING");

TracerProvider? azureMonitor = null;
if (!string.IsNullOrEmpty(aiConnectionString))
{
    azureMonitor = Sdk.CreateTracerProviderBuilder()
        .AddSource("AceriaData")
        .AddAzureMonitorTraceExporter(options =>
            options.ConnectionString = aiConnectionString)
        .Build();
}
```

La demo de Azure Monitor es opcional si no existe una cadena de conexión real. El laboratorio local sigue siendo verificable mediante `ILogger`, `Serilog`, `DiagnosticListener`, `EventCounters` y archivo. No se afirma que los eventos específicos de `DiagnosticListener` de EF Core se exporten automáticamente como trazas de Application Insights; el pipeline de telemetría debe instrumentarse de forma explícita.

Paso 16: Evidencia final de 5.10

La ejecución debe dejar evidencia observable de:

1. logs de EF Core filtrados por categoría/nivel;
2. logging estructurado con propiedades;
3. archivo de log y configuración de rotación/retención;
4. eventos recibidos mediante `DiagnosticListener`;
5. `EventCounters` de EF Core 8 observables con `dotnet-counters`;

6. configuración opcional de exportación a Azure Monitor/Application Insights mediante OpenTelemetry.

| Analogía final

El logging y el diagnóstico en una acería son como el sistema de registro del panel de control. Cada operación que se realiza en la planta se anota en el registro: qué se hizo, cuándo, con qué parámetros y cuánto tardó. El registro permite detectar problemas, analizar el rendimiento y auditar los cambios. El logging estructurado es como el registro con columnas: cada campo tiene su propio espacio y se puede consultar por separado. Serilog es como el sistema de registro que escribe en varios destinos: la consola, el archivo y el servicio de monitorización. El filtrado por categoría es como elegir qué secciones del registro se quieren consultar. El diagnóstico es como el análisis del registro: se buscan patrones, se detectan anomalías y se toman decisiones. Así funcionan el logging y el diagnóstico en EF Core: se registra, se filtra y se analiza para monitorizar el comportamiento de la aplicación.

| Resultado esperado

Al final del ejercicio, deberías haber:

Instalado los paquetes de Serilog.

Configurado Serilog en el método Main.

Configurado el filtrado por categoría con `MinimumLevel.Override`.

Creado el caso de uso `LoggingDiagnosticoUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba iniciales.

Ejecutado el proyecto y verificado los logs en la consola y en el archivo.

Diagnosticado el error de no filtrar las categorías.

Añadido logging estructurado al repositorio.

| Conexión con el siguiente punto

En este punto se han estudiado el logging y el diagnóstico en EF Core, incluyendo la integración con ILogger, la configuración de Serilog, el filtrado por categoría y el logging estructurado. Se ha comprobado que el logging permite monitorizar el comportamiento de EF Core en producción. En el siguiente punto se estudiará el testing con EF Core: InMemory y SQLite in-memory.

Punto 5.11 — Testing con EF Core: InMemory y SQLite in-memory

Práctica

Ejercicio: Crear el proyecto de pruebas `AceriaData.Tests` con `xUnit`. Configurar los proveedores `InMemory` y `SQLite` en memoria. Escribir tests que verifiquen el comportamiento del repositorio de órdenes y de la unidad de trabajo. Comparar el comportamiento de ambos proveedores.

Contexto del proyecto: En el punto 5.10 se estudió el logging y el diagnóstico, incluyendo la integración con `Serilog` y el logging estructurado. En este punto se profundiza en el testing con EF Core, que es esencial para garantizar la calidad de la capa de persistencia. Esta técnica se usará en el punto 5.12 para las buenas prácticas y anti-patrones.

Paso 1: Crear el proyecto de pruebas

```
cd AceriaData
dotnet new xunit -n AceriaData.Tests -f net8.0 -o tests/AceriaData.Tests
dotnet sln add tests/AceriaData.Tests/AceriaData.Tests.csproj
```

`dotnet new xunit` → crea un proyecto de pruebas con `xUnit`. `-n AceriaData.Tests` → asigna el nombre al proyecto. `-f net8.0` → especifica .NET 8. `-o tests/AceriaData.Tests` → especifica la carpeta de salida. `dotnet sln add` → añade el proyecto a la solución.

Resultado esperado: se crea el proyecto `AceriaData.Tests` en la carpeta `tests`.

Error común: si la carpeta `tests` no existe, el comando la crea. Si el proyecto ya existe, el comando falla.

Paso 2: Añadir las referencias a los proyectos

```
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj reference src/AceriaData.Domain/
AceriaData.Domain.csproj
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj reference src/AceriaData.Application/
AceriaData.Application.csproj
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj reference src/AceriaData.Infrastructure/
AceriaData.Infrastructure.csproj
```

`dotnet add reference` → añade una referencia entre proyectos.

Resultado esperado: el proyecto de pruebas referencia a los tres proyectos.

Error común: si la ruta es incorrecta, el comando falla. Se debe verificar la ruta.

Paso 3: Añadir los paquetes de EF Core para pruebas

```
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj package Microsoft.EntityFrameworkCore.InMemory
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj package Microsoft.EntityFrameworkCore.Sqlite
```

`dotnet add package` → añade una referencia a un paquete NuGet.

Resultado esperado: el proyecto de pruebas referencia a los dos proveedores.

Error común: si se olvida alguno de los paquetes, las pruebas no compilan.

Paso 4: Crear la clase base de pruebas

Crear el archivo `tests/AceriaData.Tests/Base/TestBase.cs`:

```

using AceriaData.Infrastructure.Persistence;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Tests.Base;

public abstract class TestBase : IDisposable
{
    protected readonly AceriaDbContext Context;
    private readonly SqliteConnection _connection;

    protected TestBase()
    {
        _connection = new SqliteConnection("Data Source=:memory:");
        _connection.Open();

        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlite(_connection)
            .Options;

        Context = new AceriaDbContext(options);
        Context.Database.EnsureCreated();
    }

    public void Dispose()
    {
        Context.Dispose();
        _connection.Dispose();
    }
}

```

Línea 1: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 2: using Microsoft.Data.Sqlite; → importa SQLite.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 5: namespace AceriaData.Tests.Base; → declara el espacio de nombres.

Línea 7: public abstract class TestBase : IDisposable → declara la clase base.

Línea 9: protected readonly AceriaDbContext Context; → campo del contexto.

Línea 10: private readonly SqliteConnection _connection; → campo de la conexión.

Línea 12: protected TestBase() → constructor.

Línea 14: _connection = new SqliteConnection("Data Source=:memory:"); → crea la conexión en memoria.

Línea 15: _connection.Open(); → abre la conexión.

Línea 17: var options = new DbContextOptionsBuilder() → crea el constructor de opciones.

Línea 18: .UseSqlite(_connection) → configura SQLite.

Línea 19: .Options; → obtiene las opciones.

Línea 21: Context = new AceriaDbContext(options); → crea el contexto.

Línea 22: Context.Database.EnsureCreated(); → crea la base de datos.

Línea 25: public void Dispose() → declara el método Dispose.

Línea 27: Context.Dispose(); → libera el contexto.

Línea 28: _connection.Dispose(); → libera la conexión.

Error común: si se olvida abrir la conexión, la base de datos en memoria no existe y las pruebas fallan.

Paso 5: Crear tests del repositorio de órdenes

Crear el archivo tests/AceriaData.Tests/Repositories/OrdenRepositorioTests.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Repositories;
using AceriaData.Tests.Base;
using Microsoft.Extensions.Logging.Abstractions;

namespace AceriaData.Tests.Repositories;

public class OrdenRepositorioTests : TestBase
{
    private readonly OrdenRepositorio _repositorio;

    public OrdenRepositorioTests()
    {
        _repositorio = new OrdenRepositorio(Context, NullLogger<OrdenRepositorio>.Instance);
    }

    [Fact]
    public void ObtenerPorId_ConIdExistente_DevuelveLaOrden()
    {
        var orden = new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "Constructora del Norte",
        Estado = "Pendiente", FechaCreacion = DateTime.Now };
        Context.OrdenesFabricacion.Add(orden);
        Context.SaveChanges();

        var recuperada = _repositorio.ObtenerPorId(orden.Id);

        Assert.NotNull(recuperada);
        Assert.Equal("0F-001", recuperada.NumeroOrden);
    }

    [Fact]
    public void ObtenerPorId_ConIdInexistente_DevuelveNull()
    {
        var recuperada = _repositorio.ObtenerPorId(99999);

        Assert.Null(recuperada);
    }

    [Fact]
    public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes()
    {
        Context.OrdenesFabricacion.AddRange(
            new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "A", Estado = "Pendiente",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "0F-002", Cliente = "B", Estado = "EnProceso",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "0F-003", Cliente = "C", Estado = "Pendiente",
            FechaCreacion = DateTime.Now });
        Context.SaveChanges();

        var pendientes = _repositorio.ObtenerPendientes();
        Assert.Equal(2, pendientes.Count);
    }

    [Fact]
    public void ContarOrdenes_ConVariasOrdenes_DevuelveElTotal()
    {
        Context.OrdenesFabricacion.AddRange(
            new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "A", Estado = "Pendiente",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "0F-002", Cliente = "B", Estado = "EnProceso",
            FechaCreacion = DateTime.Now });
        Context.SaveChanges();

        var total = _repositorio.ContarOrdenes();

        Assert.Equal(2, total);
    }
}
```

```

[Fact]
public void ExisteAlgunaOrden_SinOrdenes_DevuelveFalse()
{
    var existe = _repositorio.ExisteAlgunaOrden();

    Assert.False(existe);
}
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Repositories; → importa el repositorio.

Línea 3: using AceriaData.Tests.Base; → importa la clase base.

Línea 4: using Microsoft.Extensions.Logging.Abstractions; → importa NullLogger.

Línea 6: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 8: public class OrdenRepositorioTests : TestBase → declara la clase de tests.

Línea 10: private readonly OrdenRepositorio _repositorio; → campo del repositorio.

Línea 12: public OrdenRepositorioTests() → constructor.

Línea 14: _repositorio = new OrdenRepositorio(Context, NullLogger.Instance); → crea el repositorio con NullLogger.

Línea 17: [Fact] → atributo de test.

Línea 18: public void ObtenerPorId_ConIdExistente_DevuelveLaOrden() → declara el test.

Línea 20: var orden = new OrdenFabricacion { ... }; → crea la orden.

Línea 21: Context.OrdenesFabricacion.Add(orden); → registra la orden.

Línea 22: Context.SaveChanges(); → guarda los cambios.

Línea 24: var recuperada = _repositorio.ObtenerPorId(orden.Id); → llama al método.

Línea 26: Assert.NotNull(recuperada); → comprueba que existe.

Línea 27: Assert.Equal("OF-001", recuperada.NumeroOrden); → comprueba el número.

Línea 30: [Fact] → atributo de test.

Línea 31: public void ObtenerPorId_ConIdInexistente_DevuelveNull() → declara el test.

Línea 33: var recuperada = _repositorio.ObtenerPorId(99999); → llama al método.

Línea 35: Assert.Null(recuperada); → comprueba que es nulo.

Línea 38: [Fact] → atributo de test.

Línea 39: public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes() → declara el test.

Línea 41: Context.OrdenesFabricacion.AddRange(→ añade varias órdenes.

Línea 45: Context.SaveChanges(); → guarda los cambios.

Línea 47: var pendientes = _repositorio.ObtenerPendientes(); → llama al método.

Línea 49: Assert.Equal(2, pendientes.Count); → comprueba el número.

Línea 52: [Fact] → atributo de test.

Línea 53: public void ContarOrdenes_ConVariasOrdenes_DevuelveElTotal() → declara el test.

Línea 55: Context.OrdenesFabricacion.AddRange(→ añade varias órdenes.

Línea 58: `Context.SaveChanges();` → guarda los cambios.

Línea 60: `var total = _repositorio.ContarOrdenes();` → llama al método.

Línea 62: `Assert.Equal(2, total);` → comprueba el total.

Línea 65: `[Fact]` → atributo de test.

Línea 66: `public void ExisteAlgunaOrden_SinOrdenes_DevuelveFalse()` → declara el test.

Línea 68: `var existe = _repositorio.ExisteAlgunaOrden();` → llama al método.

Línea 70: `Assert.False(existe);` → comprueba que es falso.

Error común: si el repositorio no acepta `ILogger` en el constructor, el código no compila. Se debe inyectar el logger o usar `NullLogger`.

| Paso 6: Crear tests de la unidad de trabajo

Crear el archivo `tests/AceriaData.Tests/Repositories/UnidadDeTrabajoTests.cs`:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Repositories;
using AceriaData.Tests.Base;

namespace AceriaData.Tests.Repositories;

public class UnidadDeTrabajoTests : TestBase
{
    [Fact]
    public void Guardar_ConOrdenYPlancha_InsertaAmbasEnUnaTransaccion()
    {
        using var unidad = new UnidadDeTrabajo(Context);

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-UOW-001",
            Cliente = "Constructora del Norte",
            Estado = "Pendiente",
            FechaCreacion = DateTime.Now
        };
        unidad Ordenes.Agregar(orden);

        var plancha = new PlanchaAcero
        {
            Orden = orden,
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 370.5m,
            Activa = true
        };
        unidad Planchas.Agregar(plancha);

        var filas = unidad.Guardar();

        Assert.Equal(2, filas);
        Assert.NotEqual(0, orden.Id);
        Assert.NotEqual(0, plancha.Id);
    }

    [Fact]
    public void Guardar_SinCambios_DevuelveCero()
    {
        using var unidad = new UnidadDeTrabajo(Context);

        var filas = unidad.Guardar();

        Assert.Equal(0, filas);
    }
}
```

```

[Fact]
public void Guardar_ConDosRepositorios_CompartenElMismoContexto()
{
    using var unidad = new UnidadDeTrabajo(Context);

    var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "A", Estado = "Pendiente",
FechaCreacion = DateTime.Now };
    unidad Ordenes.Agregar(orden);
    unidad.Guardar();

    var plancha = new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo =
3000, Peso = 370.5m, Activa = true };
    unidad.Planchas.Agregar(plancha);
    var filas = unidad.Guardar();

    Assert.Equal(1, filas);
}
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Repositories; → importa la unidad de trabajo.

Línea 3: using AceriaData.Tests.Base; → importa la clase base.

Línea 5: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 7: public class UnidadDeTrabajoTests : TestBase → declara la clase de tests.

Línea 9: [Fact] → atributo de test.

Línea 10: public void Guardar_ConOrdenYPlancha_InsertaAmbasEnUnaTransaccion() → declara el test.

Línea 12: using var unidad = new UnidadDeTrabajo(Context); → crea la unidad de trabajo.

Línea 14: var orden = new OrdenFabricacion → crea la orden.

Línea 22: unidad Ordenes.Agregar(orden); → agrega la orden.

Línea 24: var plancha = new PlanchaAcero → crea la plancha.

Línea 34: unidad.Planchas.Agregar(plancha); → agrega la plancha.

Línea 36: var filas = unidad.Guardar(); → guarda los cambios.

Línea 38: Assert.Equal(2, filas); → comprueba el número de filas.

Línea 39: Assert.NotEqual(0, orden.Id); → comprueba que el Id se ha generado.

Línea 40: Assert.NotEqual(0, plancha.Id); → comprueba que el Id se ha generado.

Línea 43: [Fact] → atributo de test.

Línea 44: public void Guardar_SinCambios_DevuelveCero() → declara el test.

Línea 46: using var unidad = new UnidadDeTrabajo(Context); → crea la unidad de trabajo.

Línea 48: var filas = unidad.Guardar(); → guarda los cambios.

Línea 50: Assert.Equal(0, filas); → comprueba que devuelve cero.

Línea 53: [Fact] → atributo de test.

Línea 54: public void Guardar_ConDosRepositorios_CompartenElMismoContexto() → declara el test.

Línea 56: using var unidad = new UnidadDeTrabajo(Context); → crea la unidad de trabajo.

Línea 58: var orden = new OrdenFabricacion { ... }; → crea la orden.

Línea 59: unidad Ordenes.Agregar(orden); → agrega la orden.

Línea 60: unidad.Guardar(); → guarda los cambios.

Línea 62: `var plancha = new PlanchaAcero { ... };` → crea la plancha.

Línea 63: `unidad.Planchas.Agregar(plancha);` → agrega la plancha.

Línea 64: `var filas = unidad.Guardar();` → guarda los cambios.

Línea 66: `Assert.Equal(1, filas);` → comprueba el número de filas.

Error común: si se crea una unidad de trabajo por repositorio, no comparten el mismo contexto y no se pueden guardar en una sola transacción. Se debe usar una sola unidad de trabajo.

Paso 7: Crear tests con InMemory

Crear el archivo `tests/AceriaData.Tests/InMemory/OrdenRepositorioInMemoryTests.cs`:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging.Abstractions;

namespace AceriaData.Tests.InMemory;

public class OrdenRepositorioInMemoryTests : IDisposable
{
    private readonly AceriaDbContext _context;
    private readonly OrdenRepositorio _repositorio;

    public OrdenRepositorioInMemoryTests()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        _context = new AceriaDbContext(options);
        _repositorio = new OrdenRepositorio(_context, NullLogger<OrdenRepositorio>.Instance);
    }

    [Fact]
    public void InsertarOrden_ConDatosValidos_SeGuardaEnMemoria()
    {
        var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
        Estado = "Pendiente", FechaCreacion = DateTime.Now };
        _repositorio.Agregar(orden);
        _context.SaveChanges();

        var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001");

        Assert.NotNull(recuperada);
        Assert.Equal("Constructora del Norte", recuperada.Cliente);
    }

    [Fact]
    public void InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion()
    {
        var plancha = new PlanchaAcero { OrdenId = 99999, Espesor = 10.5, Ancho = 1500, Largo = 3000,
        Peso = 370.5m, Activa = true };
        _context.PlanchasAcero.Add(plancha);

        var excepcion = Record.Exception(() => _context.SaveChanges());

        Assert.Null(excepcion);
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 2: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext.

Línea 3: `using AceriaData.Infrastructure.Repositories;` → importa el repositorio.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa EF Core.

Línea 5: `using Microsoft.Extensions.Logging.Abstractions;` → importa NullLogger.

Línea 7: `namespace AceriaData.Tests.InMemory;` → declara el espacio de nombres.

Línea 9: `public class OrdenRepositorioInMemoryTests : IDisposable` → declara la clase de tests.

Línea 11: `private readonly AceriaDbContext _context;` → campo del contexto.

Línea 12: `private readonly OrdenRepositorio _repositorio;` → campo del repositorio.

Línea 14: `public OrdenRepositorioInMemoryTests()` → constructor.

Línea 16: `var options = new DbContextOptionsBuilder()` → crea el constructor de opciones.

Línea 17: `.UseInMemoryDatabase(Guid.NewGuid().ToString())` → configura InMemory con un nombre único.

Línea 18: `.Options;` → obtiene las opciones.

Línea 20: `_context = new AceriaDbContext(options);` → crea el contexto.

Línea 21: `_repositorio = new OrdenRepositorio(_context, NullLogger.Instance);` → crea el repositorio.

Línea 24: `[Fact]` → atributo de test.

Línea 25: `public void InsertarOrden_ConDatosValidos_SeGuardaEnMemoria()` → declara el test.

Línea 27: `var orden = new OrdenFabricacion { ... };` → crea la orden.

Línea 28: `_repositorio.Agregar(orden);` → agrega la orden.

Línea 29: `_context.SaveChanges();` → guarda los cambios.

Línea 31: `var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001");` → recupera la orden.

Línea 33: `Assert.NotNull(recuperada);` → comprueba que existe.

Línea 34: `Assert.Equal("Constructora del Norte", recuperada.Cliente);` → comprueba el cliente.

Línea 37: `[Fact]` → atributo de test.

Línea 38: `public void InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion()` → declara el test.

Línea 40: `var plancha = new PlanchaAcero { OrdenId = 99999, ... };` → crea la plancha con clave foránea inválida.

Línea 41: `_context.PlanchasAcero.Add(plancha);` → registra la plancha.

Línea 43: `var excepcion = Record.Exception(() => _context.SaveChanges());` → ejecuta y captura la excepción.

Línea 45: `Assert.Null(excepcion);` → comprueba que no hay excepción.

Línea 48: `public void Dispose()` → declara el método Dispose.

Línea 50: `_context.Dispose();` → libera el contexto.

Error común: el test `InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion` demuestra una limitación del proveedor InMemory. En SQLite en memoria, este test fallaría porque la restricción de integridad referencial se aplica.

Paso 8: Ejecutar los tests

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

dotnet test → ejecuta los tests del proyecto.

Resultado esperado: todos los tests pasan. El test InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion pasa con InMemory pero fallaría con SQLite en memoria.

Error común: si algún test falla, se debe revisar el mensaje de error y corregir el código.

Paso 9: Crear tests con SQLite en memoria

Crear el archivo tests/AceriaData.Tests/Sqlite/OrdenRepositorioSqliteTests.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Repositories;
using AceriaData.Tests.Base;

namespace AceriaData.Tests.Sqlite;

public class OrdenRepositorioSqliteTests : TestBase
{
    private readonly OrdenRepositorio _repositorio;

    public OrdenRepositorioSqliteTests()
    {
        _repositorio = new OrdenRepositorio(Context,
        Microsoft.Extensions.Logging.Abstractions.NullLogger<OrdenRepositorio>.Instance);
    }

    [Fact]
    public void InsertarOrden_ConDatosValidos_SeGuardaEnSqlite()
    {
        var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
        Estado = "Pendiente", FechaCreacion = DateTime.Now };
        _repositorio.Agregar(orden);
        Context.SaveChanges();

        var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001");

        Assert.NotNull(recuperada);
        Assert.Equal("Constructora del Norte", recuperada.Cliente);
    }

    [Fact]
    public void InsertarPlanchaConOrdenInvalida_LanzaExcepcion()
    {
        var plancha = new PlanchaAcero { OrdenId = 99999, Espesor = 10.5, Ancho = 1500, Largo = 3000,
        Peso = 370.5m, Activa = true };
        Context.PlanchasAcero.Add(plancha);

        var excepcion = Record.Exception(() => Context.SaveChanges());

        Assert.NotNull(excepcion);
    }

    [Fact]
    public void InsertarOrdenConNumeroDuplicado_LanzaExcepcion()
    {
        Context OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "A",
        Estado = "Pendiente", FechaCreacion = DateTime.Now });
        Context.SaveChanges();

        Context OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "B",
        Estado = "Pendiente", FechaCreacion = DateTime.Now });

        var excepcion = Record.Exception(() => Context.SaveChanges());
    }
}
```

```

        Assert.NotNull(excepcion);
    }
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Repositories; → importa el repositorio.

Línea 3: using AceriaData.Tests.Base; → importa la clase base.

Línea 5: namespace AceriaData.Tests.Sqlite; → declara el espacio de nombres.

Línea 7: public class OrdenRepositorioSqliteTests : TestBase → declara la clase de tests.

Línea 9: private readonly OrdenRepositorio _repositorio; → campo del repositorio.

Línea 11: public OrdenRepositorioSqliteTests() → constructor.

Línea 13: _repositorio = new OrdenRepositorio(Context, ...); → crea el repositorio.

Línea 17: [Fact] → atributo de test.

Línea 18: public void InsertarOrden_ConDatosValidos_SeGuardaEnSqlite() → declara el test.

Línea 20: var orden = new OrdenFabricacion { ... }; → crea la orden.

Línea 21: _repositorio.Agregar(orden); → agrega la orden.

Línea 22: Context.SaveChanges(); → guarda los cambios.

Línea 24: var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001"); → recupera la orden.

Línea 26: Assert.NotNull(recuperada); → comprueba que existe.

Línea 27: Assert.Equal("Constructora del Norte", recuperada.Cliente); → comprueba el cliente.

Línea 30: [Fact] → atributo de test.

Línea 31: public void InsertarPlanchaConOrdenInvalida_LanzaExcepcion() → declara el test.

Línea 33: var plancha = new PlanchaAcero { OrdenId = 99999, ... }; → crea la plancha con clave foránea inválida.

Línea 34: Context.PlanchasAcero.Add(plancha); → registra la plancha.

Línea 36: var excepcion = Record.Exception(() => Context.SaveChanges()); → captura la excepción.

Línea 38: Assert.NotNull(excepcion); → comprueba que hay excepción.

Línea 41: [Fact] → atributo de test.

Línea 42: public void InsertarOrdenConNumeroDuplicado_LanzaExcepcion() → declara el test.

Línea 44: Context OrdenesFabricacion.Add(new OrdenFabricacion { ... }); → añade la primera orden.

Línea 45: Context.SaveChanges(); → guarda los cambios.

Línea 47: Context OrdenesFabricacion.Add(new OrdenFabricacion { ... }); → añade la segunda orden con el mismo número.

Línea 49: var excepcion = Record.Exception(() => Context.SaveChanges()); → captura la excepción.

Línea 51: Assert.NotNull(excepcion); → comprueba que hay excepción.

Error común: el test InsertarPlanchaConOrdenInvalida_LanzaExcepcion pasa con SQLite en memoria porque la restricción de integridad referencial se aplica. En InMemory, este test fallaría porque la restricción no se aplica.

Paso 10: Ejecutar todos los tests

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

dotnet test → ejecuta todos los tests.

Resultado esperado: todos los tests pasan. Los tests con SQLite en memoria aplican las restricciones de integridad referencial y de unicidad.

Paso 11: Analizar la salida

La salida del comando muestra información como la siguiente:

```
Passed! - Failed: 0, Passed: 10, Skipped: 0, Total: 10, Duration: 1 s
La primera línea muestra el resultado. Todos los tests pasan.
```

Observaciones: los tests con SQLite en memoria aplican las restricciones de integridad referencial y de unicidad. Los tests con InMemory no las aplican. La elección del proveedor depende del tipo de prueba.

Paso 12: Diagnosticar un error común

Modificar el test InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion para ejecutarlo con SQLite en memoria:

```
[Fact]
public void InsertarPlanchaConOrdenInvalida_ConSqlite_LanzaExcepcion()
{
    var plancha = new PlanchaAcero { OrdenId = 99999, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso
= 370.5m, Activa = true };
    Context.PlanchasAcero.Add(plancha);

    var excepcion = Record.Exception(() => Context.SaveChanges());

    Assert.NotNull(excepcion);
}
```

Resultado esperado: con SQLite en memoria, el test pasa porque la restricción de integridad referencial se aplica. Con InMemory, el test fallaría.

Solución: usar SQLite en memoria para las pruebas que dependen de las restricciones de la base de datos. Usar InMemory para las pruebas que no dependen de las restricciones.

Resultado esperado con la solución: cada prueba usa el proveedor adecuado según lo que verifica.

Errores comunes del ejercicio

Error Causa Solución Contexto no creado Falta EnsureCreated Llamar a EnsureCreated en el constructor
 Conexión cerrada La conexión SQLite se cierra antes de la prueba Mantener la conexión abierta durante la prueba
 Datos compartidos entre pruebas Se usa la misma base de datos en todas las pruebas Usar un nombre único o una conexión por prueba
 InMemory no aplica restricciones El proveedor no simula SQL Usar SQLite en memoria para pruebas realistas
 Logger no inyectado Falta ILogger en el constructor Inyectar el logger o usar NullLogger
 Tests dependientes Se reutiliza el mismo contexto Crear un contexto por prueba

Reto resuelto: Test de concurrencia con SQLite en memoria

Reto: Escribir un test que verifique el comportamiento de la concurrencia optimista con SQLite en memoria. El test debe cargar la misma orden en dos contextos, modificar el cliente en uno y el estado en el otro, y verificar que el segundo SaveChanges lanza DbUpdateConcurrencyException.

Solución paso a paso:

Paso 1: Crear el test en tests/AceriaData.Tests/Sqlite/ConcurrenciaSqliteTests.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Tests.Sqlite;

public class ConcurrenciaSqliteTests : IDisposable
{
    private readonly SqliteConnection _connection;

    public ConcurrenciaSqliteTests()
    {
        _connection = new SqliteConnection("Data Source=:memory:");
        _connection.Open();

        using var context = CrearContexto();
        context.Database.EnsureCreated();

        context.OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente =
"Constructora del Norte", Estado = "Pendiente", FechaCreacion = DateTime.Now });
        context.SaveChanges();
    }

    private AceriaDbContext CrearContexto()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlite(_connection)
            .Options;
        return new AceriaDbContext(options);
    }

    [Fact]
    public void ActualizarOrden_ConDosContextos_LanzaExcepcionDeConcurrencia()
    {
        using var contextA = CrearContexto();
        using var contextB = CrearContexto();

        var ordenA = contextA.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");
        var ordenB = contextB.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");

        ordenA.Cliente = "Constructora del Norte Actualizada";
        contextA.SaveChanges();

        ordenB.Estado = "EnProceso";

        var excepcion = Record.Exception(() => contextB.SaveChanges());

        Assert.NotNull(excepcion);
        Assert.IsType<DbUpdateConcurrencyException>(excepcion);
    }

    public void Dispose()
    {
        _connection.Dispose();
    }
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 3: using Microsoft.Data.Sqlite; → importa SQLite.

Línea 4: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 6: namespace AceriaData.Tests.Sqlite; → declara el espacio de nombres.

Línea 8: `public class ConcurrenciaSqliteTests : IDisposable` → declara la clase de tests.

Línea 10: `private readonly SqlConnection _connection;` → campo de la conexión.

Línea 12: `public ConcurrenciaSqliteTests()` → constructor.

Línea 14: `_connection = new SqlConnection("Data Source=:memory:");` → crea la conexión.

Línea 15: `_connection.Open();` → abre la conexión.

Línea 17: `using var context = CrearContexto();` → crea el contexto.

Línea 18: `context.Database.EnsureCreated();` → crea la base de datos.

Línea 20: `context OrdenesFabricacion.Add(new OrdenFabricacion { ... });` → añade la orden.

Línea 21: `context.SaveChanges();` → guarda los cambios.

Línea 24: `private AceriaDbContext CrearContexto()` → declara el método.

Línea 26: `var options = new DbContextOptionsBuilder()` → crea las opciones.

Línea 27: `.UseSqlite(_connection)` → configura SQLite.

Línea 28: `.Options;` → obtiene las opciones.

Línea 29: `return new AceriaDbContext(options);` → devuelve el contexto.

Línea 32: `[Fact]` → atributo de test.

Línea 33: `public void ActualizarOrden_ConDosContextos_LanzaExcepcionDeConcurrencia()` → declara el test.

Línea 35: `using var contextA = CrearContexto();` → crea el contexto A.

Línea 36: `using var contextB = CrearContexto();` → crea el contexto B.

Línea 38: `var ordenA = contextA.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");` → carga la orden en A.

Línea 39: `var ordenB = contextB.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");` → carga la orden en B.

Línea 41: `ordenA.Cliente = "Constructora del Norte Actualizada";` → modifica el cliente en A.

Línea 42: `contextA.SaveChanges();` → guarda los cambios de A.

Línea 44: `ordenB.Estado = "EnProceso";` → modifica el estado en B.

Línea 46: `var excepcion = Record.Exception(() => contextB.SaveChanges());` → captura la excepción.

Línea 48: `Assert.NotNull(excepcion);` → comprueba que hay excepción.

Línea 49: `Assert.IsType(excepcion);` → comprueba el tipo.

Línea 52: `public void Dispose()` → declara el método Dispose.

Línea 54: `_connection.Dispose();` → libera la conexión.

| Paso 2: Ejecutar el test:

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj --filter "ConcurrenciaSqliteTests"
```

Resultado esperado: el test pasa y verifica que EF Core lanza `DbUpdateConcurrencyException` cuando dos contextos modifican la misma orden.

Ampliación técnica obligatoria — estrategia de testing en cuatro niveles

La fuente original cubre xUnit, EF Core InMemory y SQLite en memoria. El contrato del curso amplía el punto para separar pruebas unitarias de Application, dobles de proveedor y pruebas reales contra SQL Server.

Paso 13: Añadir unit tests de Application con xUnit + Moq, sin EF Core

Mockea las abstracciones del repositorio, no DbSet para consultas LINQ.

```
var repo = new Mock<IOrdenRepositorio>();
repo.Setup(r => r.ObtenerPorNumero("OF-UNIT-001"))
    .Returns(new OrdenFabricacion
    {
        NumeroOrden = "OF-UNIT-001",
        Cliente = "Cliente unitario",
        Estado = "Pendiente"
    });

var sut = new ConsultarOrdenUseCase(repo.Object);
var resultado = sut.Ejecutar("OF-UNIT-001");

Assert.Equal("Cliente unitario", resultado.Cliente);
repo.Verify(r => r.ObtenerPorNumero("OF-UNIT-001"), Times.Once);
```

Este nivel no referencia Microsoft.EntityFrameworkCore desde Application.

Paso 14: Mantener InMemory y SQLite como dobles con limitaciones explícitas

- InMemory no es una base relacional y no reproduce transacciones, SQL ni semántica del proveedor real.
- SQLite en memoria permite pruebas relacionales rápidas, pero su dialecto, funciones, tipos, collation y comportamiento difieren de SQL Server.
- No uses SQLite para afirmar que el rowversion autogenerado de SQL Server funciona igual.

Paso 15: Crear una DatabaseFixture contra SQL Server LocalDB real

```
public sealed class SqlServerDatabaseFixture : IAsyncLifetime
{
    private readonly string _dbName = $"AceriaData_Test_{Guid.NewGuid():N}";
    public string ConnectionString =>
        $"Server=(localdb)\
\MSSQLLocalDB;Database={_dbName};Trusted_Connection=True;TrustServerCertificate=True";

    public async Task InitializeAsync()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer(ConnectionString)
            .Options;

        await using var context = new AceriaDbContext(options);
        await context.Database.MigrateAsync();
    }

    public async Task DisposeAsync()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer(ConnectionString)
            .Options;

        await using var context = new AceriaDbContext(options);
        await context.Database.EnsureDeletedAsync();
    }
}
```

La fixture crea una base aislada y aplica **las migraciones reales** antes de ejecutar los tests.

Paso 16: Usar Respawn para restaurar los datos entre tests

Respawn no sustituye las migraciones: se usa después de preparar el esquema.

```
await using var connection = new SqlConnection(fixture.ConnectionString);
await connection.OpenAsync();

var checkpoint = await Respawner.CreateAsync(connection, new RespawnerOptions
{
    DbAdapter = DbAdapter.SqlServer,
    TablesToIgnore = new Table[] { "__EFMigrationsHistory" }
});

await checkpoint.ResetAsync(connection);
```

Tras el reset, comprueba que el historial de migraciones se conserva y que los datos del test anterior han desaparecido.

Paso 17: Añadir un host `AceriaData.Api` mínimo y probarlo con `WebApplicationFactory<Program>`

El host API puede añadirse sin mover responsabilidades de Domain/Application/Infrastructure.

```
public class ApiIntegrationTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public ApiIntegrationTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task Get0rdenes_DevuelveSuccess()
    {
        var response = await _client.GetAsync("/api/ordenes");
        response.EnsureSuccessStatusCode();
    }
}
```

La configuración de pruebas debe sustituir la cadena de conexión por la base LocalDB aislada de la fixture.

Paso 18: Ejecutar toda la matriz de pruebas

```
dotnet test AceriaData.sln --configuration Release
```

El CI debe fallar si falla cualquiera de estos niveles:

1. Unit: xUnit + Moq, sin EF Core.
2. Provider-double: InMemory / SQLite con limitaciones documentadas.
3. Integration DB: SQL Server LocalDB + migraciones + DatabaseFixture + Respawn.
4. Integration HTTP: WebApplicationFactory + API + base SQL Server de pruebas.

Analogía final

El testing con EF Core en una acería es como las pruebas de calidad de las planchas antes de enviarlas al cliente. El proveedor InMemory es como una prueba rápida que verifica que la plancha tiene el tamaño correcto pero no comprueba la resistencia. SQLite en memoria es como una prueba completa que verifica la resistencia, la composición y las tolerancias. Las pruebas con InMemory son más rápidas

pero menos fiables. Las pruebas con SQLite en memoria son más lentas pero más realistas. La elección depende de lo que se quiere verificar. Las pruebas se ejecutan en cada cambio para detectar errores antes de que lleguen al cliente. Así funciona el testing con EF Core: se elige el proveedor adecuado según lo que se verifica y se ejecutan las pruebas de forma continua.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado el proyecto `AceriaData.Tests` con xUnit.

Añadido las referencias a los proyectos.

Añadido los paquetes de EF Core para pruebas.

Creado la clase base `TestBase`.

Creado los tests del repositorio de órdenes.

Creado los tests de la unidad de trabajo.

Creado los tests con `InMemory`.

Creado los tests con SQLite en memoria.

Ejecutado todos los tests.

Diagnosticado el error de la conexión cerrada.

Creado el test de concurrencia con SQLite en memoria.

| Conexión con el siguiente punto

En este punto se ha profundizado en el testing con EF Core, incluyendo la configuración del proyecto de pruebas, el uso del proveedor `InMemory` y SQLite en memoria, y la escritura de tests que verifican el comportamiento de la capa de persistencia. Se ha comprobado que `InMemory` es más rápido pero menos realista, mientras que SQLite en memoria aplica las restricciones de integridad referencial y de unicidad. En el siguiente punto se estudiarán las buenas prácticas y anti-patronos en persistencia empresarial, cerrando el Módulo 5.

Punto 5.12 — Buenas prácticas y anti-patrones en persistencia empresarial

Práctica

Ejercicio: Revisar el código del proyecto AceriaData para aplicar las buenas prácticas y eliminar los anti-patrones. Crear un caso de uso que documente las buenas prácticas y los anti-patrones. Aplicar las correcciones necesarias.

Contexto del proyecto: En el punto 5.11 se profundizó en el testing con EF Core, incluyendo el uso de InMemory y SQLite en memoria. En este punto se consolidan las buenas prácticas y los anti-patrones, cerrando el Módulo 5. Este punto es el cierre del curso.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el caso de uso de buenas prácticas y anti-patrones

Crear el archivo src/AceriaData.Application/UseCases/BuenasPracticasAntiPatronesUseCase.cs:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class BuenasPracticasAntiPatronesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public BuenasPracticasAntiPatronesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== BUENAS PRÁCTICAS Y ANTI-PATRONES ===");

        MostrarBuenasPracticasCicloDeVida();
        MostrarBuenasPracticasModelado();
        MostrarBuenasPracticasConsultas();
        MostrarBuenasPracticasEscritura();
        MostrarBuenasPracticasMigraciones();
        MostrarBuenasPracticasTesting();
        MostrarAntiPatronesHabituales();
    }

    private void MostrarBuenasPracticasCicloDeVida()
    {
        Console.WriteLine("\n--- Ciclo de vida del DbContext ---");
        Console.WriteLine("1. Crear el DbContext por unidad de trabajo.");
        Console.WriteLine("2. Liberar el DbContext con using.");
        Console.WriteLine("3. No compartir el DbContext entre hilos.");
        Console.WriteLine("4. No mantener el DbContext vivo durante toda la aplicación.");
    }

    private void MostrarBuenasPracticasModelado()
    {
        Console.WriteLine("\n--- Modelado y configuración ---");
        Console.WriteLine("1. Usar Fluent API para la configuración.");
        Console.WriteLine("2. Configurar claves, índices y restricciones explícitamente.");
    }
}
```

```

        Console.WriteLine("3. Configurar longitudes máximas y precisión decimal.");
        Console.WriteLine("4. Usar filtros globales para Soft Delete.");
    }

    private void MostrarBuenasPracticasConsultas()
    {
        Console.WriteLine("\n--- Consultas y carga de datos ---");
        Console.WriteLine("1. Usar proyecciones para reducir el volumen de datos.");
        Console.WriteLine("2. Usar AsNoTracking en consultas de solo lectura.");
        Console.WriteLine("3. Usar Include para cargar entidades relacionadas.");
        Console.WriteLine("4. Usar AsSplitQuery cuando se incluyen varias colecciones.");
        Console.WriteLine("5. Aplicar filtros y paginación en el servidor.");
        Console.WriteLine("6. Evitar funciones en Where.");
        Console.WriteLine("7. Evitar métodos personalizados en Where.");
    }

    private void MostrarBuenasPracticasEscritura()
    {
        Console.WriteLine("\n--- Escritura y transacciones ---");
        Console.WriteLine("1. Agrupar operaciones en una unidad de trabajo.");
        Console.WriteLine("2. Usar transacciones explícitas cuando sea necesario.");
        Console.WriteLine("3. Mantener las transacciones cortas.");
        Console.WriteLine("4. Gestionar los conflictos de concurrencia.");
    }

    private void MostrarBuenasPracticasMigraciones()
    {
        Console.WriteLine("\n--- Migraciones y despliegue ---");
        Console.WriteLine("1. Generar migraciones con nombres descriptivos.");
        Console.WriteLine("2. No modificar migraciones ya aplicadas.");
        Console.WriteLine("3. Usar scripts idempotentes en producción.");
        Console.WriteLine("4. Hacer copias de seguridad antes de aplicar.");
        Console.WriteLine("5. Preparar planes de reversión.");
    }

    private void MostrarBuenasPracticasTesting()
    {
        Console.WriteLine("\n--- Testing y diagnóstico ---");
        Console.WriteLine("1. Usar SQLite en memoria para tests realistas.");
        Console.WriteLine("2. Usar InMemory para tests rápidos.");
        Console.WriteLine("3. Configurar logging con ILogger o Serilog.");
        Console.WriteLine("4. Usar observadores de diagnóstico para detectar consultas lentas.");
    }

    private void MostrarAntiPatronesHabituales()
    {
        Console.WriteLine("\n--- Anti-patrones habituales ---");
        Console.WriteLine("1. DbContext estático compartido.");
        Console.WriteLine("2. Repositorio genérico excesivo.");
        Console.WriteLine("3. Exponer IQueryable.");
        Console.WriteLine("4. Problema N+1.");
        Console.WriteLine("5. Over-fetching.");
        Console.WriteLine("6. Carga Lazy sin control.");
        Console.WriteLine("7. Materialización prematura.");
        Console.WriteLine("8. Funciones en Where.");
        Console.WriteLine("9. Métodos personalizados en Where.");
        Console.WriteLine("10. Transacciones largas.");
        Console.WriteLine("11. Migraciones modificadas.");
        Console.WriteLine("12. Tests que siempre pasan.");
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class BuenasPracticasAntiPatronesUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public BuenasPracticasAntiPatronesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: `_unidad = unidad;` → asigna el parámetro al campo.

Línea 14: `public void Ejecutar()` → declara el método principal.

Línea 16: `Console.WriteLine("=== BUENAS PRÁCTICAS Y ANTI-PATRONES ===");` → muestra la cabecera.

Línea 18: `MostrarBuenasPracticasCicloDeVida();` → llama al método del ciclo de vida.

Línea 19: `MostrarBuenasPracticasModelado();` → llama al método de modelado.

Línea 20: `MostrarBuenasPracticasConsultas();` → llama al método de consultas.

Línea 21: `MostrarBuenasPracticasEscritura();` → llama al método de escritura.

Línea 22: `MostrarBuenasPracticasMigraciones();` → llama al método de migraciones.

Línea 23: `MostrarBuenasPracticasTesting();` → llama al método de testing.

Línea 24: `MostrarAntiPatronesHabituales();` → llama al método de anti-patrones.

Línea 27: `private void MostrarBuenasPracticasCicloDeVida()` → declara el método.

Línea 29: `Console.WriteLine("\n--- Ciclo de vida del DbContext ---");` → muestra la cabecera.

Línea 30: `Console.WriteLine("1. Crear el DbContext por unidad de trabajo.");` → describe la primera práctica.

Línea 31: `Console.WriteLine("2. Liberar el DbContext con using.");` → describe la segunda práctica.

Línea 32: `Console.WriteLine("3. No compartir el DbContext entre hilos.");` → describe la tercera práctica.

Línea 33: `Console.WriteLine("4. No mantener el DbContext vivo durante toda la aplicación.");` → describe la cuarta práctica.

Línea 36: `private void MostrarBuenasPracticasModelado()` → declara el método.

Línea 38: `Console.WriteLine("\n--- Modelado y configuración ---");` → muestra la cabecera.

Línea 39: `Console.WriteLine("1. Usar Fluent API para la configuración.");` → describe la primera práctica.

Línea 40: `Console.WriteLine("2. Configurar claves, índices y restricciones explícitamente.");` → describe la segunda práctica.

Línea 41: `Console.WriteLine("3. Configurar longitudes máximas y precisión decimal.");` → describe la tercera práctica.

Línea 42: `Console.WriteLine("4. Usar filtros globales para Soft Delete.");` → describe la cuarta práctica.

Línea 45: `private void MostrarBuenasPracticasConsultas()` → declara el método.

Línea 47: `Console.WriteLine("\n--- Consultas y carga de datos ---");` → muestra la cabecera.

Línea 48: `Console.WriteLine("1. Usar proyecciones para reducir el volumen de datos.");` → describe la primera práctica.

Línea 49: `Console.WriteLine("2. Usar AsNoTracking en consultas de solo lectura.");` → describe la segunda práctica.

Línea 50: `Console.WriteLine("3. Usar Include para cargar entidades relacionadas.");` → describe la tercera práctica.

Línea 51: `Console.WriteLine("4. Usar AsSplitQuery cuando se incluyen varias colecciones.");` → describe la cuarta práctica.

Línea 52: `Console.WriteLine("5. Aplicar filtros y paginación en el servidor.");` → describe la quinta práctica.

Línea 53: `Console.WriteLine("6. Evitar funciones en Where.");` → describe la sexta práctica.

Línea 54: `Console.WriteLine("7. Evitar métodos personalizados en Where.");` → describe la séptima práctica.

Línea 57: `private void MostrarBuenasPracticasEscritura()` → declara el método.

Línea 59: `Console.WriteLine("\n--- Escritura y transacciones ---");` → muestra la cabecera.

Línea 60: `Console.WriteLine("1. Agrupar operaciones en una unidad de trabajo.");` → describe la primera práctica.

Línea 61: `Console.WriteLine("2. Usar transacciones explícitas cuando sea necesario.");` → describe la segunda práctica.

Línea 62: `Console.WriteLine("3. Mantener las transacciones cortas.");` → describe la tercera práctica.

Línea 63: `Console.WriteLine("4. Gestionar los conflictos de concurrencia.");` → describe la cuarta práctica.

Línea 66: `private void MostrarBuenasPracticasMigraciones()` → declara el método.

Línea 68: `Console.WriteLine("\n--- Migraciones y despliegue ---");` → muestra la cabecera.

Línea 69: `Console.WriteLine("1. Generar migraciones con nombres descriptivos.");` → describe la primera práctica.

Línea 70: `Console.WriteLine("2. No modificar migraciones ya aplicadas.");` → describe la segunda práctica.

Línea 71: `Console.WriteLine("3. Usar scripts idempotentes en producción.");` → describe la tercera práctica.

Línea 72: `Console.WriteLine("4. Hacer copias de seguridad antes de aplicar.");` → describe la cuarta práctica.

Línea 73: `Console.WriteLine("5. Preparar planes de reversión.");` → describe la quinta práctica.

Línea 76: `private void MostrarBuenasPracticasTesting()` → declara el método.

Línea 78: `Console.WriteLine("\n--- Testing y diagnóstico ---");` → muestra la cabecera.

Línea 79: `Console.WriteLine("1. Usar SQLite en memoria para tests realistas.");` → describe la primera práctica.

Línea 80: `Console.WriteLine("2. Usar InMemory para tests rápidos.");` → describe la segunda práctica.

Línea 81: `Console.WriteLine("3. Configurar logging con ILogger o Serilog.");` → describe la tercera práctica.

Línea 82: `Console.WriteLine("4. Usar observadores de diagnóstico para detectar consultas lentas.");` → describe la cuarta práctica.

Línea 85: `private void MostrarAntiPatronesHabituales()` → declara el método.

Línea 87: `Console.WriteLine("\n--- Anti-patrones habituales ---");` → muestra la cabecera.

Línea 88: `Console.WriteLine("1. DbContext estático compartido.");` → describe el primer anti-patrón.

Línea 89: `Console.WriteLine("2. Repositorio genérico excesivo.");` → describe el segundo anti-patrón.

Línea 90: `Console.WriteLine("3. Exponer IQueryable.");` → describe el tercer anti-patrón.

Línea 91: `Console.WriteLine("4. Problema N+1.");` → describe el cuarto anti-patrón.

Línea 92: `Console.WriteLine("5. Over-fetching.");` → describe el quinto anti-patrón.

Línea 93: `Console.WriteLine("6. Carga Lazy sin control.");` → describe el sexto anti-patrón.

Línea 94: `Console.WriteLine("7. Materialización prematura.");` → describe el séptimo anti-patrón.

Línea 95: `Console.WriteLine("8. Funciones en Where.");` → describe el octavo anti-patrón.

Línea 96: `Console.WriteLine("9. Métodos personalizados en Where.");` → describe el noveno anti-patrón.

Línea 97: `Console.WriteLine("10. Transacciones largas.");` → describe el décimo anti-patrón.

Línea 98: `Console.WriteLine("11. Migraciones modificadas.");` → describe el undécimo anti-patrón.

Línea 99: `Console.WriteLine("12. Tests que siempre pasan.");` → describe el duodécimo anti-patrón.

Error común: si el caso de uso no documenta los anti-patrones, los desarrolladores pueden repetirlos. Se deben documentar todos.

Paso 3: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<BuenasPracticasAntiPatronesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

Paso 4: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasAntiPatronesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 5: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con las buenas prácticas y los anti-patrones.

Paso 6: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== BUENAS PRÁCTICAS Y ANTI-PATRONES ===

--- Ciclo de vida del DbContext ---
1. Crear el DbContext por unidad de trabajo.
2. Liberar el DbContext con using.
3. No compartir el DbContext entre hilos.
4. No mantener el DbContext vivo durante toda la aplicación.

--- Modelado y configuración ---
1. Usar Fluent API para la configuración.
2. Configurar claves, índices y restricciones explícitamente.
3. Configurar longitudes máximas y precisión decimal.
```

4. Usar filtros globales para Soft Delete.

--- Consultas y carga de datos ---

1. Usar proyecciones para reducir el volumen de datos.
2. Usar AsNoTracking en consultas de solo lectura.
3. Usar Include para cargar entidades relacionadas.
4. Usar AsSplitQuery cuando se incluyen varias colecciones.
5. Aplicar filtros y paginación en el servidor.
6. Evitar funciones en Where.
7. Evitar métodos personalizados en Where.

--- Escritura y transacciones ---

1. Agrupar operaciones en una unidad de trabajo.
2. Usar transacciones explícitas cuando sea necesario.
3. Mantener las transacciones cortas.
4. Gestionar los conflictos de concurrencia.

--- Migraciones y despliegue ---

1. Generar migraciones con nombres descriptivos.
2. No modificar migraciones ya aplicadas.
3. Usar scripts idempotentes en producción.
4. Hacer copias de seguridad antes de aplicar.
5. Preparar planes de reversión.

--- Testing y diagnóstico ---

1. Usar SQLite en memoria para tests realistas.
2. Usar InMemory para tests rápidos.
3. Configurar logging con ILogger o Serilog.
4. Usar observadores de diagnóstico para detectar consultas lentas.

--- Anti-patrones habituales ---

1. DbContext estático compartido.
 2. Repositorio genérico excesivo.
 3. Exponer IQueryable.
 4. Problema N+1.
 5. Over-fetching.
 6. Carga Lazy sin control.
 7. Materialización prematura.
 8. Funciones en Where.
 9. Métodos personalizados en Where.
 10. Transacciones largas.
 11. Migraciones modificadas.
 12. Tests que siempre pasan.
- La salida muestra las buenas prácticas y los anti-patrones agrupados por categoría.

Observaciones: las buenas prácticas cubren todos los aspectos del acceso a datos. Los anti-patrones son los errores más habituales que se deben evitar.

Paso 7: Diagnosticar un error común

Modificar el caso de uso para usar un DbContext estático compartido:

```
public class BuenasPracticasAntiPatronesUseCase
{
    private static readonly AceriaDbContext _contextCompartido = new AceriaDbContext();
    // ...
}
```

Resultado esperado: el código compila pero el DbContext compartido acumula entidades en el Change Tracker y consume memoria. Además, no es seguro para subprocesos.

Solución: usar la unidad de trabajo inyectada por el contenedor.

```
public class BuenasPracticasAntiPatronesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public BuenasPracticasAntiPatronesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }
}
```

Resultado esperado con la solución: el DbContext se crea por ámbito y se libera al final.

Errores comunes del ejercicio

Error Causa Solución DbContext estático Se comparte entre toda la aplicación Usar contexto por unidad de trabajo Repositorio expone IQueryable Rompe la abstracción Devolver listas N+1 No se usa Include Usar Include Over-fetching Se cargan entidades completas Usar proyecciones Carga Lazy sin control Se accede a propiedades de navegación en bucle Usar carga Eager Materialización prematura Se llama a ToList antes de filtrar Filtrar antes de materializar Funciones en Where Se aplican funciones sobre columnas Comparar directamente Métodos personalizados en Where No se traducen a SQL Reescribir con expresiones traducibles Transacciones largas Se mantienen abiertas durante mucho tiempo Mantener cortas Migraciones modificadas Se modifican migraciones aplicadas Generar una nueva migración Tests que siempre pasan No verifican nada Verificar el comportamiento real

Reto resuelto: Auditoría de buenas prácticas en el repositorio

Reto: Crear un método en el repositorio que audite las consultas del proyecto y reporte cuáles usan buenas prácticas y cuáles no. El método debe devolver una lista de hallazgos.

Solución paso a paso:

Paso 1: Crear el DTO HallazgoAuditoriaDto:

```
namespace AceriaData.Application.Dtos;

public class HallazgoAuditoriaDto
{
    public string Metodo { get; set; } = string.Empty;
    public string Practica { get; set; } = string.Empty;
    public string Recomendacion { get; set; } = string.Empty;
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class HallazgoAuditoriaDto → declara el DTO.

Línea 5: public string Metodo { get; set; } = string.Empty; → nombre del método.

Línea 6: public string Practica { get; set; } = string.Empty; → práctica evaluada.

Línea 7: public string Recomendacion { get; set; } = string.Empty; → recomendación.

Paso 2: Añadir el método a la interfaz IOrdenRepositorio:

```
List<HallazgoAuditoriaDto> AuditarBuenasPracticas();
```

Línea 1: List AuditarBuenasPracticas(); → declara el método.

Paso 3: Implementar el método en OrdenRepositorio:

```
public List<HallazgoAuditoriaDto> AuditarBuenasPracticas()
{
    return new List<HallazgoAuditoriaDto>
    {
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerResumenes",
            Practica = "Usa AsNoTracking y proyección",
            Recomendacion = "Correcto"
        },
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerConPlanchasInclude",
            Practica = "Usa Include y AsNoTracking",
            Recomendacion = "Correcto"
        },
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerConPlanchasYDetalleSplitQuery",
            Practica = "Usa AsSplitQuery",
            Recomendacion = "Correcto"
        },
        new HallazgoAuditoriaDto
        {
            Metodo = "ObtenerPorIdParaActualizar",
            Practica = "Usa tracking para modificar",
            Recomendacion = "Correcto"
        }
    };
}
```

Línea 1: public List AuditarBuenasPracticas() → declara el método.

Línea 3: return new List → crea la lista.

Línea 5: new HallazgoAuditoriaDto → crea el primer hallazgo.

Línea 7: Metodo = "ObtenerResumenes", → asigna el método.

Línea 8: Practica = "Usa AsNoTracking y proyección", → asigna la práctica.

Línea 9: Recomendacion = "Correcto" → asigna la recomendación.

Línea 11: new HallazgoAuditoriaDto → crea el segundo hallazgo.

Línea 17: new HallazgoAuditoriaDto → crea el tercer hallazgo.

Línea 23: new HallazgoAuditoriaDto → crea el cuarto hallazgo.

Paso 4: Añadir la demostración en el caso de uso:

```
private void DemostrarAuditoria()
{
    Console.WriteLine("\n--- Auditoría de buenas prácticas ---");

    var hallazgos = _unidad Ordenes.AuditarBuenasPracticas();
    foreach (var hallazgo in hallazgos)
    {
        Console.WriteLine($"Método: {hallazgo.Metodo} | Práctica: {hallazgo.Practica} | Recomendación: {hallazgo.Recomendacion}");
    }
}
```

Línea 1: private void DemostrarAuditoria() → declara el método.

Línea 3: Console.WriteLine("\n--- Auditoría de buenas prácticas ---"); → muestra la cabecera.

Línea 5: `var hallazgos = _unidad Ordenes.AuditarBuenasPracticas();` → llama al método del repositorio.

Línea 6: `foreach (var hallazgo in hallazgos)` → itera sobre los hallazgos.

Línea 8: `Console.WriteLine($"Método: {hallazgo.Metodo} | Práctica: {hallazgo.Practica} | Recomendación: {hallazgo.Recomendacion}");` → muestra el hallazgo.

Paso 5: Llamar al método desde Ejecutar:

```
DemostrarAuditoria();
```

Paso 6: Ejecutar dotnet run y verificar que los hallazgos se muestran.

Resultado esperado: se muestra la auditoría de buenas prácticas con los métodos y sus recomendaciones.

Ampliación técnica obligatoria — refactorización ejecutable de N+1

La lista de buenas prácticas no basta. El cierre del módulo debe demostrar al menos un anti-patrón completo, su evidencia y dos refactorizaciones.

Paso 8: Reproducir un N+1 intencionado

Carga las órdenes y ejecuta una consulta adicional por orden para contar sus planchas. Reinicia el interceptor antes del escenario y conserva el resultado funcional para compararlo después.

```
SqlCommandCounterInterceptor.Instance.Reset();

var ordenes = context.OrdenesFabricacion
    .AsNoTracking()
    .OrderBy(o => o.Id)
    .ToList();

var resultadoN1 = ordenes.Select(o => new
{
    o.NumeroOrden,
    TotalPlanchas = context.PlanchasAcero.Count(p => p.OrdenId == o.Id)
}).ToList();

var comandosN1 = SqlCommandCounterInterceptor.Instance.Count;
```

Paso 9: Primera corrección con carga anticipada cuando necesitas el grafo

```
SqlCommandCounterInterceptor.Instance.Reset();

var conInclude = context.OrdenesFabricacion
    .AsNoTracking()
    .Include(o => o.Planchas)
    .OrderBy(o => o.Id)
    .ToList()
    .Select(o => new
    {
        o.NumeroOrden,
        TotalPlanchas = o.Planchas.Count
    })
    .ToList();

var comandosInclude = SqlCommandCounterInterceptor.Instance.Count;
```

`Include` es una posible corrección cuando el grafo completo es necesario; no se presenta como solución universal.

Paso 10: Segunda mejora con proyección

```
SqlCommandCounterInterceptor.Instance.Reset();

var proyectado = context.OrdenesFabricacion
    .AsNoTracking()
    .OrderBy(o => o.Id)
    .Select(o => new
    {
        o.NumeroOrden,
        TotalPlanchas = o.Planchas.Count()
    })
    .ToList();

var comandosProyeccion = SqlCommandCounterInterceptor.Instance.Count;
```

La proyección evita materializar el grafo completo cuando solo se necesita el total.

Paso 11: Verificar equivalencia funcional y evidencia técnica

```
Assert.Equal(
    resultadoN1.Select(x => (x.NumeroOrden, x.TotalPlanchas)),
    conInclude.Select(x => (x.NumeroOrden, x.TotalPlanchas)));

Assert.Equal(
    resultadoN1.Select(x => (x.NumeroOrden, x.TotalPlanchas)),
    proyectado.Select(x => (x.NumeroOrden, x.TotalPlanchas)));
```

Registra para cada versión:

- comandos SQL / roundtrips observados;
- entidades rastreadas;
- forma de las columnas materializadas a partir del SQL real;
- resultado funcional.

No fijas porcentajes de mejora universales. La conclusión procede de la evidencia de esa ejecución.

Paso 12: Matriz final de anti-patrones y trade-offs

Anti-patrón	Síntoma	Consecuencia	Evidencia	Refactor	Trade-off
N+1	una consulta inicial + consultas por fila	más roundtrips	interceptor / logs SQL	Include o proyección según necesidad	Include materializa grafo; proyección cambia la forma devuelta
Over-fetching	columnas/ entidades no usadas	más datos materializados	SQL y forma del resultado	proyección	requiere DTO/ forma específica
Tracking innecesario	ChangeTracker crece en lectura	coste de tracking	ChangeTracker.Entries()	AsNoTracking	no usarlo cuando se editarán esas entidades
SplitQuery automático	varias consultas sin justificación	más roundtrips	logs/interceptor	elegir Single/ Split por escenario	Split reduce explosión cartesiana pero aumenta roundtrips

Anti-patrón	Síntoma	Consecuencia	Evidencia	Refactor	Trade-off
Repository como dogma	abstracción sin necesidad	mantenimiento adicional	arquitectura/cobertura	usarlo cuando aporta aislamiento/testabilidad	DbContext ya ofrece UoW/Repository-like capabilities

Paso 13: Reglas que deben quedar matizadas

- Data Annotations no son un anti-patrón por sí mismas; Fluent API ofrece más capacidad y centralización, pero la elección depende del modelo.
- `Include` no es siempre la mejor corrección para N+1; una proyección puede ser más adecuada.
- `AsSplitQuery` no se aplica automáticamente: reduce determinadas explosiones cartesianas a cambio de más roundtrips y consideraciones de consistencia.
- Un método `.NET` no traducible dentro de un `Where` de EF Core 8 normalmente produce un error de traducción salvo que se introduzca explícitamente una frontera cliente.
- Aplicar funciones a columnas puede perjudicar la sargabilidad, pero el uso real de índices debe comprobarse con el proveedor, el índice, la collation y el plan de ejecución.

Analogía final

Las buenas prácticas y los anti-patrones en una acería son como el manual de procedimientos de la planta. Las buenas prácticas son los procedimientos que han demostrado ser seguros y eficientes. Los anti-patrones son los procedimientos que parecen correctos pero que provocan accidentes o pérdidas. El manual se actualiza con la experiencia y se consulta antes de cada operación. Las buenas prácticas del ciclo de vida del DbContext son como las normas de uso de las herramientas: se usan y se devuelven al taller. Las buenas prácticas de modelado son como los planos bien dibujados: se entienden y se mantienen. Las buenas prácticas de consultas son como las órdenes de búsqueda eficientes: se pide solo lo necesario. Las buenas prácticas de escritura son como los lotes de producción: se agrupan y se confirman. Las buenas prácticas de migraciones son como las reformas controladas: se planifican y se revierten si fallan. Las buenas prácticas de testing son como los controles de calidad: se ejecutan antes de enviar. Los anti-patrones son los errores que se deben evitar. Así funcionan las buenas prácticas y los anti-patrones en EF Core: se documentan, se aplican y se revisan.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado el caso de uso BuenasPracticasAntiPatronesUseCase.

Registrado el caso de uso en el contenedor.

Ejecutado el caso de uso y verificado las buenas prácticas y los anti-patrones.

Diagnosticado el error del DbContext estático.

Creado el DTO HallazgoAuditoriaDto y el método AuditarBuenasPracticas.

Resumen del estado del proyecto AceriaData al final del Módulo 5 Al final del Módulo 5, el proyecto AceriaData tiene:

La arquitectura limpia configurada en cuatro proyectos: AceriaData.Domain, AceriaData.Application, AceriaData.Infrastructure y AceriaData.Console.

El dominio con las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.

La aplicación con las interfaces IRepository, IOrdenRepository, IPlanchaRepository, IAleacionRepository, IDetalleOrdenRepository e IUnidadDeTrabajo, los casos de uso de todos los módulos y los DTOs.

La infraestructura con el `AceriaDbContext`, las configuraciones de Fluent API, las implementaciones de repositorios, la unidad de trabajo, las consultas compiladas y el observador de diagnóstico.

La consola con el método `Main` y la configuración del contenedor de dependencias.

El modelo de datos completo con relaciones uno a muchos, uno a uno y muchos a muchos.

Las claves primarias, alternativas y compuestas configuradas.

Los índices y las restricciones configurados.

El `Soft Delete` implementado con filtros globales.

La arquitectura limpia aplicada con la regla de dependencia respetada.

Las consultas LINQ con filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga Eager, carga Lazy, carga Explicit y composición de consultas.

Las buenas prácticas de acceso a datos aplicadas.

La optimización del rendimiento aplicada: análisis del SQL, Tracking vs No Tracking, resolución de identidad, solución al N+1, over-fetching, consultas ineficientes, Split Queries, Compiled Queries, paginación eficiente, diagnóstico y checklist de rendimiento.

La persistencia empresarial aplicada: concurrencia optimista, tokens de concurrencia, resolución de conflictos, transacciones, transacciones ambientales, migraciones en producción, migraciones idempotentes, migraciones en equipos, patrón Repositorio y unidad de trabajo, logging y diagnóstico, testing con EF Core y buenas prácticas.
